

ELMAR
CNC-Cutter

Autoren: Alexandra Roth 7025637
David Atrops 7013979
Dennis Smirnow 7023434
Leon Ronge 7026641
Samuel Drees 7026354

Studiengang: Maschinenbau und Design

Erstprüfer: Prof. Dr. Elmar Wings
Malena Wilken

Abgabedatum: 23. Mai 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	3
Abbildungsverzeichnis	9
I. Software	11
1. Arduino IDE 2.3.x	13
1.1. Beschreibung der Arduino IDE	13
1.2. Installation der Arduino IDE	14
1.2.1. Download der Arduino IDE	14
1.2.2. Zusammenfassung der Optionen	14
1.2.3. Installation der Arduino IDE für Windows	15
1.3. Verknüpfen des Boards mit der IDE	17
1.3.1. Beispielcode „Blinkende LED“	18
1.3.2. Installation der Arduino IDE für MacOS	20
1.3.3. Bibliotheken installieren	25
1.3.4. Board Manager	26
1.3.5. Beispielprogramm „Blink“	28
2. Arduino IDE 2.3.6	31
2.1. Arduino IDE Beschreibung	31
2.2. Installation der Arduino IDE	32
2.2.1. Download der Arduino IDE	32
2.2.2. Anforderungen Betriebssystem	33
2.2.3. Installation auf Windows	33
2.2.4. Installation (MacOS)	36
2.3. Übersicht der Eigenschaften	37
2.3.1. Sketchbook	38
2.3.2. Boards Manager	39
2.3.3. Library Manager	39
2.4. Integration und Verwendung einer benutzerdefinierten Bibliothek in der Arduino IDE	40
2.4.1. Erstellen der Bibliotheksdateien	40
2.4.2. Installation und Integration der Arduino IDE	40
2.4.3. Verwendung symbolischer Verknüpfungen zur Bibliotheksintegration	41
2.4.4. Windows-Tool mklink zum Erstellen symbolischer Verknüpfungen	41
2.4.5. Überprüfen der Verfügbarkeit der Bibliothek in der Arduino IDE	42
2.4.6. Serieller Monitor	43
2.4.7. Serieller Plotter	43
2.5. Beispiele	44
2.6. Debuggen	44
2.7. Autovervollständigung	45
2.8. Bibliotheken und Modularität	46

3. Arduino Web Editor	47
3.1. Using the Arduino Web Editor	47
3.1.1. Using the online IDE	47
3.2. Arduino Libraries	48
3.3. Data Security	49
3.3.1. Authentication and Authorization	49
3.3.2. Secure Code Execution	49
3.3.3. Protection Against Malware	49
3.3.4. Privacy Policies	49
3.3.5. Regular Updates and Maintenance	50
3.3.6. User Awareness	50
 II. Hardware	 51
4. Arduino Uno	53
4.1. Arduino Allgemein	53
4.2. Arduino Uno	53
4.3. Module	54
4.3.1. Mikrocontroller ATmega328P	54
4.3.2. Eingebaute LED	54
4.3.3. Reset-Taster	55
4.3.4. USB-Schnittstelle	55
4.3.5. Spannungsversorgung	55
4.4. Testtaster	56
4.4.1. Hintergrundwissen	56
4.4.2. Beschreibung	56
4.4.3. Eigenschaften	56
4.4.4. Testcode	57
4.5. PINs	58
4.5.1. Hintergrundwissen	58
4.5.2. Digitale Pins	58
4.5.3. PWM-Pins	59
4.5.4. Analoge Eingänge	59
4.5.5. Versorgungspins	60
4.5.6. Serielle Pins D0 und D1	60
4.5.7. I2C-Pins	61
4.5.8. SPI-Pins	61
4.5.9. AREF-Pin	62
4.5.10. RESET-Pin	62
4.6. Beispiel Code	63
4.6.1. Manual	63
4.6.2. Code	63
 5. CNCShieldV3	 65
5.1. CNC Shields	65
5.2. CNCShieldV3	65
5.3. Specification	65
5.4. Library	65
5.4.1. Description	66
5.4.2. Installation	66
5.4.3. Functions	66
5.5. Example	66
5.5.1. Manual	66
5.5.2. Inside the Example	66

5.5.3. Code	66
5.5.4. Files	66
5.6. Calibration	66
5.7. Simple Code	66
5.7.1. Manual	66
5.7.2. Code	67
5.8. Simple Application	68
5.8.1. Manual	68
5.8.2. Code	69
5.9. Tests	70
5.9.1. Manual	70
5.9.2. Code	71
6. Sensor: Endschalter	73
6.1. Einleitung: Positionssensoren	73
6.2. Definition und Funktion: Elektromechanischer Endschalter	73
6.3. Specification	73
6.4. Verwendung und Schaltfunktionen	74
6.5. Analyse: Vor- und Nachteile	75
6.6. Ausblick: Intelligente und vernetzte Sensorik	76
6.7. Library	76
6.8. Beispiel	76
6.8.1. Manual	76
6.8.2. Erklärung des Beispiel-Codes	76
6.9. Justage	77
6.10. Tests	77
6.10.1. Einfacher Funktions-Test	77
6.10.2. Manual	78
6.10.3. Einfacher Test-Code	78
6.11. Einfache Anwendung	78
6.11.1. Manual	79
6.12. Weiterführende Literatur	80
7. OLED-Display	81
7.1. Allgemein	81
7.2. OLED-Display mit SSD1306 Controller	81
7.2.1. Aufbau und Funktionsweise	82
7.2.2. Anwendung	83
7.3. Installation	83
7.4. Bibliotheken	84
7.5. Verwendete Bibliotheken	84
7.5.1. Wire.h	84
7.5.2. Adafruit_GFX.h	84
7.6. Codebeispiel	85
7.6.1. Kurzbeschreibung des Programms	85
7.6.2. Verwendete Funktionen	85
7.6.3. Globale Variablen und Parameter	85
7.6.4. Testcode	85
7.7. Tests	87
7.7.1. Test: Fortschrittsbalken per Taster	87
7.7.2. Manual	87
7.7.3. Test-Code	87
7.8. Weiterführende Literatur	88

8. Druckknopf	91
8.1. General	91
8.2. Joy IT	91
8.3. Spezifikationen	91
8.3.1. Functions	91
8.4. Beispiel Code	91
8.4.1. Manual	91
8.4.2. Code	92
8.5. Einfacher Testcode	92
8.5.1. Manual	92
8.5.2. Code	93
8.6. Einfache Anwendung	93
8.6.1. Manual	93
8.6.2. Code	94
8.7. Further Readings	96
9. MOSFET PWM-Schaltmodul	97
9.1. Einleitung	97
9.2. Grundlegende Informationen	97
9.3. Spezifisches MOSFET-Modul	97
9.4. Spezifikation	97
9.4.1. Eigenschaften des IRF9540N	98
9.4.2. Anschlüsse	98
9.5. Bibliothek	98
9.5.1. Beschreibung	98
9.5.2. Installation	98
9.5.3. Funktionen	99
9.6. Beispiel	99
9.6.1. Versuchsaufbau	99
9.6.2. Funktionsweise des Beispiels	99
9.6.3. Code	99
9.6.4. Dateien	99
9.7. Kalibrierung	99
9.8. Einfacher Code	100
9.9. Einfache Anwendung	100
9.10. Tests	102
9.10.1. Einfacher Funktionstest	102
9.10.2. Test aller Funktionen	102
9.11. Weiterführende Literatur	102
10. A4988-Schrittmotortreiber	105
10.1. Einleitung	105
10.2. Grundlagen	105
10.3. Spezifischer Sensor/Aktor	105
10.4. Spezifikation	105
10.5. Bibliothek	106
10.5.1. Beschreibung	106
10.5.2. Installation	106
10.5.3. Funktionen	106
10.6. Beispiel	106
10.6.1. Versuchsaufbau	106
10.6.2. Funktionsweise des Beispiels	106
10.6.3. Code	107
10.6.4. Dateien	107
10.7. Kalibrierung	107

Inhaltsverzeichnis	7
10.8. Einfacher Code	107
10.9. Einfache Anwendung	108
10.10Tests	110
10.10.1Einfacher Funktionstest	110
10.10.2.Test aller Funktionen	110
10.11Weiterführende Literatur	112
III. Domain Knowledge - Tools	113
IV. Developmenmt	115
V. Monitoring	117
VI. Verification - Evaluation - Conclusion	119
VII.Anhang	121
11.Bill of Material	123
11.1. Mechanische Komponenten	123
11.2. Elektronische Komponenten	124
11.3. Schrauben und Befestigungen	124
Literaturverzeichnis	125
Literaturverzeichnis	125

Abbildungsverzeichnis

1.1. Menu Button	13
1.2. Arduino IDE 2.3.3 Download	14
1.3. Zusammenfassung der Download-Optionen	15
1.4. Downloadfenster Arduino IDE	15
1.5. Lizenzabkommen	16
1.6. Installationsoption	16
1.7. Zielverzeichnisauswahl	17
1.8. Boardauswahl	18
1.9. Portauswahl	18
1.10. Aufruf der Arduino-Webseite über die Suchmaschine	20
1.11. Ergebnisse der Websuche zur Arduino IDE	20
1.12. Download der Arduino IDE starten	21
1.13. Optional: Spendenaufruf beim Download	21
1.14. Optional: Newsletter abonnieren	22
1.15. Download-Erlaubnis erteilen	22
1.16. Download-Fortschritt einsehen	23
1.17. Heruntergeladene Datei öffnen	23
1.18. Installation durch Ziehen in den Programme-Ordner	24
1.19. Arduino IDE starten	24
1.20. Sicherheitsabfrage bei erstmaligem Start	25
1.21. Bibliotheken-Suchmaske der Arduino IDE	26
1.22. Erfolgreiche Installation einer Bibliothek	26
1.23. Der Board Manager in der Arduino IDE	27
1.24. Board- und Portauswahl öffnen	27
1.25. Passenden Board und Port auswählen	28
1.26. Beispielprogramme öffnen	28
1.27. Das geöffnete Beispielprogramm „Blink“	29
2.1. Arduino IDE 2.3.6	33
2.2. Arduino IDE Downloadoptionen	34
2.3. Lizenzbestätigung Installation	35
2.4. Wahl Zielverzeichnis	35
2.5. Startfenster Arduino-IDE	36
2.6. ArduinoIDE macOS	37
2.7. Überblick	38
2.8. Sketchbook	38
2.9. Boards Manager	39
2.10. Library Manager	40
2.11. Eingabeaufforderung mit Administrator Rechten	41
2.12. Eingabeaufforderung	42
2.13. Einbinden der Nano33BLESenseLED Bibliothek	43
2.14. Serieller Monitor	43
2.15. Beispiele	44
2.16. Beispiel Debuggen	45
2.17. Autovervollständigung	45
3.1. Arduino Web Editor	47

3.2. finding-an-example	48
6.1. Darstellung des im Projekt verwendeten Endschalters	74
6.2. Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO	75
7.1. OLED-Display	82
7.2. I ² C-Bus mit 8 Teilnehmern nach [Schreiter.2019]	83

Teil I.

Software

1. Arduino IDE 2.3.x

Dieses Kapitel bietet eine umfassende Untersuchung der Arduino-IDE und behandelt ihre Merkmale, Schnittstellenoptionen und Funktionen. Es enthält eine detaillierte Erläuterung des Zwecks und der Benutzerfreundlichkeit jeder Komponente, eine schrittweise Installationsanleitung und Anweisungen zur Initialisierung und Konfiguration der Umgebung. Mit diesem grundlegenden Überblick wird der Leser in die Lage versetzt, die IDE effektiv für die Programmierung und Fehlerbehebung von Arduino-kompatibler Hardware zu nutzen.

1.1. Beschreibung der Arduino IDE

Die Arduino IDE ist eine offizielle Open-Source-Software zum Bearbeiten, Kompilieren und Hochladen von Code auf Arduino-Module. Sie ist plattformübergreifend und kompatibel mit Betriebssystemen wie Windows, Linux und macOS. Die IDE basiert auf der Java-Plattform und unterstützt verschiedene Arduino-Module sowie die Programmierung in C und C++.

Die Mikrocontroller auf den Arduino-Boards sind so programmiert, dass sie die in Form von Code bereitgestellten Informationen verarbeiten. Programme, die in der IDE geschrieben werden, werden als Sketches bezeichnet. Diese Skizzen erzeugen eine Hex-Datei, die dann an den Mikrocontroller übertragen und hochgeladen wird.

Die IDE-Umgebung besteht aus zwei Hauptkomponenten: dem Editor und dem Compiler. Der Editor wird zum Schreiben von Code verwendet, während der Compiler den Code kompiliert und in das Arduino-Modul hochlädt.[FAD18] In der Version 2.3.3 der Arduino-IDE spielt die Menüleiste, die sich am oberen Rand der Benutzeroberfläche befindet, eine entscheidende Rolle, da sie Zugriff auf wichtige Befehle wie Hochladen, Überprüfen/Kompilieren und Speichern bietet. Darüber hinaus enthält sie das Dateimenü zum Öffnen neuer oder vorhandener Dateien und den Abschnitt Beispiele, der vorgefertigte Skizzen für verschiedene Anwendungen wie Blink und Fade bietet.

Die 6 Schaltflächen am oberen Rand des Bildschirms sind wie folgt:

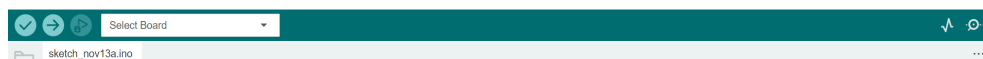


Abbildung 1.1.: Menu Button

- Das Häkchen im Symbol wird verwendet, um den geschriebenen Code zu überprüfen. Nachdem der Code eingegeben wurde, wird durch die Auswahl dieses Symbols auf Fehler geprüft und bestätigt, dass der Code richtig strukturiert ist. Dieser Schritt stellt sicher, dass der Code für die Verwendung mit der Hardware bereit ist.
- Das Symbol **Upload** in der Arduino IDE kompiliert Ihren Code und lädt ihn auf das angeschlossene Board hoch, so dass er lauffähig ist.
- Das Symbol **Start Debugging** Symbol in der Arduino IDE startet eine Debugging-Sitzung, die es Ihnen ermöglicht, Ihren Code zu analysieren, indem Sie Haltepunkte setzen, durch die Ausführung schreiten und Variablen auf kompatiblen Boards untersuchen.

Das Icon **Serial Plotter** in der Arduino IDE öffnet ein Tool, das die vom Board über die serielle Verbindung gesendeten Echtzeitdaten visualisiert und zur einfacheren Analyse als Graphen anzeigt.

Das Symbol **Serial Monitor** in der Arduino IDE öffnet ein Terminal zum Anzeigen und Senden von textbasierten Daten über die serielle Verbindung und ermöglicht die Kommunikation mit dem angeschlossenen Board in Echtzeit.

VS: hier noch einmal die letzten Grafiken in der Auflistung nach links verschieben

1.2. Installation der Arduino IDE

1.2.1. Download der Arduino IDE

Der Arduino Nano 33 BLE Sense nutzt zur Programmierung die Arduino Software **Integrated Development Environment (IDE)**, die am weitesten verbreitete IDE für alle Arduino-Boards und kann sowohl online als auch offline ausgeführt werden. Diese Open-Source-IDE für Arduino vereinfacht das Schreiben von Code und das Hochladen auf das Board. Verschiedene Versionen der Software sind für jedes Betriebssystem (OS) verfügbar, einschließlich macOS, Linux und Windows. Die Arduino-Community bietet auch eine Online-Plattform für das Programmieren und Speichern von Skizzen in der Cloud. Dieser Online-Arduino-Editor ist die neueste Version der IDE und enthält alle Bibliotheken und Unterstützung für neue Arduino-Boards. Um auf diese Softwarepakete zuzugreifen, besuchen Sie die folgende Website: [Arduino-Software](#), um auf dem Laufenden zu bleiben, da unter dem genannten Link täglich Updates verfügbar sind.

Der Editor kann problemlos direkt von der Arduino-Software-Seite heruntergeladen werden. Die Download-Optionen sind in Abbildung 1.2 zu sehen.

Downloads



Arduino IDE 2.3.3

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits

Windows MSI Installer

Windows ZIP file

Linux ApplImage 64 bits (X86-64)

Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits

macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Abbildung 1.2.: Arduino IDE 2.3.3 Download

1.2.2. Zusammenfassung der Optionen

Nachfolgend finden Sie eine Zusammenfassung 1.3 der verfügbaren Download-Optionen für die Arduino IDE, die auf verschiedene Betriebssysteme und individuelle Bedürfnisse

zugeschnitten sind. Wählen Sie die passende Version für Ihr Gerät, um Kompatibilität und Zugang zu den neuesten Funktionen zu gewährleisten.

Summary of Options:

Operating System	Option	Description
Windows	Windows 10 and newer (64-Bit)	Direct download for Windows 10 and newer 64-bit versions, without using the MSI installer.
	MSI Installer	Easy installation through an <code>.msi</code> package.
	ZIP File	Portable, no installation required, just extract and run directly.
Linux	ApplImage 64-Bit (x86-64)	Portable, no installation required.
	ZIP File 64-Bit (x86-64)	Portable, extract and run directly.
macOS	macOS Intel (10.15: Catalina or newer)	For Intel Macs with macOS 10.15 or newer.
	macOS Apple Silicon (11: Big Sur or newer)	For Apple Silicon Macs (M1, M2) with macOS Big Sur 11 or newer.

Abbildung 1.3.: Zusammenfassung der Download-Optionen

1.2.3. Installation der Arduino IDE für Windows

Der Download wird für die gewünschte Software ausgewählt Abbildung 1.4, in diesem Fall „Arduino IDE 2.3.2 für Windows10 and newer, 64 bits“. Nach Öffnen der Download Datei öffnet sich wie Abbildung 1.5 ein Fenster zur Abfrage des Lizenzabkommens.

Downloads



Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits

Windows MSI installer

Windows ZIP file

Linux ApplImage 64 bits (X86-64)

Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits

macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Abbildung 1.4.: Downloadfenster Arduino IDE

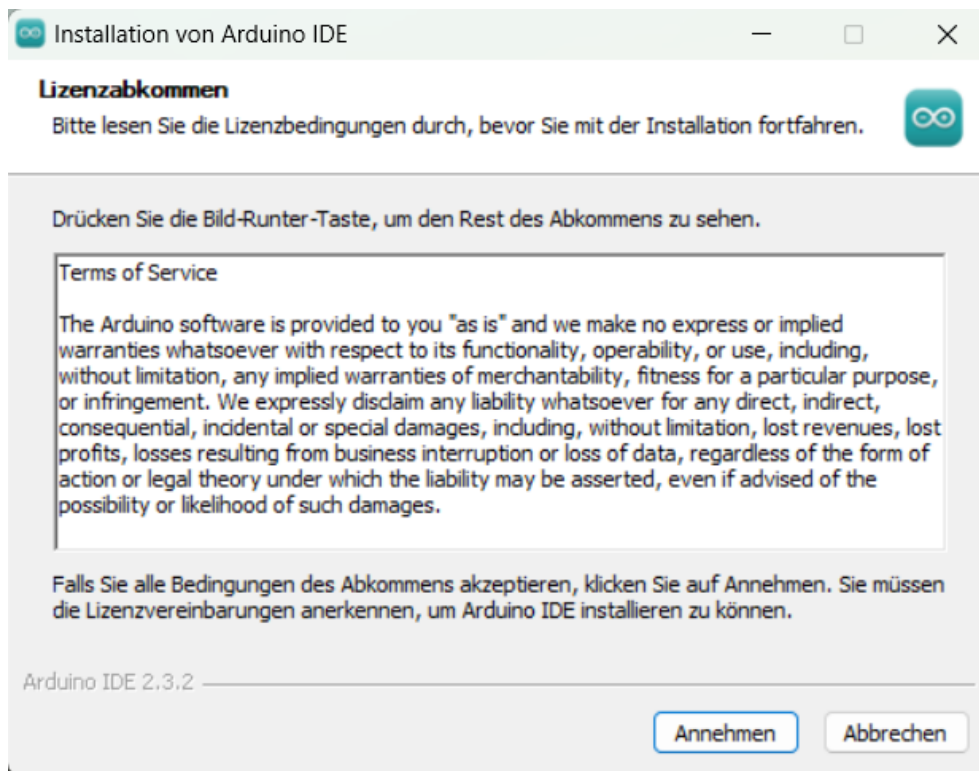


Abbildung 1.5.: Lizenzabkommen

Die 1.6 zeigt das darauf folgende Fenster zur Abfrage auf welches Konto auf dem PC das Programm installiert werden soll.

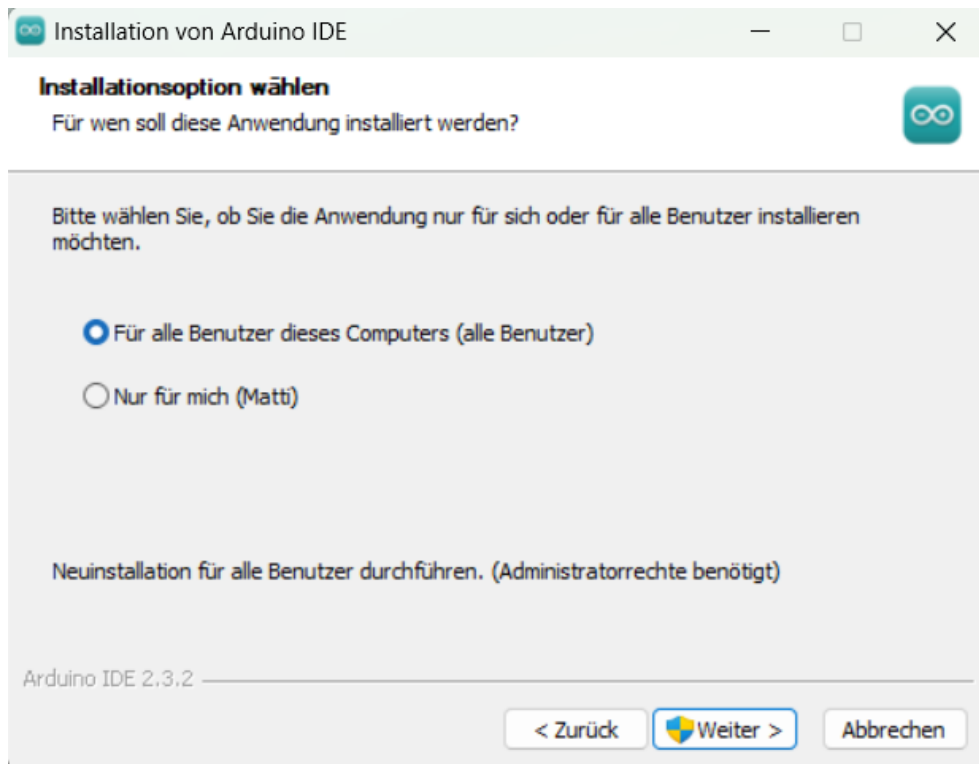


Abbildung 1.6.: Installationsoption

Die Installation FFür alle Benutzer dieses Computers"benötigt der Installer die Administrator rechte, nach der Genehmigung muss erneut das Lizenzabkommen bestätigt werden Abbildung 1.5. Darauf folgt Abbildung 1.7, die Auswahl des Zielverzeichnisses auf dem PC.

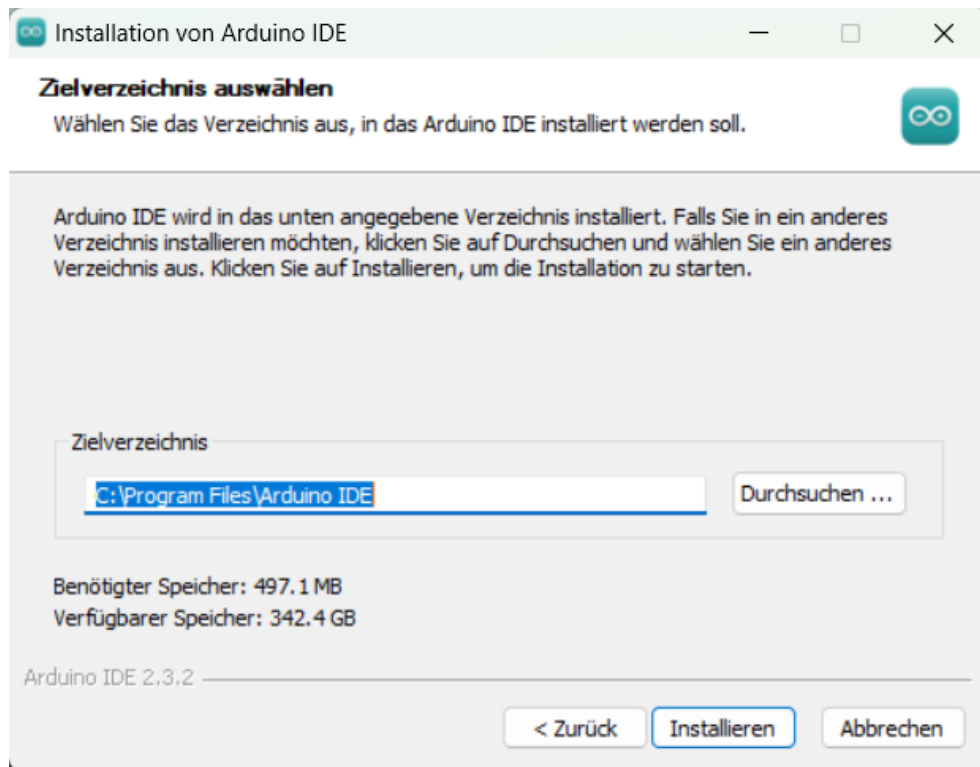


Abbildung 1.7.: Zielverzeichnisauswahl

Nach der Installation wird diese durch Fertigstellen beendet. Nun kann die Arduino IDE geöffnet werden.

1.3. Verknüpfen des Boards mit der IDE

Zur Verbindung des Arduino Boards mit der Arduino IDE muss in dieser zuerst das richtige Board ausgewählt werden Abbildung 1.8. Dazu wird in der Menüleiste unter dem Punkt **Tools** der Unterpunkt **Board** ausgewählt. Unter den Boards „Arduino AVR“ und hier der „Arduino Nano“ ausgewählt.

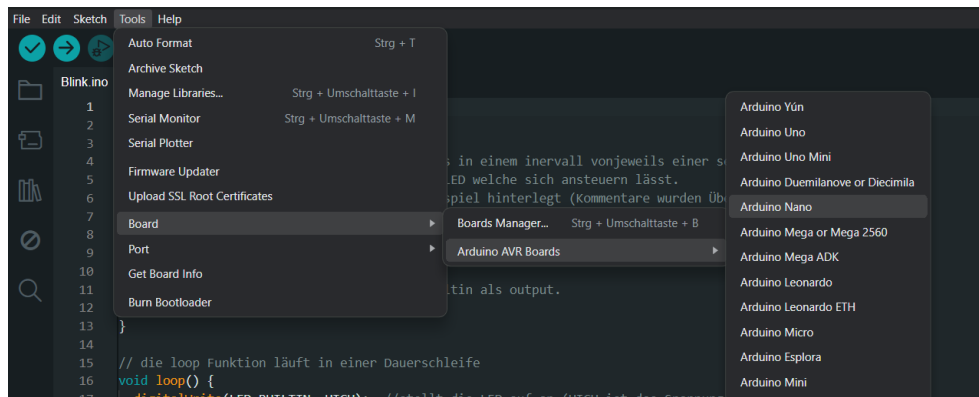


Abbildung 1.8.: Boardauswahl

Nach der Boardauswahl wird nun wieder in der Menüleiste unter dem Punkt **Tools** der Unterpunkt **Ports** ausgewählt Abbildung 1.9. Hierunter befinden sich die Anschlüsse des PC. Hier kann es sein, dass hier bereits erkennbar ist, an welchem Port der Arduino angeschlossen ist. Ist dies nicht der Fall, trennt man den Arduino vom PC, der nun nicht mehr angezeigte Port ist der richtige. Der Arduino kann nun wieder angeschlossen werden und der Port ausgewählt werden.

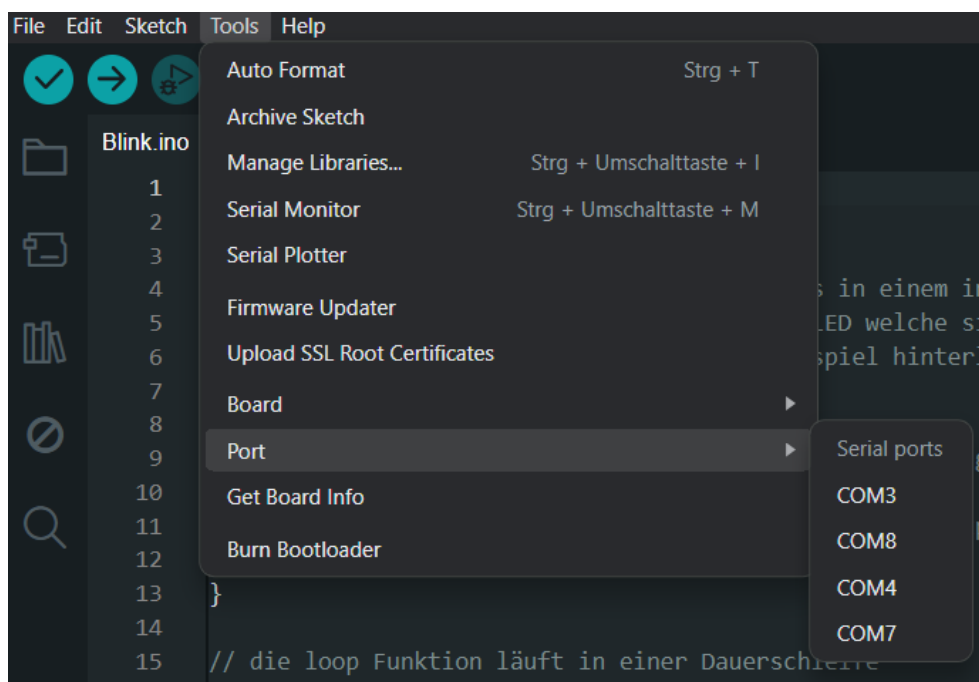


Abbildung 1.9.: Portauswahl

1.3.1. Beispielcode „Blinkende LED“

Der Aufbau und die Funktion der Programmierung in der Arduino IDE wird anhand des Beispiels Blinkenden LED gezeigt und erklärt Abbildung 1.1. Die LED ist auf dem Arduino Board fest installiert, somit benötigt das Beispiel keine Externe Hardware.

Listing 1.1.: Blink Programm

```

1000 /**
1001  * @file blink.ino
1002  * @brief Simple example to blink the on-board LED on and off.
1003  *
1004  * @details
1005  * Turns the built-in LED on for one second, then off for one second
1006  * repeatedly.
1007  * Most Arduino boards have a built-in LED connected to a specific
1008  * digital pin.
1009  * The macro LED_BUILTIN is used to ensure compatibility across
1010  * different boards.
1011  *
1012  * @note For more details about your board's built-in LED, see:
1013  *       https://www.arduino.cc/en/Main/Products
1014  *
1015  * @see https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
1016  *
1017  * @author
1018  * - Scott Fitzgerald (original)
1019  * - Arturo Guadalupi (modification)
1020  * - Colby Newman (modification)
1021  *
1022  * @date Modified:
1023  * - 8 May 2014
1024  * - 2 Sep 2016
1025  * - 8 Sep 2016
1026  *
1027  * @copyright Public Domain
1028  */
1029
1030 /**
1031  * @brief Initializes the LED pin as an output.
1032  *
1033  * This function runs once at startup after reset or power-up.
1034  */
1035 void setup() {
1036   pinMode(LED_BUILTIN, OUTPUT); ///< Set the LED pin as output
1037 }
1038
1039 /**
1040  * @brief Toggles the LED on and off every second.
1041  *
1042  * This function runs repeatedly after setup().
1043  */
1044 void loop() {
1045   digitalWrite(LED_BUILTIN, HIGH); ///< Turn the LED on
1046   delay(1000);                      ///< Wait for one second
1047   digitalWrite(LED_BUILTIN, LOW);  ///< Turn the LED off
1048   delay(1000);                      ///< Wait for one second
1049 }

```

../Code/Blink/Blink.ino

In den Zeilen 32 bis 34 steht die **Setup Funktion**, diese setzt in Zeile 33 über **pinMode** den Ausgang auf die integrierte LED über **LED BUILTIN** und setzt diesen als Output. Das Blinken wird in Zeile 41 bis 46 über die **loop** Funktion realisiert. In Zeile 42 wird über **digitalWrite** wird die LED auf **HIGH** gesetzt und ist damit an. Zeile 43 sorgt mit der Funktion **delay(1000)** für eine Pause von einer Sekunde, in der der Zustand der LED gleich bleibt. In Zeile 44 wird nun wieder über **digitalwrite** der zustand der LED auf **LOW** gesetzt und die LED ist aus. Zeile 45 ist nun wieder eine Pause von einer Sekunde. Dannach wird die **loop** Funktion endlos wiederholt.

1.3.2. Installation der Arduino IDE für MacOS

Der Download der Arduino IDE erfolgt über die Website . Geben Sie dazu „Arduino IDE Download “in Ihren Browser ein (vgl. Abbildung 1.10).

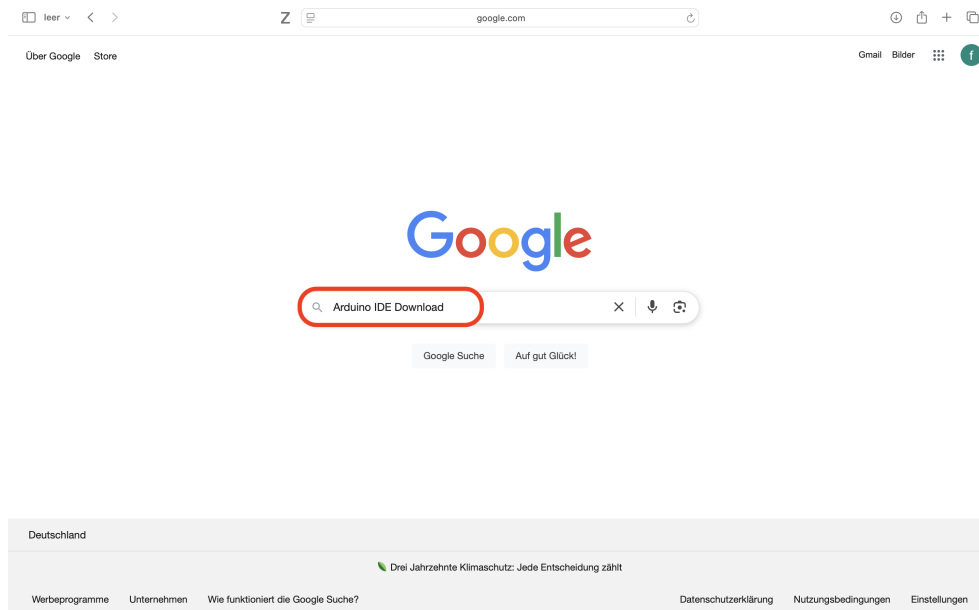


Abbildung 1.10.: Aufruf der Arduino-Webseite über die Suchmaschine

Abbildung 1.11 zeigt die Ergebnisse der Websuche. Klicken Sie auf den Eintrag mit „Arduino.cc“im Adresskopf.

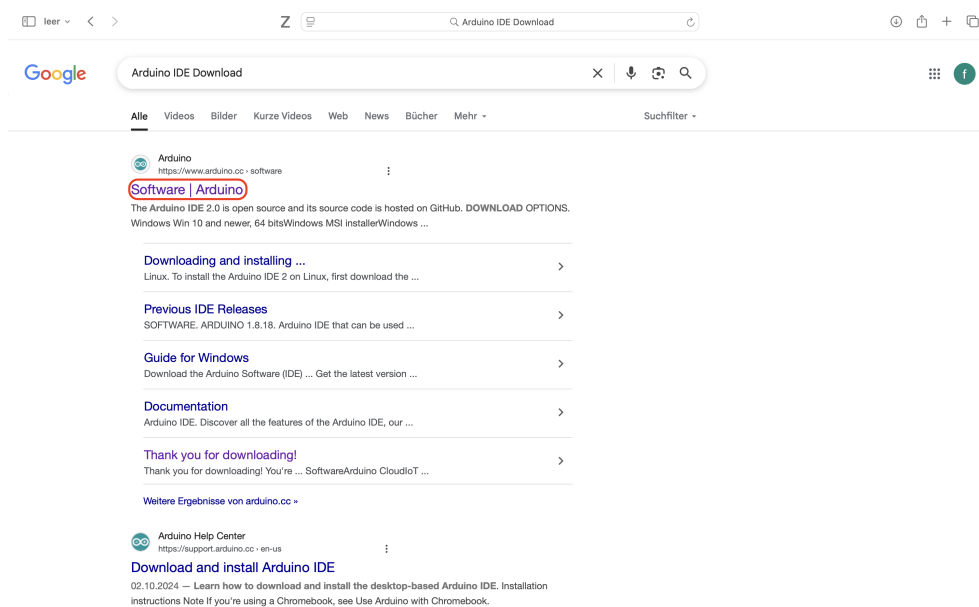


Abbildung 1.11.: Ergebnisse der Websuche zur Arduino IDE

Wählen Sie die passende Version für Ihren Mac aus (vgl. Abbildung 1.12).

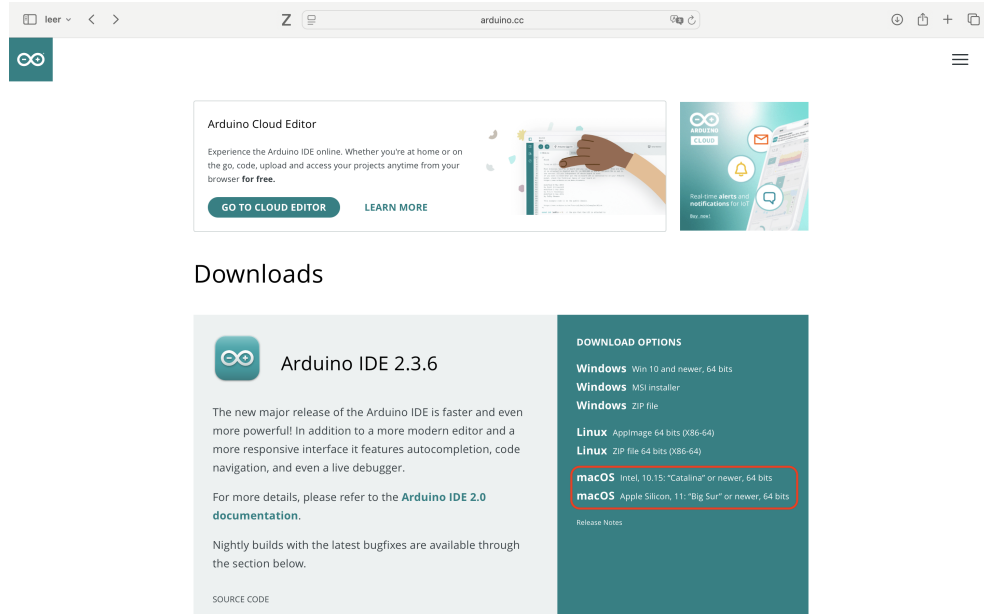


Abbildung 1.12.: Download der Arduino IDE starten

Es erscheint eine Spendenanfrage. Sie können freiwillig spenden oder über „JUST DOWNLOAD“ fortfahren (vgl. Abbildung 1.13).

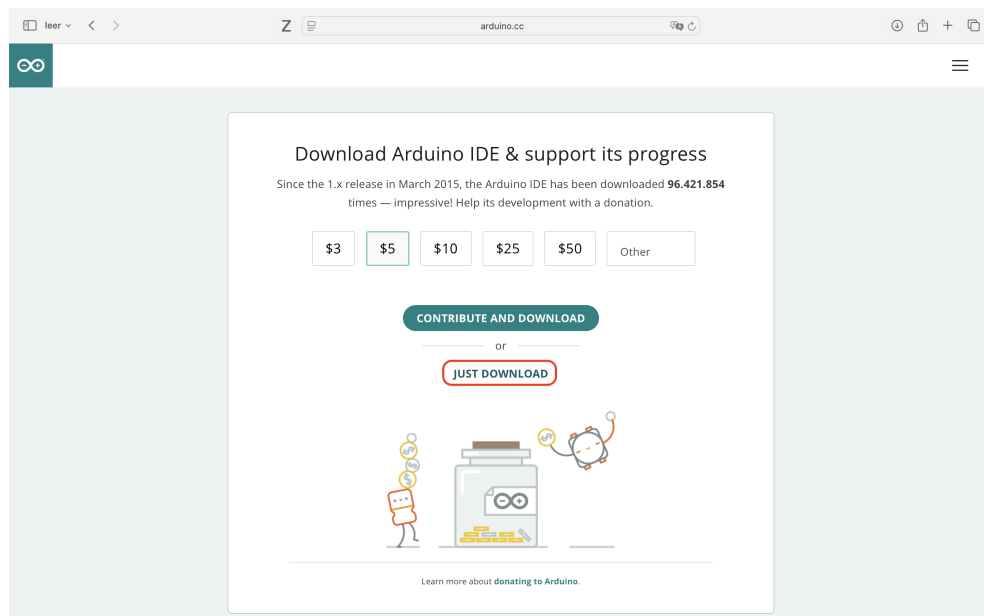


Abbildung 1.13.: Optional: Spendenaufruf beim Download

Vor dem Download können Sie optional den Newsletter abonnieren, vgl. Abbildung 1.14.

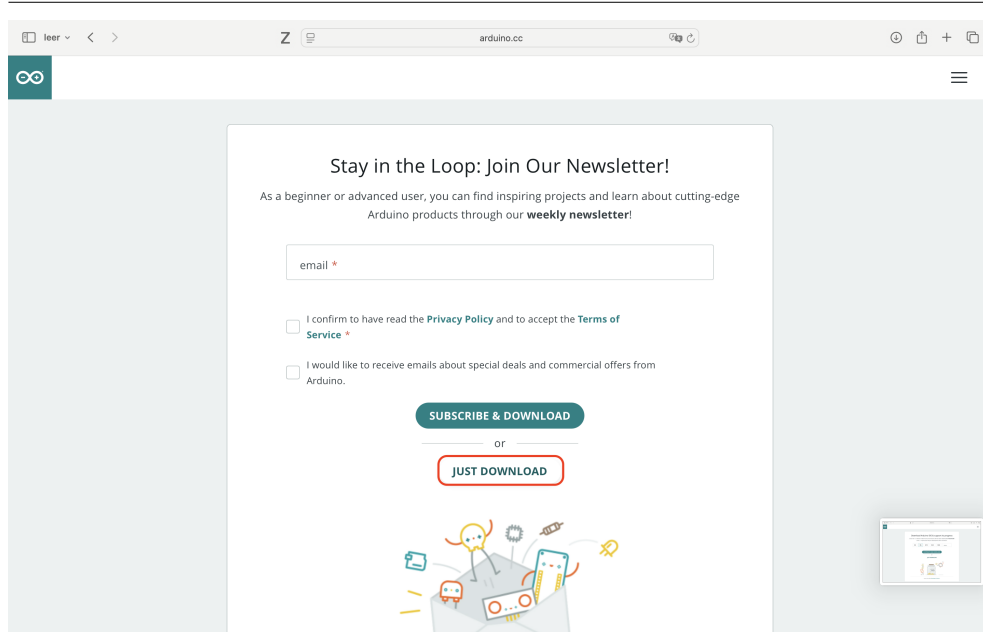


Abbildung 1.14.: Optional: Newsletter abonnieren

Da es sich um ein Programm aus dem Internet handelt, muss der Download zunächst erlaubt werden, vgl. Abbildung 1.15.

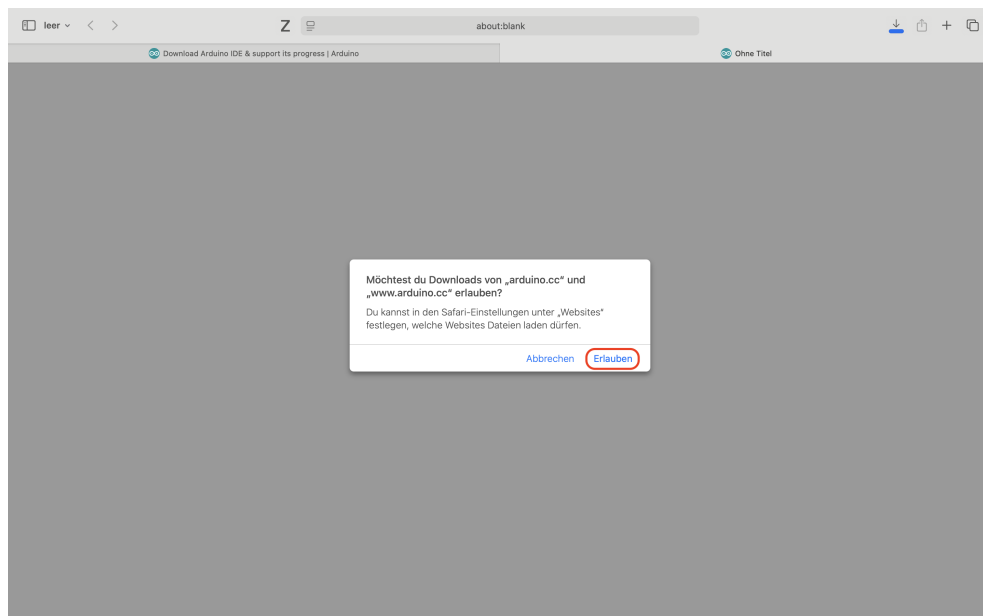


Abbildung 1.15.: Download-Erlaubnis erteilen

Über den Pfeil oben rechts lässt sich der Fortschritt anzeigen (vgl. Abbildung 1.16).

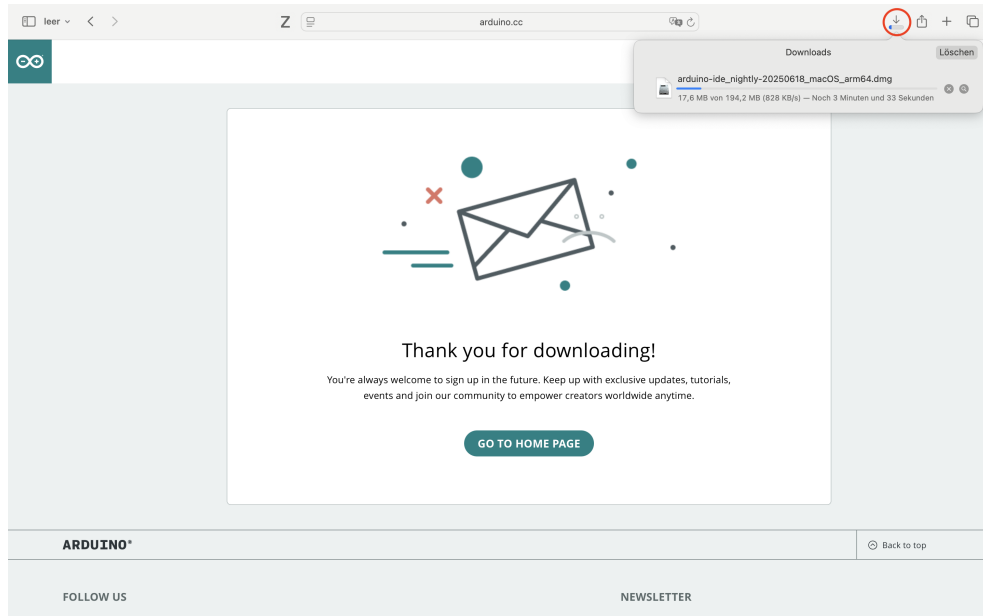


Abbildung 1.16.: Download-Fortschritt einsehen

Öffnen Sie nach dem Download die Datei per Doppelklick (vgl. Abbildung 1.17).

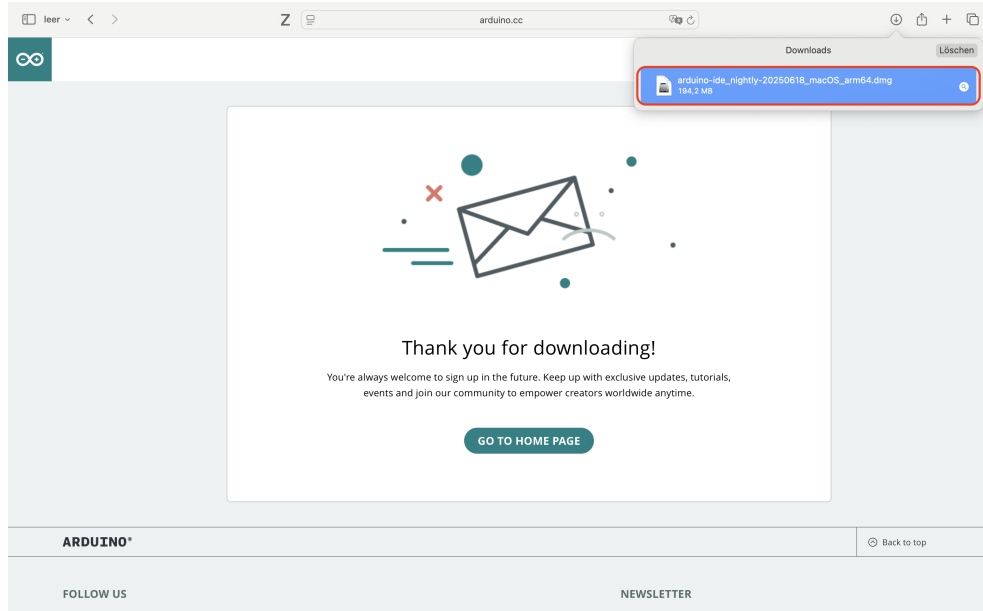


Abbildung 1.17.: Heruntergeladene Datei öffnen

Ziehen Sie nun die App in den Programme-Ordner (vgl. Abbildung 1.18).



Abbildung 1.18.: Installation durch Ziehen in den Programme-Ordner

Die Installation ist abgeschlossen. Öffnen Sie die App über den Programme-Ordner (vgl. Abbildung 1.19).

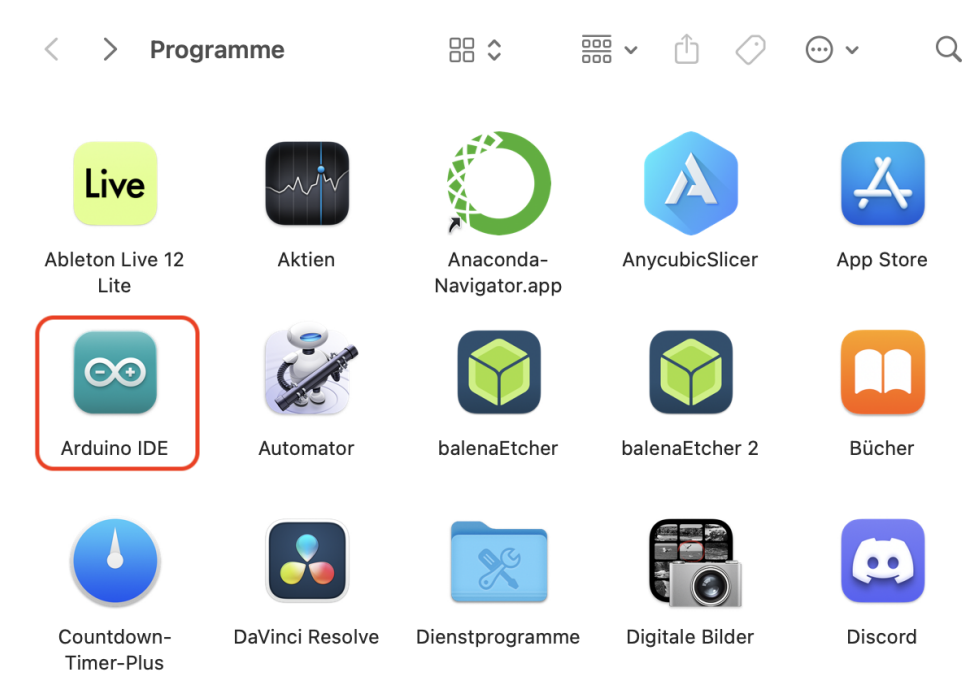


Abbildung 1.19.: Arduino IDE starten

Bestätigen Sie die Sicherheitsabfrage mit „Öffnen“ (vgl. Abbildung 1.20).



Abbildung 1.20.: Sicherheitsabfrage bei erstmaligem Start

1.3.3. Bibliotheken installieren

Klicken Sie auf das Buch-Symbol (vgl. Abbildung 1.21) und suchen Sie die gewünschte Bibliothek. Die vollständige Liste finden Sie in Kapitel ???. Installieren Sie jede Bibliothek durch Klick auf „Install“.

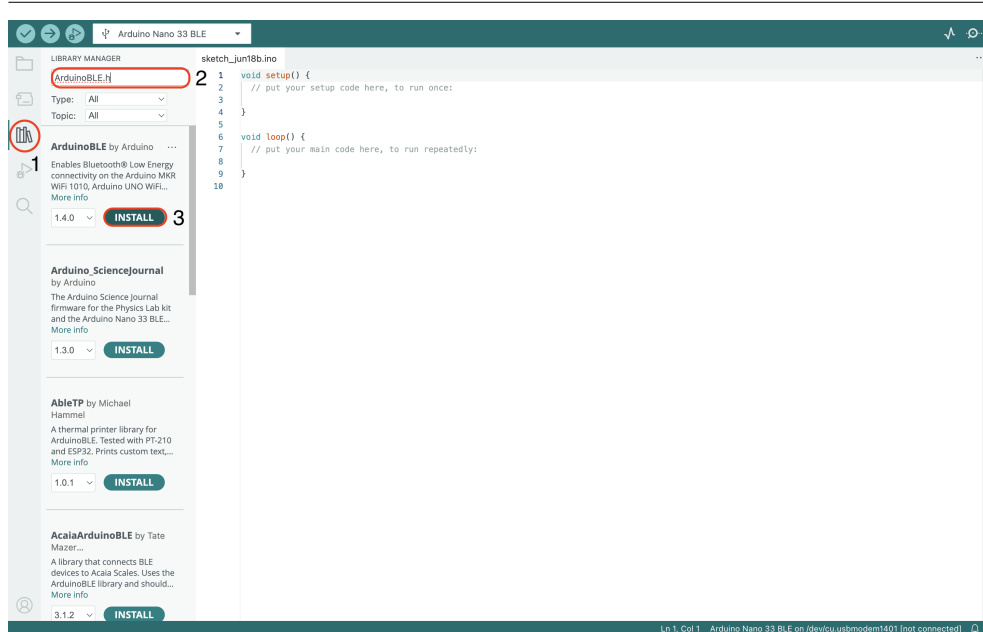


Abbildung 1.21.: Bibliotheken-Suchmaske der Arduino IDE

Abbildung 1.22 zeigt eine erfolgreich installierte Bibliothek.

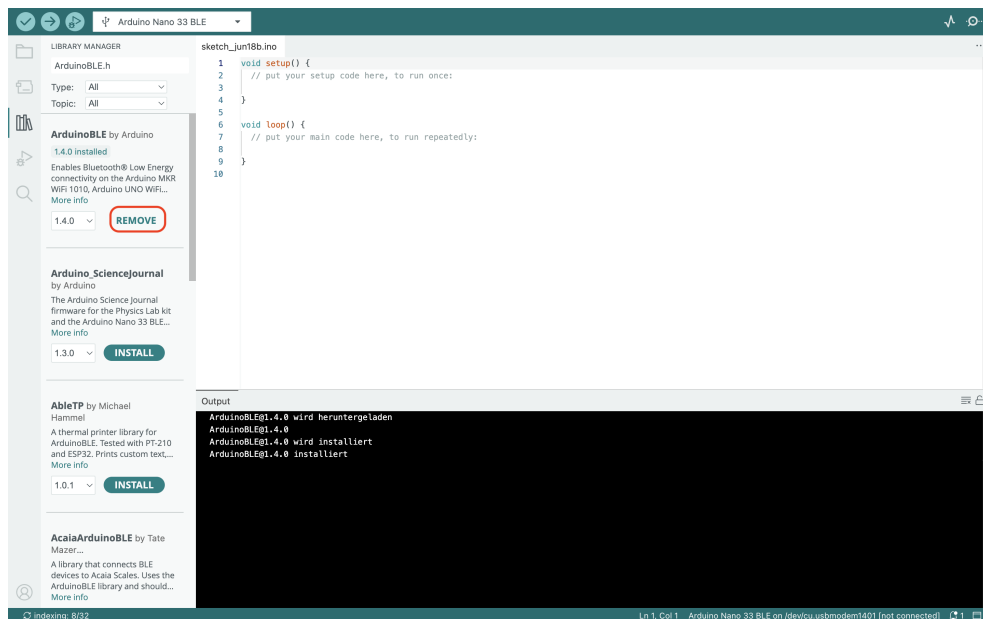


Abbildung 1.22.: Erfolgreiche Installation einer Bibliothek

1.3.4. Board Manager

Mit dem Board Manager können Sie zusätzliche Boards hinzufügen (vgl. Abbildung 1.23). Suchen Sie zum Beispiel nach dem „ESP“ oder „Nano 33 BLE Sense“ und installieren Sie das passende Board.

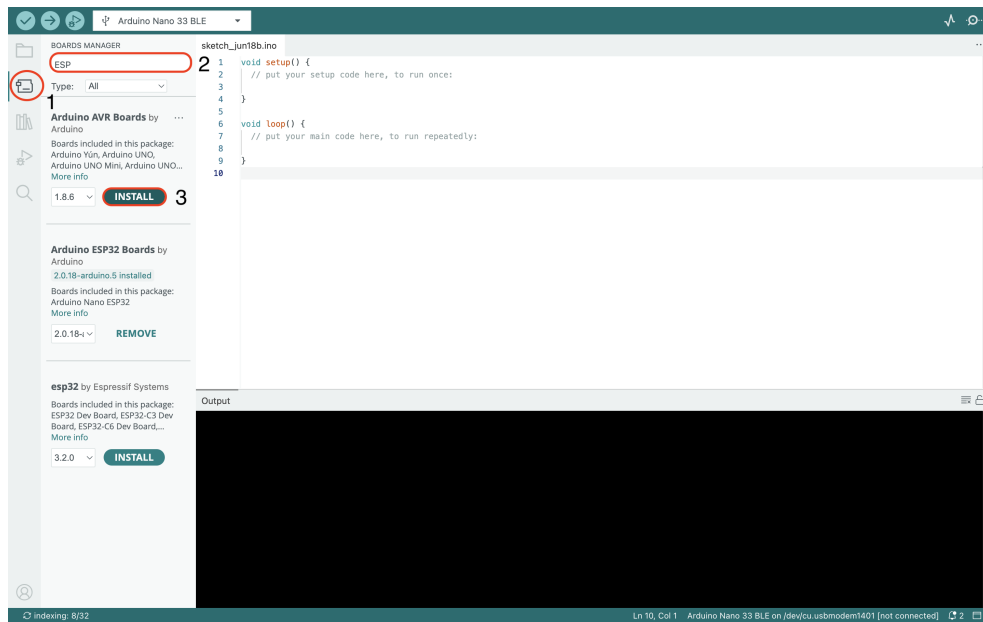


Abbildung 1.23.: Der Board Manager in der Arduino IDE

Klappen Sie das USB-Menü aus (vgl. Abbildung 1.24) und wählen Sie „Select other board and port“.

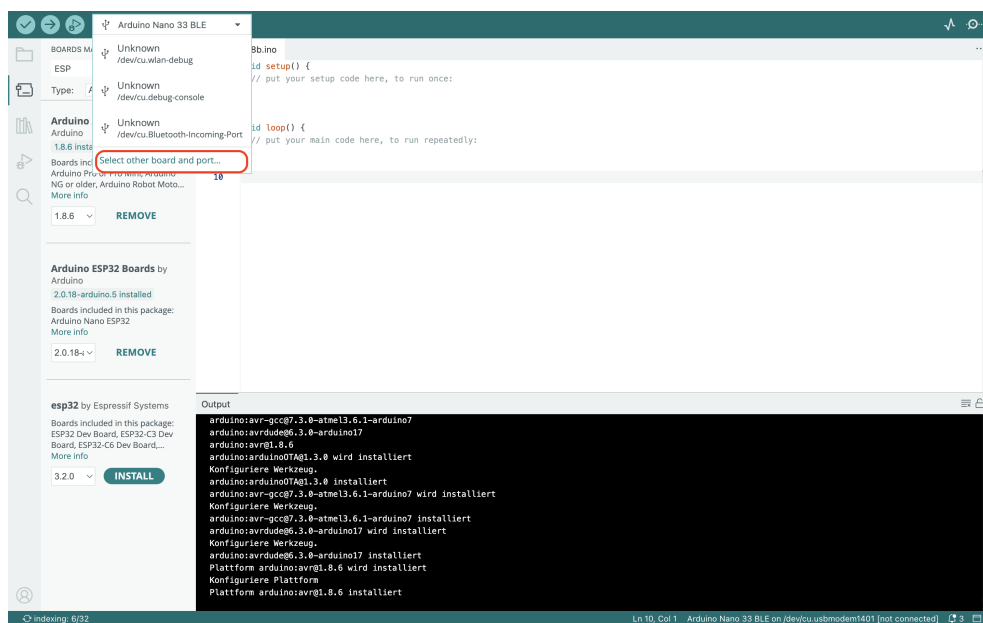


Abbildung 1.24.: Board- und Portauswahl öffnen

Geben Sie „Nano 33 BLE Sense“ in das Suchfeld ein, verbinden Sie das Board und wählen Sie den richtigen Port (vgl. Abbildung 1.25).

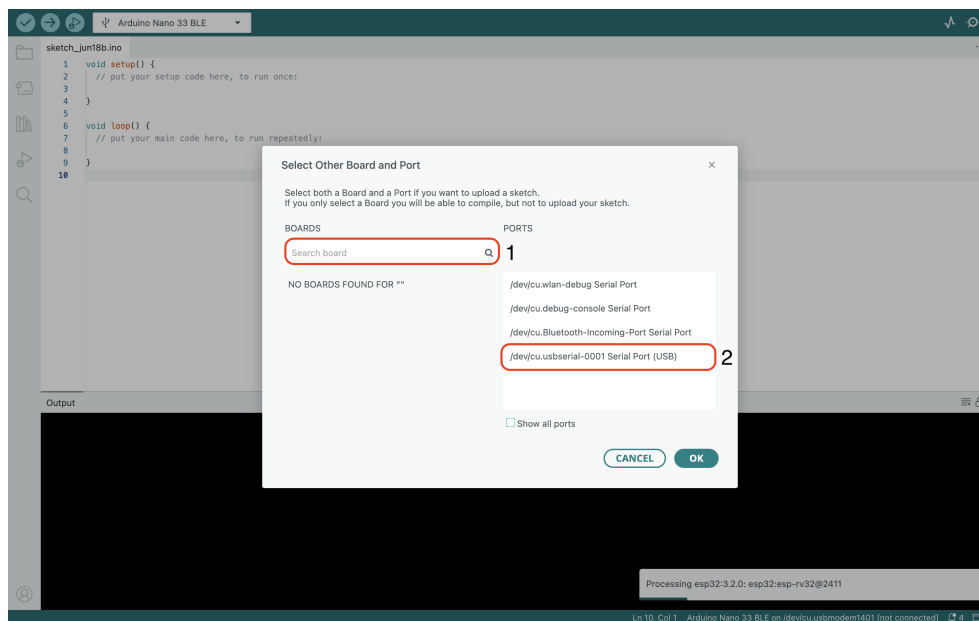


Abbildung 1.25.: Passenden Board und Port auswählen

1.3.5. Beispielprogramm „Blink“

Wählen Sie den Pfad gemäß Abbildung 1.26, um ein Beispielprogramm zu laden.

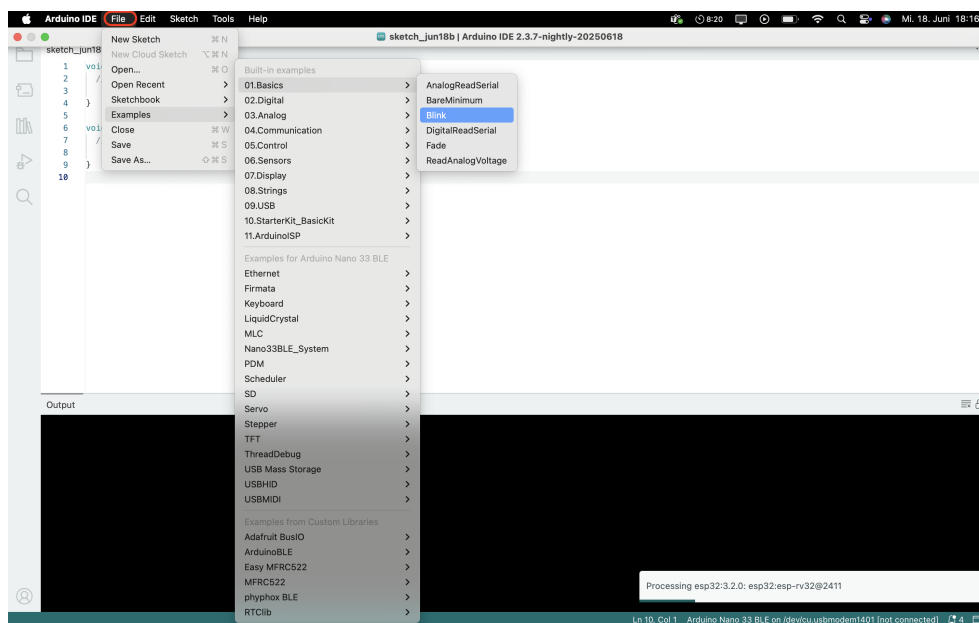
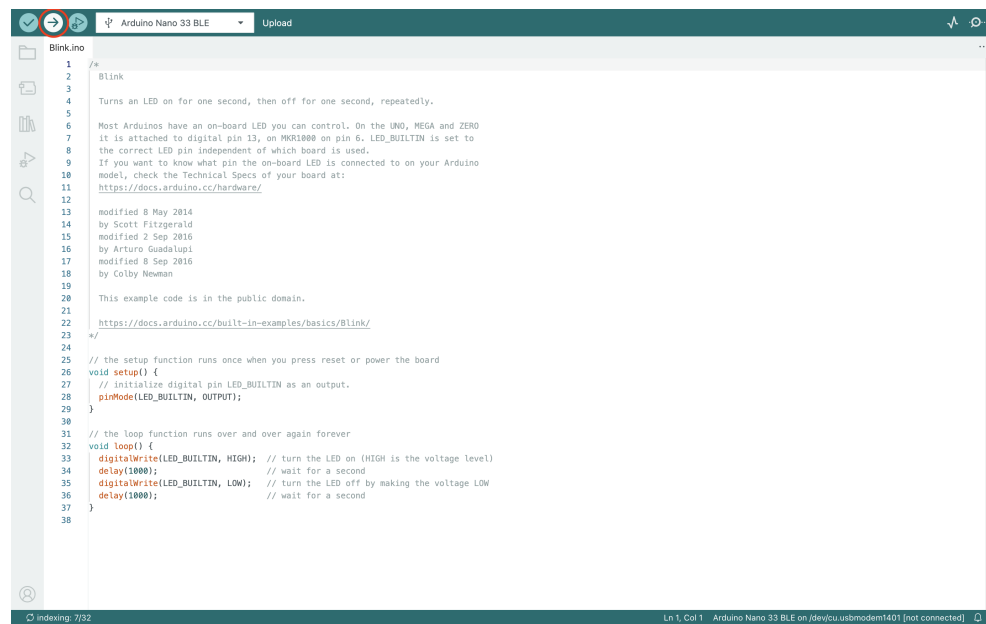


Abbildung 1.26.: Beispielprogramme öffnen

Das Blink-Programm wird geöffnet (vgl. Abbildung 1.27) und kann nach Auswahl des Ports hochgeladen werden (siehe 1.3.4). Die Kompilierung erfolgt automatisch vor dem Upload oder manuell über das Häkchen-Symbol.



The screenshot shows the Arduino IDE interface with the 'Blink' example code loaded. The top toolbar includes icons for opening files, saving, and uploading. The 'Blink' tab is active, showing the code for an Arduino Nano 33 BLE. The code is as follows:

```
1 /*  
2  * Blink  
3  */  
4  Turns an LED on for one second, then off for one second, repeatedly.  
5  
6  Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO  
7  it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to  
8  the correct LED pin independent of which board is used.  
9  If you want to know what pin the on-board LED is connected to on your Arduino  
10 model, check the Technical Specs of your board at:  
11 https://docs.arduino.cc/hardware/  
12  
13 modified 8 May 2014  
14 by Scott Fitzgerald  
15 modified 2 Sep 2016  
16 by Arturo Guadalupi  
17 modified 8 Sep 2016  
18 by Colby Newman  
19  
20 This example code is in the public domain.  
21  
22 https://docs.arduino.cc/built-in-examples/basics/Blink/  
23 */  
24  
25 // the setup function runs once when you press reset or power the board  
26 void setup() {  
27   // initialize digital pin LED_BUILTIN as an output.  
28   pinMode(LED_BUILTIN, OUTPUT);  
29 }  
30  
31 // the loop function runs over and over again forever  
32 void loop() {  
33   digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)  
34   delay(1000); // wait for a second  
35   digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW  
36   delay(1000); // wait for a second  
37 }  
38
```

The status bar at the bottom indicates 'Ln 1, Col 1' and 'Arduino Nano 33 BLE on /dev/cu.usbmodem1401 [not connected]'.

Abbildung 1.27.: Das geöffnete Beispielprogramm „Blink“

Nach erfolgreichem Upload erscheint ein Terminalfenster mit einer Bestätigung.

2. Arduino IDE 2.3.6

Dieses Kapitel liefert eine detaillierte Einführung in die Arduino IDE, einschließlich ihrer zentralen Funktionen, verfügbaren Schnittstellen und grundlegenden Arbeitsweise. Es erläutert die einzelnen Bestandteile der Entwicklungsumgebung, beschreibt den Installationsprozess anhand klar strukturierter Schritte und bietet eine Anleitung zur Ersteinrichtung sowie zur grundlegenden Konfiguration der Programmierungsumgebung.

2.1. Arduino IDE Beschreibung

Die Arduino IDE ist die offizielle, Open-Source Entwicklungsumgebung zur Programmierung und Verwaltung von Arduino Mikrocontrollern. Sie ermöglicht das Schreiben, Kompilieren und Übertragen von Quellcode auf eine Vielzahl unterstützter Boards. Als plattformübergreifende Anwendung ist sie mit allen gängigen Betriebssystemen wie Windows, Linux und macOS kompatibel. Ursprünglich auf der Java-Plattform basierend, bietet die IDE Unterstützung für zahlreiche Arduino-Boards sowie eine auf C und C++ basierende Programmiersprache. Durch ihre intuitive Benutzeroberfläche richtet sie sich sowohl an Einsteiger als auch an erfahrene Entwickler im Bereich der Mikrocontroller-Programmierung [FAD18].

Die Mikrocontroller auf Arduino-Boards werden mithilfe von Programmen gesteuert, die in Form von Quellcode verfasst und über die Entwicklungsumgebung verarbeitet werden. In der Arduino IDE erstellte Programme werden als sogenannte „Sketche“ bezeichnet. Dabei handelt es sich um Skripte, die den Funktionsablauf des Mikrocontrollers definieren – etwa das Einlesen von Sensorwerten, die Ansteuerung von Aktoren oder die Kommunikation mit anderen Geräten. Nach dem Schreiben wird der Sketch von der IDE kompiliert und in eine maschinenlesbare Hex-Datei übersetzt. Diese Datei wird im Anschluss über die serielle Schnittstelle auf den Mikrocontroller hochgeladen, wo sie dauerhaft gespeichert und beim nächsten Start ausgeführt wird [FAD18].

Die Arduino-IDE setzt sich im Wesentlichen aus zwei funktionalen Kernkomponenten zusammen: dem Editor und dem Compiler. Der integrierte Editor dient als zentrale Arbeitsfläche zum Schreiben und Bearbeiten von Quellcode, während der Compiler für die Übersetzung des Codes in maschinenlesbare Anweisungen sowie dessen Übertragung auf das angeschlossene Arduino-Modul verantwortlich ist [FAD18].

In der Version 2.3.6 der Arduino IDE spielt die Menüleiste am oberen Rand der Benutzeroberfläche eine zentrale Rolle. Sie bietet direkten Zugriff auf essentielle Funktionen wie das Hochladen von Sketchen auf das Board, das Kompilieren zur Überprüfung auf Fehler sowie das Speichern von Projekten. Unter dem Menüpunkt „Datei“ lassen sich neue Sketche anlegen, bestehende Projekte öffnen oder speichern. Ein weiterer bedeutender Abschnitt ist „Beispiele“ – dort finden sich eine Vielzahl vorgefertigter Sketches, die typische Anwendungsszenarien demonstrieren, darunter klassische Programme wie „Blink“ zur LED-Steuerung oder „Fade“ zur Helligkeitssteuerung mittels Pulsweitenmodulation. Diese Beispiele erleichtern insbesondere Einsteigern den Zugang zur Mikrocontroller-Programmierung und dienen oft als Ausgangspunkt für eigene Entwicklungen [FAD18].

Die fünf Schaltflächen am oberen Bildschirmrand sind wie folgt:



-  Das Häkchen-Symbol wird verwendet, um den geschriebenen Code zu **Überprüfen**, nachdem der Code eingegeben wurde, wird der Code auf Syntaxfehler geprüft. Dieser Schritt stellt sicher, dass der Code für die Verwendung mit der Hardware bereit ist.
-  Das Symbol **Hochladen** in der Arduino IDE kompiliert den Code und lädt ihn auf den angeschlossenen Arduino hoch.
-  Das Symbol **Start Debugging** in der Arduino IDE startet eine Debugging Sitzung. Damit kann der Code analysiert werden, indem Haltepunkte gesetzt werden. Beim Debuggen wird die Ausführung schrittweise durchlaufen und Variablen auf Kompatibilität und Funktion geprüft, ohne das ganze Programm in einem Stück zu starten.
-  Das Symbol **Serial Plotter** in der Arduino IDE öffnet ein Tool, das Echtzeitdaten, die vom Board über die serielle Verbindung gesendet werden, grafisch darstellt. Dadurch wird die Analyse der Daten erleichtert.
-  Das Symbol **Serial Monitor** in der Arduino IDE öffnet ein Terminal, um textbasierte Daten über die serielle Verbindung anzuzeigen und zu senden. Dadurch wird eine Echtzeitkommunikation mit dem angeschlossenen Board ermöglicht.

2.2. Installation der Arduino IDE

2.2.1. Download oder Arduino IDE

Der Arduino Nano 33 BLE Sense wird über die Arduino Integrated Development Environment (IDE) programmiert, die als Standardumgebung für sämtliche Arduino-Boards etabliert ist. Sie zählt zu den am weitesten verbreiteten Entwicklungsplattformen im Bereich der Mikrocontroller-Programmierung. Die IDE ist sowohl als Offline-Variante zur lokalen Installation als auch als Online-Editor verfügbar. Als Open-Source-Software ist sie darauf ausgelegt, den Programmierprozess möglichst zugänglich zu gestalten – vom Schreiben des Codes bis hin zum Hochladen auf das jeweilige Board.

Für alle gängigen Betriebssysteme – einschließlich macOS, Linux und Windows – stehen passende Versionen zur Verfügung. Die Installation ist unkompliziert und wird durch die breite Unterstützung der Arduino-Community zusätzlich erleichtert. Neben der Desktop-Version bietet Arduino auch eine cloudbasierte Entwicklungsumgebung an: den Online-Arduino-Editor. Diese moderne Webanwendung ermöglicht das plattformunabhängige Programmieren, Speichern und Verwalten von Sketchen direkt im Browser. Sie enthält alle gängigen Bibliotheken und bietet sofortige Unterstützung für aktuelle Arduino-Boards, einschließlich des Nano 33 BLE Sense.

Zugriff auf die verschiedenen Softwarepakete erhalten Sie über die offizielle Website der Arduino-Entwickler: [Arduino-Software](https://www.arduino.cc/en/software). Dort finden sich stets die neuesten Versionen der IDE sowie weiterführende Informationen. Eine Übersicht der verfügbaren Downloadoptionen ist in Abbildung 2.1 dargestellt.



Abbildung 2.1.: Arduino IDE 2.3.6

2.2.2. Anforderungen Betriebssystem

Im Folgenden sind die Systemanforderungen und Varianten der verfügbaren Downloadoptionen für die Arduino-IDE aufgeführt. Diese sind jeweils auf unterschiedliche Betriebssysteme und Nutzerbedürfnisse zugeschnitten. Um einen reibungslosen Betrieb sowie den Zugriff auf alle aktuellen Funktionen und Sicherheitsupdates zu gewährleisten, sollte stets die zur eigenen Plattform passende Version der IDE ausgewählt werden.

- Windows– Win 10 und neuer, 64 Bit
- Linux– 64 Bit
- macOS– Intel, Version 10.15: „Catalina“ oder neuer, 64 Bit
- macOS– Apple Silicon, 11: „Big Sur“ oder neuer, 64 Bit

2.2.3. Installation auf Windows

Die Installation der Arduino IDE auf einem Windows-System wird im folgenden Abschnitt Schritt für Schritt erläutert. Im Anschluss daran folgt eine Einführung in zentrale Funktionen der Umgebung, darunter der Board-Manager, der Bibliotheks-Manager, das Sketchbook sowie weitere essentielle Werkzeuge der Benutzeroberfläche.

Um die Arduino IDE auf einem Windows-Computer zu installieren, laden Sie zunächst die entsprechende Installationsdatei von der offiziellen Website unter [Arduino-Software](#) herunter. Starten Sie anschließend die heruntergeladene Datei und folgen Sie den angezeigten Installationsanweisungen. Diese beinhalten in der Regel die

Auswahl des Installationsverzeichnisses sowie optionaler Komponenten wie Verknüpfungen. Sobald der Vorgang abgeschlossen ist, steht die Entwicklungsumgebung zur sofortigen Nutzung bereit.

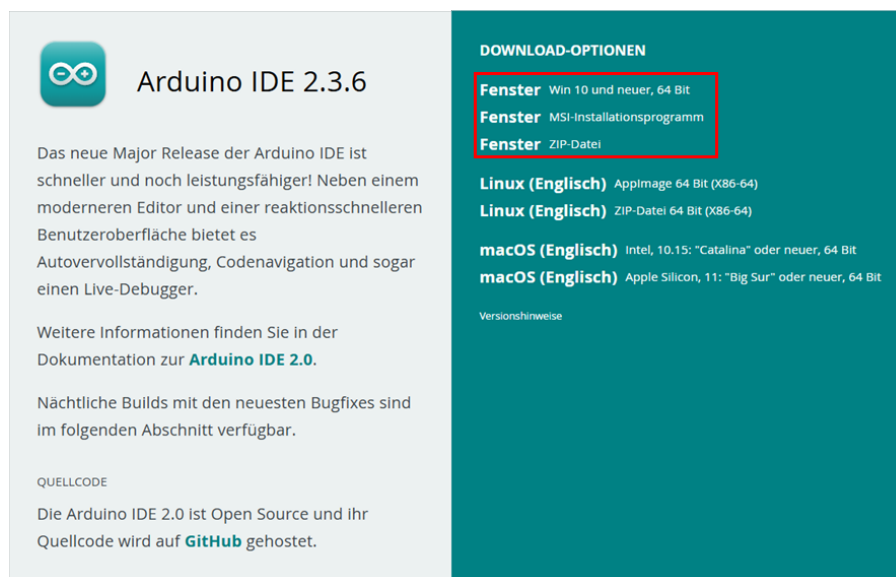


Abbildung 2.2.: Arduino IDE Downloadoptionen

Nachdem der Download der Arduino IDE abgeschlossen wurde, erscheint im nächsten Schritt das Fenster mit den Lizenzbedingungen. Diese sollten aufmerksam durchgelesen werden, bevor mit der Installation fortgefahren wird. Um die Installation zu starten, müssen alle Lizenzbedingungen akzeptiert werden. Dies geschieht durch einen Klick auf die Schaltfläche **Annehmen**, wie in Abbildung 2.3 dargestellt.

Im Anschluss wird der Benutzer aufgefordert, ein Zielverzeichnis für die Installation auszuwählen. Standardmäßig ist ein Pfad vorgegeben, dieser kann jedoch bei Bedarf individuell angepasst werden. Nachdem das gewünschte Verzeichnis ausgewählt wurde, wird der Installationsvorgang durch einen Klick auf **Installieren** gestartet, siehe Abbildung 2.4.

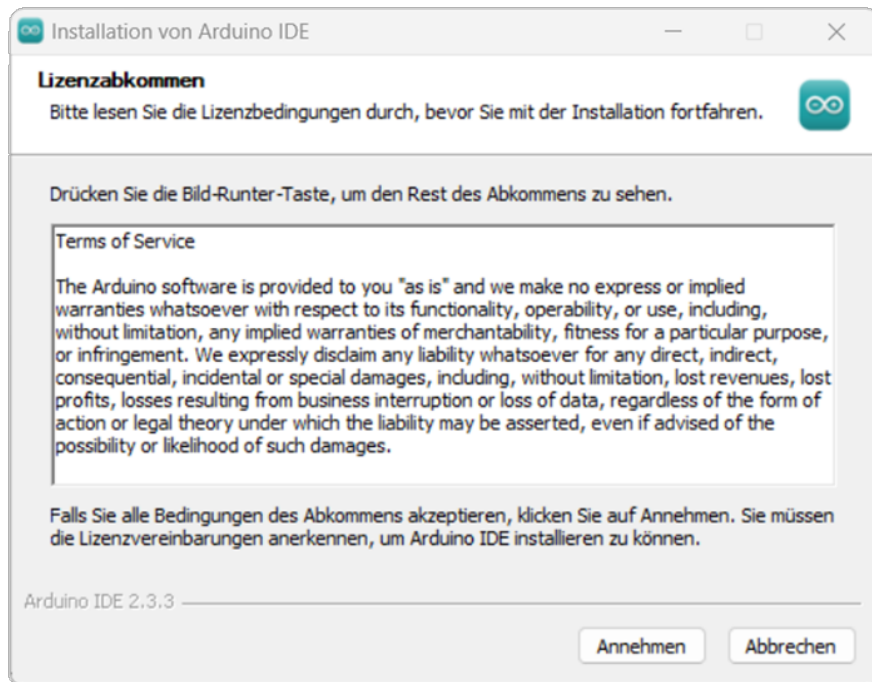


Abbildung 2.3.: Lizenzbestätigung Installation

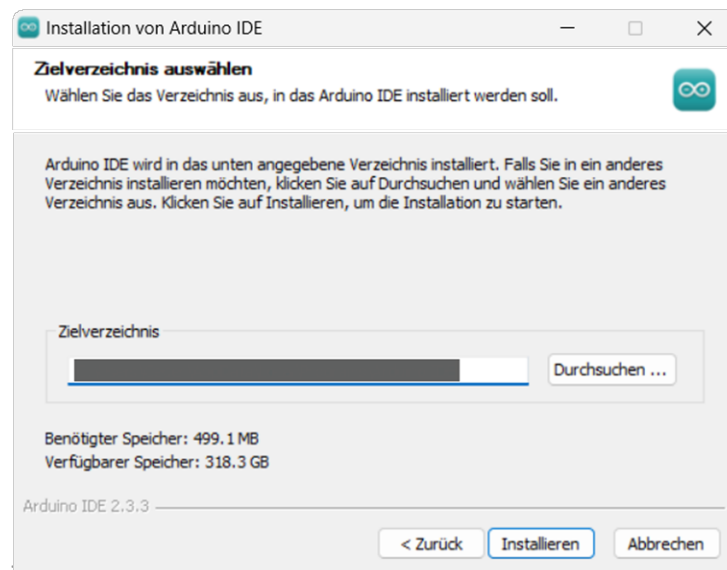


Abbildung 2.4.: Wahl Zielverzeichnis

Sobald die Installation der Arduino IDE erfolgreich abgeschlossen ist, wird die Anwendung automatisch gestartet. Beim ersten Öffnen der Arduino IDE wird ein Standard-Sketch angezeigt, der als Beispiel dient. Dieser ist im Startfenster sichtbar, wie in Abbildung 2.5 dargestellt.

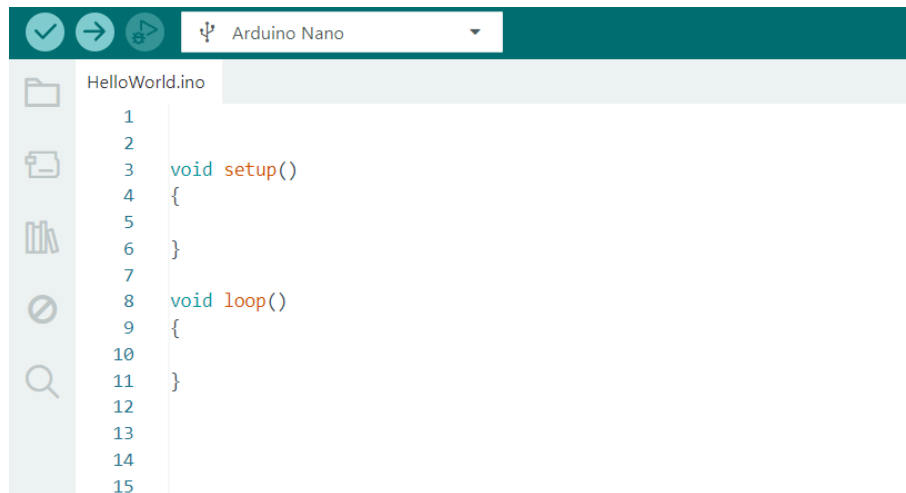


Abbildung 2.5.: Startfenster Arduino-IDE

Aus der oben dargestellten Abbildung 2.5 geht hervor, dass der grundlegende Arduino-Sketch aus zwei Hauptteilen besteht. Der erste Teil ist die Funktion `void setup()`, die keinen Rückgabewert liefert und in der typischerweise die Initialisierungen vorgenommen werden. Hier werden beispielsweise Einstellungen wie die Konfiguration der Ausgangs-LED, die Festlegung von Pin-Modi oder die Initialisierung von Variablen durchgeführt.

Der zweite Teil des Sketches ist die Funktion `void loop()`, in der der Hauptcode geschrieben wird. Funktionen, die wiederholt ausgeführt werden sollen, werden hier definiert und laufen fortlaufend in der Schleife. Diese Funktionen sind ebenfalls zwischen geschweiften Klammern `{ }` eingefasst. Jede Funktion in Arduino hat einen Rückgabewert – in diesem Fall handelt es sich um den Rückgabebetyp `void`, was bedeutet, dass die Funktion keinen Wert zurückgibt.

2.2.4. Installation (MacOS)

Um die Arduino IDE zu installieren, beginnen Sie mit dem Herunterladen der neuesten Version von der offiziellen Arduino-Website unter [Arduino-Software](#). Wählen Sie die Version, die mit Ihrem Betriebssystem kompatibel ist. In diesem Beispiel wird die Version Arduino 2.6.3 für macOS (Sonoma 14.4.1) verwendet. Die Setup-Datei trägt den Namen „Arduino-Software“ und hat eine Größe von etwa 193.600 KB. Diese Datei ist im Zip-Format.

Für Nutzer, die die Arduino IDE 2.3.6 mit Safari herunterladen, wird die Datei nach Abschluss des Downloads automatisch entpackt. Andere Browser hingegen erfordern möglicherweise eine manuelle Entpackung. Die folgende Abbildung 2.6 zeigt die neueste Offline-Version der Arduino IDE 2.3.6, die ebenfalls mit allen gängigen Betriebssystemen kompatibel ist.

Da in diesem Kapitel keine entsprechende Hardware für macOS zur Verfügung steht, wird der Installationsprozess für dieses Betriebssystem nicht im Detail behandelt.

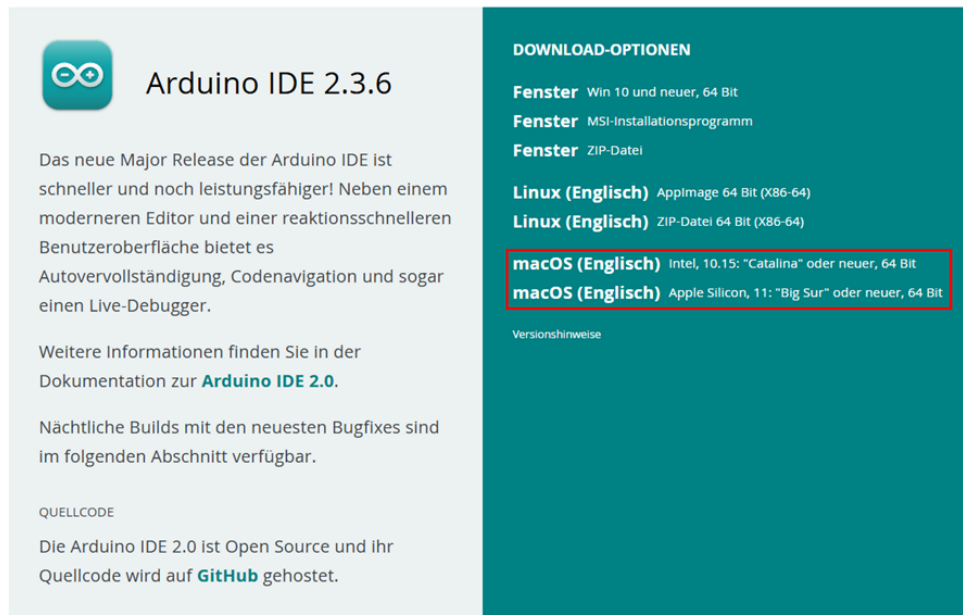


Abbildung 2.6.: ArduinoIDE macOS

2.3. Übersicht der Eigenschaften

Die Arduino IDE verfügt über eine neue Seitenleiste, die den Zugriff auf die am häufigsten verwendeten Werkzeuge erleichtert. 2.7

- **Verify / Upload** Diese Funktionen werden verwendet, um Code zu kompilieren und auf ein Arduino-Board hochzuladen.
- **Select Board and Port** Automatisch erkannte Arduino-Boards werden hier zusammen mit der Portnummer angezeigt.
- **Sketchbook** Dieser Bereich dient als zentrales Verzeichnis für alle Sketche, die lokal auf dem Computer des Benutzers gespeichert sind. Das Sketchbook bietet einen strukturierten, leicht zugänglichen Raum zum Verwalten, Organisieren und Bearbeiten von Code, der in der Arduino-Umgebung entwickelt wurde. Darüber hinaus bietet das Sketchbook eine Synchronisierungsfunktion mit der Arduino Cloud, die einen nahtlosen Zugriff auf gespeicherte Sketche über verschiedene Geräte hinweg ermöglicht. Diese Cloud-Integration erlaubt es Nutzern, Sketche aus der Online-Arduino-Umgebung abzurufen und zu bearbeiten.
- **Boards Manager** Durchsuchen Sie Arduino- und Drittanbieterpakete, die installiert werden können. Zum Beispiel erfordert die Verwendung eines MKR WiFi 1010-Boards die Installation des Arduino SAMD Boards-Pakets.
- **Library Manager** Durchsuchen Sie tausende Arduino-Bibliotheken, die von Arduino und seiner Community erstellt wurden.
- **Debugger** Programme in Echtzeit testen und debuggen.
- **Search** Nach Schlüsselwörtern im geschriebenen Code suchen.
- **Open Serial Monitor** Öffnet das Werkzeug „Serial Monitor“ als neuen Tab in der Konsole.



Abbildung 2.7.: Überblick

2.3.1. Sketchbook

Das Sketchbook ist der Ort, an dem Arduino-Code-Dateien - sogenannte Sketche - gespeichert werden. Diese Dateien haben die Dateiendung .ino. Um eine saubere Organisation zu gewährleisten, muss jeder Sketch in einem Ordner gespeichert werden, der genau denselben Namen wie die Sketch-Datei trägt. Beispiel: Ein Sketch namens MySketch.ino muss in einem Ordner namens MySketch gespeichert werden. Diese Namenskonvention ist notwendig, damit die Arduino IDE die Sketche korrekt erkennen und verwalten kann 2.8.

In der Regel werden Sketche in einem Ordner namens Arduino gespeichert, der sich im Dokumente-Verzeichnis des Systems befindet.

Um auf das Sketchbook zuzugreifen, kann in der Arduino IDE das Ordnersymbol in der Seitenleiste ausgewählt werden. Dadurch wird das Verzeichnis geöffnet, in dem die Sketche gespeichert sind, und ein effizienter Zugriff sowie eine einfache Verwaltung der Sketche ermöglicht.

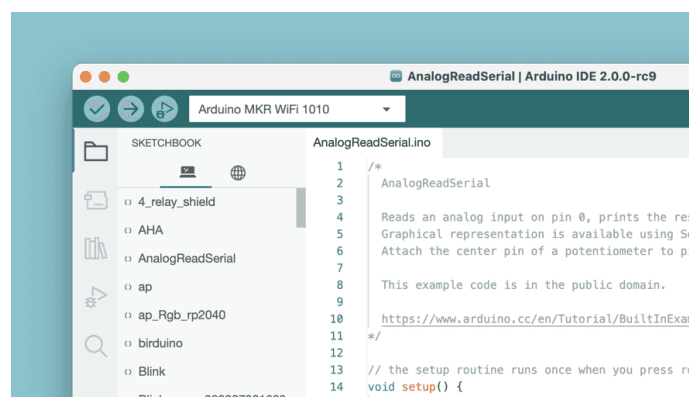


Abbildung 2.8.: Sketchbook

2.3.2. Boards Manager

Der Boards Manager kann über folgende Schritte in der Arduino IDE aufgerufen werden: **Tools** » **Board** » **Board Manager**.

Der Boards Manager ermöglicht die Installation verschiedener Board-Pakete, wie in Abbildung 2.9 dargestellt.

Ein Board-Paket enthält die notwendigen Dateien und Anweisungen, um Code für bestimmte Arduino-Boards zu kompilieren und hochzuladen.

Es gibt verschiedene Board-Pakete, z.B. avr, samd und megaavr. Jedes dieser Pakete unterstützt unterschiedliche Familien von Arduino-Boards:

- avr wird für ältere Boards wie den Arduino Uno verwendet,
- samd ist für neuere Boards wie den Arduino Zero gedacht.

Durch die Verwendung des Boards Managers wird sichergestellt, dass die richtigen Werkzeuge und Dateien für das gewählte Board installiert sind.

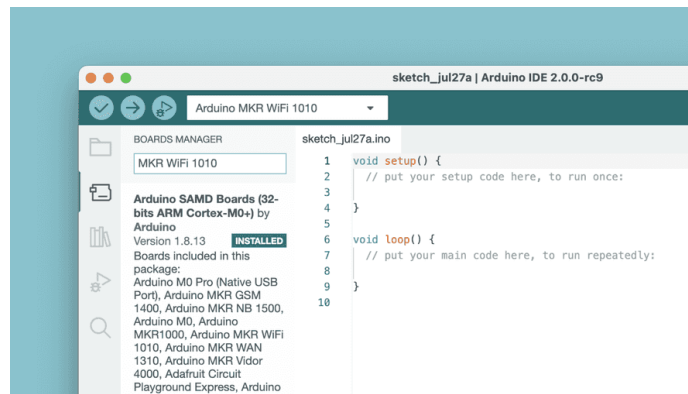


Abbildung 2.9.: Boards Manager

2.3.3. Library Manager

Der Library Manager kann in der Arduino IDE über folgende Schritte aufgerufen werden: **Tools** » **Manage Libraries**.

Der Bibliotheksverwalter ermöglicht es Nutzerinnen und Nutzern, eine große Auswahl an Bibliotheken zu durchsuchen und zu installieren. Bibliotheken erweitern die Grundfunktionen von Arduino und erleichtern Aufgaben wie:

- das Steuern von Servomotoren,
- das Auslesen von Daten bestimmter Sensoren oder
- die Arbeit mit Modulen wie WLAN.

Diese Bibliotheken enthalten vorgefertigten Code, um bestimmte Hardware-Komponenten anzusteuern und komplexe Aufgaben zu vereinfachen. Dadurch wird die Entwicklung von Arduino-Projekten schneller und effizienter – wie in 2.10 dargestellt.



Abbildung 2.10.: Library Manager

2.4. Integration und Verwendung einer benutzerdefinierten Bibliothek in der Arduino IDE

Bei der Entwicklung von eingebetteten Systemen auf der Arduino-Plattform sind Bibliotheken unerlässlich, um komplexe Funktionen zu abstrahieren und zu vereinfachen. Eine benutzerdefinierte Bibliothek ist besonders dann sinnvoll, wenn eine bestimmte Funktionalität oder eine spezifische Hardware-Schnittstelle häufig in verschiedenen Projekten verwendet wird.

Dieser Abschnitt beschreibt den Ablauf zum Erstellen, Installieren und Verwenden einer eigenen Bibliothek innerhalb eines Arduino-Projekts - in der Arduino Integrated Development Environment (IDE).

2.4.1. Erstellen der Bibliotheksdateien

Eine gut strukturierte, benutzerdefinierte Arduino-Bibliothek besteht typischerweise aus mindestens zwei zentralen Komponenten:

Header-Datei (.h): Diese Datei enthält die Deklarationen von Klassen, Funktionen und Variablen, die die Schnittstelle (Interface) der Bibliothek definieren. Sie stellt die notwendigen Definitionen bereit, damit externe Programme die Funktionen und Klassen verwenden können.

Quellcode-Datei (.cpp): Diese Datei enthält die tatsächliche Implementierung der in der Header-Datei deklarierten Funktionen und Methoden. Sie definiert das Verhalten der Funktionen und Klassen.

Die Struktur der Bibliothek ist so organisiert, dass eine klare Trennung zwischen Deklaration und Implementierung ihrer Funktionalitäten besteht.

2.4.2. Installation und Integration der Arduino IDE

Sobald die benutzerdefinierte Bibliothek erstellt wurde, muss sie korrekt installiert und in die Arduino IDE integriert werden, um in Projekten verwendet werden zu können. Um eine benutzerdefinierte Bibliothek in der Arduino IDE zu installieren, befolgen Sie diese Schritte:

1. **Bibliotheksordner finden:** Die Arduino IDE sucht nach Bibliotheken im Ordner `libraries`, der sich im Sketchbook-Verzeichnis befindet. Der typische Pfad zu diesem Verzeichnis lautet:

`C:/Users/<Benutzername>/Documents/Arduino/libraries`

2. Benutzerdefinierten Bibliotheksordner kopieren: Kopieren Sie den Ordner ihrer Bibliothek, z.B. Nano33BLESenseLED, in das Verzeichnis `libraries`. Der vollständige Pfad sollte dann wie folgt aussehen:

`C:/Users/<Benutzername>/Documents/Arduino/libraries/Nano33BLESenseLED`

3. Arduino IDE neu starten: Nachdem die Bibliothek in das richtige Verzeichnis kopiert wurde, starten sie die Arduino IDE neu, damit sie die neu hinzugefügte Bibliothek erkennt.

2.4.3. Verwendung symbolischer Verknüpfungen zur Bibliotheksintegration

In manchen Fällen kann es bequemer oder notwendig sein, die Bibliotheksdateien außerhalb des Standardbibliotheksverzeichnis zu speichern. Zum Beispiel, wenn die Bibliotheken in einem GitHub-Repository gespeichert sind oder zur besseren Versionskontrolle, kann der Befehl `mklink` verwendet werden, um eine symbolische Verknüpfung zu erstellen.

Dies ermöglicht es der Arduino IDE, auf die Bibliotheksdateien zuzugreifen, als ob sie sich im Standardverzeichnis befinden, ohne dass die Dateien dupliziert werden müssen.

2.4.4. Windows-Tool `mklink` zum Erstellen symbolischer Verknüpfungen

Symbolische Verknüpfungen ermöglichen es, Bibliotheken an einem anderen Speicherort abzulegen, während sie von der Arduino IDE trotzdem so behandelt werden, als befänden sie sich im Standardverzeichnis `libraries`.

Dies kann besonders nützlich sein in versionierten Umgebungen, z.B. bei der Verwendung von Git, oder um Bibliotheken besser zu organisieren, ohne Dateien zu duplizieren.

Schritte zum Erstellen einer symbolischen Verknüpfung:

1. **Öffnen der Eingabeaufforderung mit Administratorrechten:** Drücken Sie `Win + S` und suchen Sie nach `cmd`, klicken Sie dann auf `Eingabeaufforderung` und wählen Sie `Als Administrator ausführen` wie in 2.11 gezeigt.

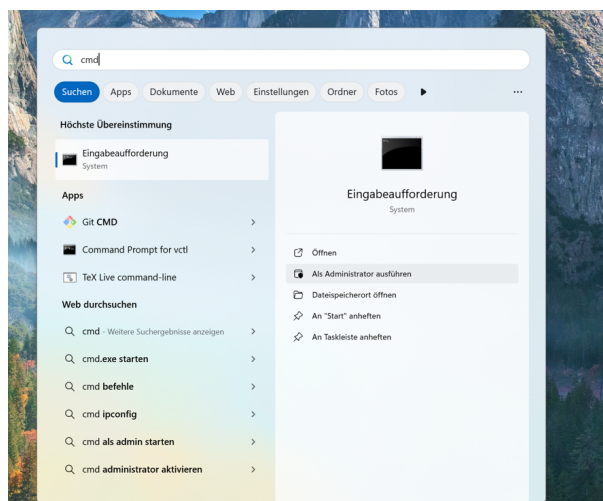


Abbildung 2.11.: Eingabeaufforderung mit Administrator Rechten

2. **Führen Sie den Befehl `mklink` aus:** Der Befehl zum Erstellen einer symbolischen Verknüpfung lautet: `mklink /D TTargetPathSourcePath`

- **TargetPath:** Der Ort, an dem die Arduino IDE die Bibliothek erwartet – also im Ordner `libraries`.
- **SourcePath:** Der tatsächliche Speicherort der Bibliothek.

In diesem Beispiel befindet sich die Bibliothek unter:

`C:/.../.../.../GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/Nano33BLESenseLED`

Der Zielpfad im libraries-Ordner der Arduino IDE lautet:

`C:/.../.../.../Arduino/libraries/Nano33BLESenseLED`

Der vollständige Command ist in 2.12 zu sehen.

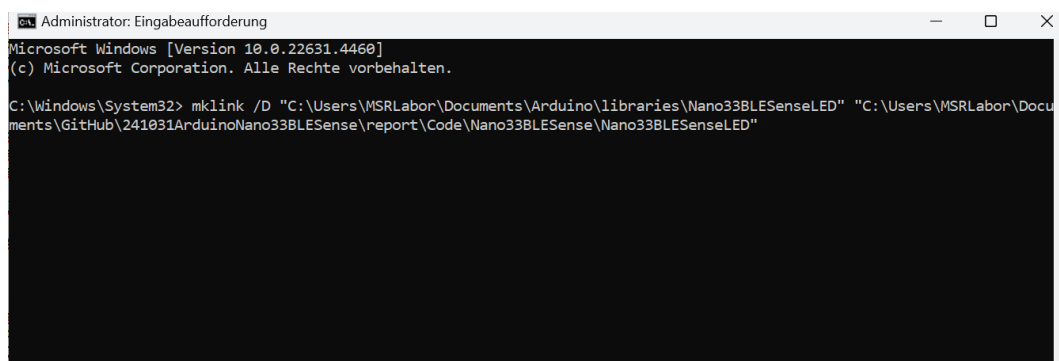


Abbildung 2.12.: Eingabeaufforderung

Überprüfen der symbolischen Verknüpfung: Nach dem Ausführen des Befehls navigieren Sie zum Ordner `libraries`. Dort sollte ein Ordner mit dem Namen `Nano33BLESenseLED` vorhanden sein, der als symbolische Verknüpfung fungiert und auf den tatsächlichen Speicherort der Bibliothek verweist.

2.4.5. Überprüfen der Verfügbarkeit der Bibliothek in der Arduino IDE

Um sicherzustellen, dass die benutzerdefinierte Bibliothek erfolgreich integriert wurde und von der Arduino IDE erkannt wird, befolgen Sie diese Schritte:

1. Arduino IDE neu starten: Falls die Arduino IDE während der Installation der Bibliothek oder nach dem Erstellen der symbolischen Verknüpfung geöffnet war, schließen und öffnen Sie sie erneut. Dadurch wird die Liste der verfügbaren Bibliotheken neu geladen.
2. Bibliotheksmenü überprüfen: Navigieren Sie zu `Sketch` gehen Sie zu `Include Library` und wählen Sie die Bibliothek `Nano33BLESenseLED` aus, wie in 2.13 gezeigt.
3. Überprüfung in der Liste: Wenn die Bibliothek in der Liste erscheint, bedeutet das, dass sie von der IDE erfolgreich erkannt wurde. Falls nicht, überprüfen Sie nochmals die Verzeichnisstruktur und die symbolische Verknüpfung, um sicherzustellen, dass sie korrekt eingerichtet sind.

4. Testprogramm kompilieren: Schreiben Sie ein einfaches Testprogramm, das Funktionen oder Klassen aus der Bibliothek verwendet. Kompilieren Sie den Sketch, um sicherzustellen, dass keine Fehler auftreten – das bestätigt, dass die Bibliothek erfolgreich integriert wurde.

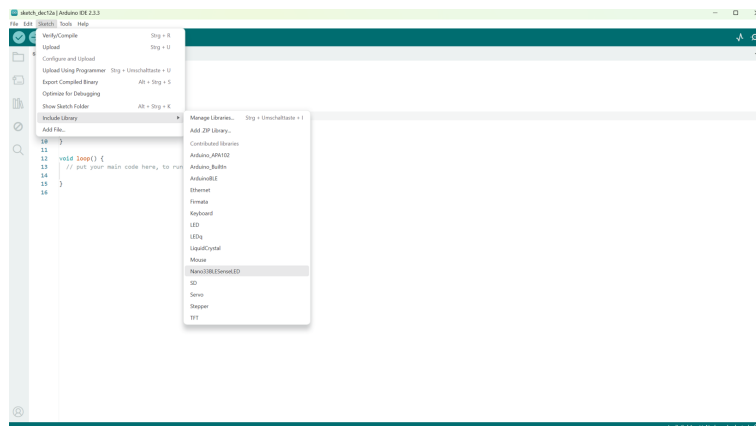


Abbildung 2.13.: Einbinden der Nano33BLESenseLED Bibliothek

Sobald das Programm ohne Fehler kompiliert, ist die Bibliothek einsatzbereit und kann bedenkenlos in Arduino-Projekten verwendet werden.

2.4.6. Serieller Monitor

Der Serieller Monitor kann über folgende Schritte aufgerufen werden: **Tool** **Serial Monitor**

Der Serieller Monitor ist ein Werkzeug, das es ermöglicht, Daten vom Board anzuzeigen, zum Beispiel mithilfe des Befehls `Serial.print()`.

Früher wurde dieses Tool in einem separaten Fenster geöffnet, ist aber jetzt in den Editor integriert, was es erleichtert, mehrere Instanzen gleichzeitig auf dem Computer auszuführen.

Der Serieller Monitor ist in 2.14 dargestellt.

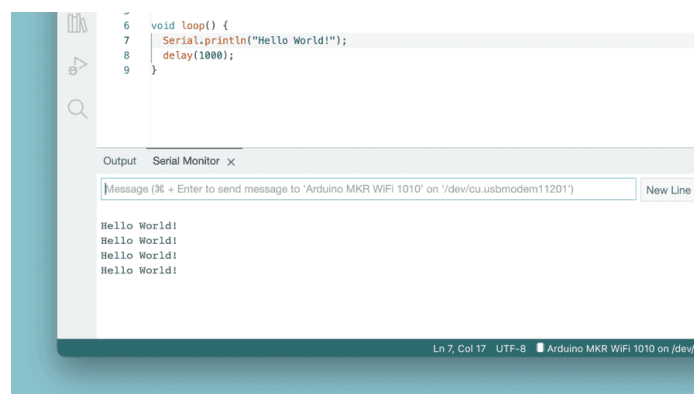


Abbildung 2.14.: Serieller Monitor

2.4.7. Serieller Plotter

Das Werkzeug **Serial Plotter** eignet sich hervorragend zur Visualisierung von Daten in Form von Diagrammen, z.B. zur Überwachung von Spannungsspitzen.

Es ermöglicht das gleichzeitige Beobachten mehrerer Variablen und bietet die Möglichkeit, bestimmte Datentypen gezielt ein- oder auszublenden.

2.5. Beispiele

Ein wesentlicher Bestandteil der Arduino Dokumentation ist die Sammlung von Beispielsketches, die mit verschiedenen Bibliotheken gebündelt sind.

Diese Beispiele zeigen die praktische Anwendung von Bibliotheksfunktionen und veranschaulichen deren typische Einsatzmöglichkeiten und Hauptfunktionen. Auch Bibliotheken, die mit bestimmten Board-Paketen geliefert werden, können eigene Beispielsketches enthalten.

Um auf diese Beispielsketches zuzugreifen – egal ob sie manuell installierten Bibliotheken oder mitgelieferten Board-Paketen zugeordnet sind – navigiere in der Arduino IDE zu: **File** » **Examples**, und wähle dann die gewünschte Bibliothek aus der Liste aus. Beispielsweise erscheint beim Anschluss eines UNO R4 WiFi-Boards eine Liste mit bezugsspezifischen Beispielen. Ein Beispielsketch mit einer vorgeladenen Tetris-Animation kann aufgerufen werden über: **File** » **Examples** » **LED Matrix** » **MatrixIntro** und auf das angeschlossene Board hochgeladen werden.

Dieses Beispiel zeigt, wie der mitgelieferte Code des Boards in der Praxis genutzt werden kann und hilft den Nutzern, verschiedene Hardwarefunktionen direkt zu steuern und kennenzulernen.

In Abbildung 2.15 wird die Beispielliste angezeigt, wenn ein UNO R4 WiFi-Board mit dem Computer verbunden ist.

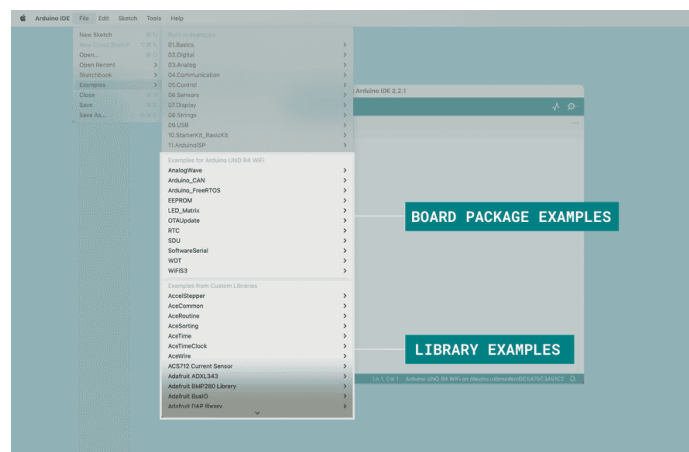


Abbildung 2.15.: Beispiele

2.6. Debuggen

Das Tool **Debugger** in der Arduino IDE 2.3.3 ist dafür konzipiert, Programme beim Testen und Debuggen zu unterstützen, indem es ermöglicht, die Codeausführung auf kontrollierte Weise Schritt für Schritt durchzugehen.

Dieses Tool erlaubt es den Nutzern, Haltepunkte zu setzen, den Code Zeile für Zeile durchzugehen und Variablen zu inspizieren, um Probleme wie Logikfehler oder falsche Variablenwerte zu identifizieren.

Der Debugger ermöglicht es, die Ausführung an bestimmten Punkten im Programm anzuhalten, um das Verhalten des Programms und den Speicherzustand im Detail zu untersuchen.

Diese Funktion verbessert den Entwicklungsprozess, indem sie es erleichtert, Fehler zur Laufzeit zu finden und zu beheben, anstatt sich ausschließlich auf print-Anweisungen oder Versuch-und-Irrtum zu verlassen 2.16.

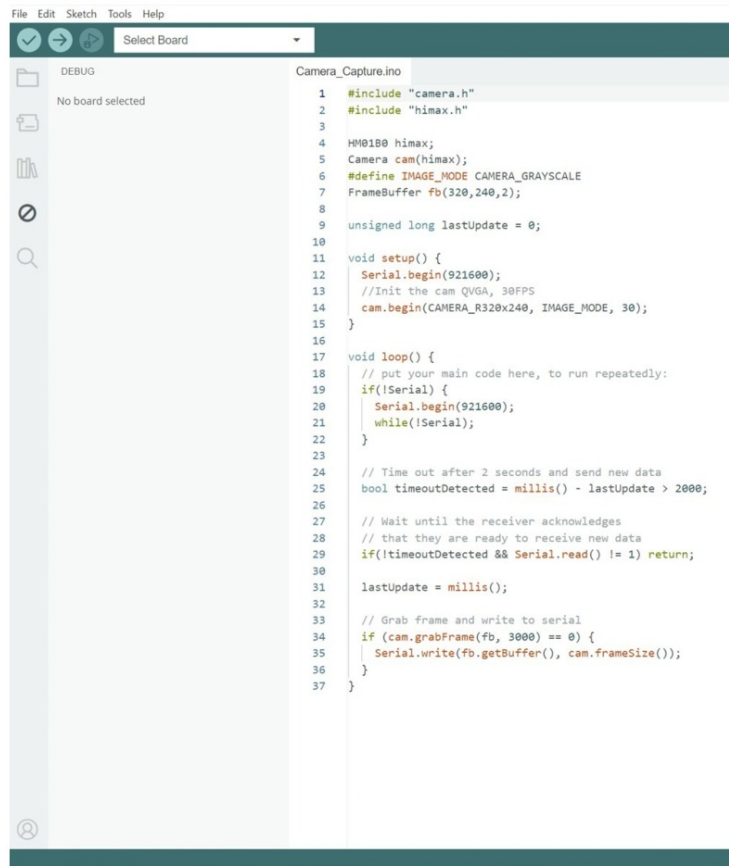


Abbildung 2.16.: Beispiel Debuggen

2.7. Autovervollständigung

Die Autovervollständigung ist eine zentrale Funktion der Arduino IDE Version 2, die das Programmieren erleichtert, indem sie Funktionen und Elemente aus der Arduino-API vorgeschlägt. Diese Funktion hilft dabei, Code schneller und genauer zu schreiben, was die Effizienz und Genauigkeit verbessert.

Damit die Autovervollständigung funktioniert, muss in der Arduino IDE das verwendete Board ausgewählt sein. Dadurch erkennt der Editor die richtigen Funktionen und Bibliotheken für das jeweilige Board und kann präzise Vorschläge machen 2.17.

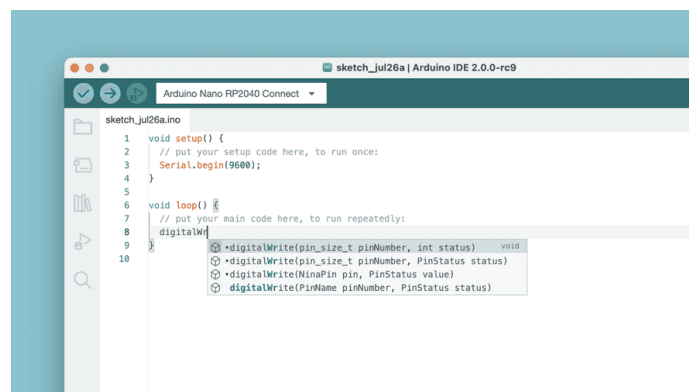


Abbildung 2.17.: Autovervollständigung

2.8. Bibliotheken und Modularität

Die Arduino IDE erlaubt es, Funktionen in Bibliotheken auszulagern und wiederzuverwenden. Viele Sensoren oder Module wie DHT11, LCD oder Motorshields bringen eigene Bibliotheken mit, die über die Bibliotheksverwaltung eingebunden werden können [Smi11]:

```
#include <ArduinoBLE.h>
```

Die Verwendung von Bibliotheksfunktionen ermöglicht es, größere Programme in modulare Teilfunktionen zu gliedern und dadurch Struktur sowie Wartbarkeit deutlich zu verbessern. So kann beispielsweise mit der Funktion `BLE.setLocalName()` ein Bluetooth-Name für die Applikation festgelegt werden, ohne dass die zugrunde liegenden Befehle und Abläufe im Detail bekannt sein müssen.

3. Arduino Web Editor

3.1. Using the Arduino Web Editor

The Arduino Web Editor allows you to write code and upload sketches to any official Arduino board directly from your web browser (Chrome, Firefox, Safari and Edge). [Ard24]

This IDE (Integrated Development Environment) is part of Arduino Create, an online platform that enables developers to write code, access tutorials, configure boards, and share projects. Designed to provide users with a continuous workflow, Arduino Create connects the dots between each part of a developer's journey from inspiration to implementation. Meaning, you now have the ability to manage every aspect of your project right from a single dashboard.

The Arduino Web Editor is hosted online, therefore it is always be up-to-date with the latest features and support for new boards. This IDE lets you write code and save it to the Cloud, always backing it up and making it accessible from any device. It automatically recognizes any Arduino board connected to your PC, and configures itself accordingly.

All you need to get started is an Arduino account. The following steps can guide you to start using the Arduino Web Editor: [Ard24]

3.1.1. Using the online IDE

After logging in, you are ready to start using the Arduino Web Editor. The web app is divided into three main columns. 3.1

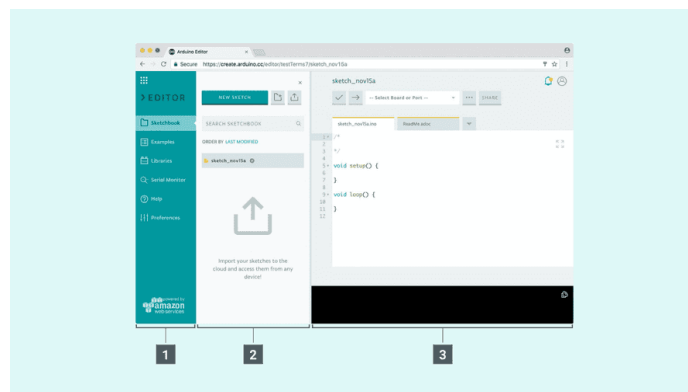


Abbildung 3.1.: Arduino Web Editor

The Arduino Web Editor's interface is as follows:

1. The first column lets you navigate between:
 - **Your Sketchbook:** a collection of all your sketches (a sketch is a program you upload on your board).
 - **Examples:** read-only sketches that demonstrate all the basic Arduino commands (built-in tab), and the behavior of your libraries (from the libraries tab).

- **Libraries:** packages that can be included to your sketch to provide extra functionalities.
 - **Serial monitor:** a feature that enables you to monitor, receive and send data to and from your board via the USB cable.
 - **Help:** helpful links and a glossary about Arduino terms.
 - **Preferences:** options to customize the look and behavior of your editor, such as text size and color theme.
2. The second column views the content of the chosen option.
 3. The third column, the code area, is the one you will use the most. Here, you can write code, verify it and upload it to your boards, save your sketches on the Cloud, and share them with anyone you want.

Now that we are all set up, let's try to make the board blink!

1. Connect your Arduino or Genuino board to your computer. Boards and serial ports are auto-discovered and selectable in a single dropdown. Pick the Arduino/Genuino board you want to upload to from the list at the top of the third column.[Ard24]
2. Let's try with an example: Choose Examples on the menu on the left (first column), then Basic and Blink. The Blink sketch is now displayed in the code area.
3. To upload it to your board, press the "Upload" button near the dropdown menu. While the code is verifying and uploading, a "BUSY" label replaces the upload button. If the upload is successful, the message "Success: done uploading" will appear in the bottom output area.
4. Once the upload is complete, you should see on your board the yellow LED with an L next to it start blinking. You can adjust the speed of blinking by changing the delay number in the parenthesis to 100, and upload the Blink sketch again. Now the LED should blink much faster ??.



Abbildung 3.2.: finding-an-example

3.2. Arduino Libraries

The process of setting up libraries on the online IDE (Arduino Cloud Editor) is quite similar to the offline one: [Ard25b; Ard25a]

1. Login to the Arduino Cloud.

2. Create or open a sketch.
3. Open the LLibrariesttab from the left menu, and search for libraries. The list displays read-only libraries, authored and maintained by the Arduino team and its partners.
4. When you find the library, you can add it to your sketch by selecting the "Include" button. You can also see the related examples, and select a specific version, if available.
5. If you can't find a specific library on the list, you can search every existing library through the search bar. You also have the option to add them to your favorites list by clicking on the star next to the library you want. Once you star a library, you can view it under the ffavoritesttab and use its examples (if available).

3.3. Data Security

- **Encryption:** The web version of Arduino IDE should use strong encryption protocols to secure data transmitted between the user's browser and the server, preventing unauthorized access.
- **Secure Storage:** User data, including login credentials and project files, should be encrypted at rest to protect against unauthorized access or tampering.

3.3.1. Authentication and Authorization

- **Strong Password Policies:** Enforce password complexity and offer multi-factor authentication to enhance account security.
- **Role-Based Access Control:** Implement role-based access to ensure users only access necessary resources, preventing unauthorized access to sensitive data.

3.3.2. Secure Code Execution

- **Sandboxing:** Execute user-uploaded code in a sandboxed environment to restrict access to system resources and prevent malicious actions.
- **Code Validation:** Validate user code to detect and prevent vulnerabilities like code injection or buffer overflows before execution.

3.3.3. Protection Against Malware

- **Code Scanning:** Use code scanning to detect and block malicious code by analyzing for malware signatures or suspicious patterns.
- **Antivirus Integration:** Integrate antivirus solutions to scan uploaded files for viruses or malicious content before execution.

3.3.4. Privacy Policies

- **Transparency:** Maintain clear privacy policies outlining data collection, storage, usage, retention periods, and third-party sharing practices.
- **User Consent:** Inform users about privacy practices and obtain explicit consent for data processing, allowing them to review and modify privacy settings.

3.3.5. Regular Updates and Maintenance

- **Patch Management:** Regularly update software components, libraries, and dependencies to address security vulnerabilities promptly.
- **Security Audits:** Conduct regular security audits and penetration testing to identify and remediate potential weaknesses.

3.3.6. User Awareness

- **Security Education:** Provide resources on secure coding practices to help users avoid vulnerabilities like XSS, SQL injection, and insecure object references.
- **Security Alerts:** Notify users about security-related issues, such as suspicious login attempts or unauthorized access.

Teil II.

Hardware

4. Arduino Uno

4.1. Arduino Allgemein

Arduino ist eine Open-Source-Plattform für Mikrocontroller. Sie besteht aus programmierbarer Hardware und einer Entwicklungsumgebung, mit der Programme geschrieben und auf ein Board übertragen werden können [Ard26a]. Arduino-Boards werden häufig für Steuerungen, Messaufgaben, Prototypen und Lernprojekte verwendet [Ard26c]. Ein Arduino kann Eingangssignale verarbeiten und Ausgangssignale erzeugen. Eingangssignale können zum Beispiel von Tastern, Sensoren oder Modulen kommen. Ausgangssignale können LEDs, Motoren, Displays oder andere Bauteile steuern [Ard25c]. Die Programmierung erfolgt meist über die Arduino IDE. Das Programm wird am Computer geschrieben, über USB auf das Board übertragen und danach direkt vom Mikrocontroller ausgeführt [Ard25c]. Arduino-Programme verwenden typische Funktionen wie `setup()` und `loop()` [Ard26a].

4.2. Arduino Uno

Der Arduino Uno ist ein Mikrocontroller-Board auf Basis des ATmega328P. Er besitzt 14 digitale Ein- und Ausgänge, davon 6 mit PWM-Funktion, 6 analoge Eingänge, eine USB-Schnittstelle, eine Power-Buchse, einen ICSP-Header und einen Reset-Taster [Ard26c][Ard26b].

Der Arduino Uno eignet sich gut für den Einstieg in Elektronik und Programmierung. Er enthält die wichtigsten Anschlüsse, um einfache Schaltungen aufzubauen und Programme direkt auf dem Mikrocontroller auszuführen [Ard26c].

Wichtige Eigenschaften:

- Mikrocontroller: ATmega328P
- Betriebsspannung: 5 V
- Empfohlene Eingangsspannung: 7 V bis 12 V
- Digitale I/O-Pins: 14
- PWM-Pins: 6
- Analoge Eingänge: 6
- Flash-Speicher: 32 KB
- SRAM: 2 KB
- EEPROM: 1 KB
- Taktfrequenz: 16 MHz [Ard26b]

Beschreibung mit Tikz Bild

4.3. Module

4.3.1. Mikrocontroller ATmega328P

Hintergrundwissen

Der ATmega328P ist der Hauptbaustein des Arduino Uno. Er führt das hochgeladene Programm aus, verarbeitet Eingangssignale und steuert Ausgangssignale [Ard26b]. Auf diesem Mikrocontroller laufen später die Funktionen `setup()` und `loop()` [Ard26a]. Der Mikrocontroller enthält Speicher für das Programm und Speicher für Daten während der Ausführung. Das Programm bleibt auch nach dem Ausschalten gespeichert, weil es im Flash-Speicher abgelegt wird [Ard26b].

Beschreibung

Der ATmega328P ist auf dem Arduino-Uno-Board verbaut. Er ist mit den digitalen Pins, den analogen Eingängen, der USB-Schnittstelle, der Spannungsversorgung und dem Taktgeber verbunden [Ard26b][Ard25c].

Das Programm wird über die Arduino IDE auf den Mikrocontroller übertragen. Nach dem Hochladen startet der Mikrocontroller das Programm automatisch [Ard25c].

Eigenschaften

- Hauptmikrocontroller des Arduino Uno
- 8-Bit-Mikrocontroller
- Taktfrequenz: 16 MHz
- Flash-Speicher: 32 KB
- SRAM: 2 KB
- EEPROM: 1 KB [Ard26b]
- Führt den Arduino-Code aus [Ard26a]

4.3.2. Eingebaute LED

Hintergrundwissen

Der Arduino Uno besitzt eine eingebaute LED. Diese LED ist mit dem digitalen Pin `D13` verbunden [Ard25c]. Sie wird häufig für erste Funktionstests verwendet [Ard26c]. Mit der eingebauten LED kann geprüft werden, ob das Board korrekt programmiert wurde und ob ein Ausgangspin gesteuert werden kann [Ardf].

Beschreibung

Die LED kann direkt über Pin `D13` geschaltet werden. Es wird keine externe Verdrahtung benötigt [Ard25c]. Wird der Pin auf `HIGH` gesetzt, leuchtet die LED. Wird der Pin auf `LOW` gesetzt, ist die LED ausgeschaltet [Ardf].

Die eingebaute LED eignet sich besonders für das Blink-Beispiel und einfache Tests mit `digitalWrite()` [Ardf].

Eigenschaften

- Mit Pin `D13` verbunden [Ard25c]
- Digital steuerbar [Ardf]
- Für einfache Funktionstests geeignet [Ard26c]

4.3.3. Reset-Taster

Hintergrundwissen

Der Reset-Taster ist auf dem Arduino Uno eingebaut [Ard26c]. Er startet den Mikrocontroller neu. Dabei wird das gespeicherte Programm nicht gelöscht [Ard25c]. Ein Reset ist hilfreich, wenn ein Programm neu gestartet werden soll oder sich das Board nicht wie erwartet verhält [Ard25c].

Beschreibung

Beim Drücken des Reset-Tasters wird der ATmega328P zurückgesetzt [Ard25c]. Danach startet das Programm wieder von Anfang an. Zuerst wird `setup()` einmal ausgeführt. Danach läuft `loop()` dauerhaft weiter [Ard26a].

Eigenschaften

- Startet den Mikrocontroller neu
- Löscht den gespeicherten Code nicht
- Hilfreich bei Tests und Fehlern [Ard25c]

4.3.4. USB-Schnittstelle

Hintergrundwissen

Die USB-Schnittstelle verbindet den Arduino Uno mit dem Computer [Ard25c]. Über USB wird das Programm auf das Board übertragen. Außerdem kann der Arduino Uno darüber mit Spannung versorgt werden [Ard26b].

Die USB-Verbindung wird auch für den seriellen Monitor verwendet. Damit können Werte und Zustände vom Arduino an den Computer gesendet werden [Ard26a].

Beschreibung

Der Arduino Uno wird über ein USB-Kabel mit dem Computer verbunden. Die Arduino IDE nutzt diese Verbindung zum Hochladen des Programms [Ard25c]. Für Ausgaben im seriellen Monitor werden Funktionen wie `Serial.begin()`, `Serial.print()` und `Serial.println()` verwendet [Ard26a].

Eigenschaften

- Verbindung zum Computer
- Hochladen von Programmen [Ard25c]
- Stromversorgung über USB möglich [Ard26b]
- Nutzung des seriellen Monitors [Ard26a]
- Wichtig für Tests und Fehlersuche [Ard25c]

4.3.5. Spannungsversorgung

Hintergrundwissen

Der Arduino Uno kann über USB, die Power-Buchse oder den Pin `VIN` mit Spannung versorgt werden [Ard26b]. Auf dem Board befinden sich Spannungsregler, die die benötigte Betriebsspannung bereitstellen [Ard25c].

Die Betriebsspannung des Arduino Uno beträgt 5 V. Für eine externe Versorgung wird eine Eingangsspannung von 7 V bis 12 V empfohlen [Ard26b].

Beschreibung

Für einfache Tests reicht meistens die Versorgung über USB [Ard25c]. Für eigenständige Projekte kann eine externe Spannungsquelle über die Power-Buchse oder VIN verwendet werden [Ard26b].

Die Pins 5V und 3.3V können kleine externe Bauteile mit Spannung versorgen [Ard26b]. Große Verbraucher wie Motoren oder starke LEDs sollten nicht direkt über diese Pins betrieben werden [Ard25c].

Eigenschaften

- Versorgung über USB möglich
- Versorgung über Power-Buchse möglich
- Versorgung über VIN möglich
- Betriebsspannung: 5 V
- Empfohlene Eingangsspannung: 7 V bis 12 V
- 5V-Pin für kleine 5-V-Verbraucher
- 3.3V-Pin für kleine 3,3-V-Verbraucher [Ard26b]

4.4. Testtaster

4.4.1. Hintergrundwissen

Der Testtaster wird verwendet, um zu prüfen, ob der Arduino Uno digitale Eingangssignale korrekt erkennt. Er ist kein eingebauter Sensor des Arduino Uno, sondern ein externer Taster für einen einfachen Funktionstest [Arde].

Ein Taster besitzt zwei Zustände: gedrückt und nicht gedrückt. Der Arduino liest diese Zustände über einen digitalen Eingangspin aus [Arde].

4.4.2. Beschreibung

Der Taster wird zwischen einen digitalen Pin und GND angeschlossen. Für den Test kann zum Beispiel Pin D2 verwendet werden [Ard26b].

Der Pin wird im Programm mit INPUT_PULLUP eingestellt. Dadurch liegt der Eingang ohne Tastendruck auf HIGH. Wird der Taster gedrückt, wird der Eingang mit GND verbunden und der Arduino liest LOW [Ardi][Arde].

Anschluss:

- Eine Seite des Tasters an D2 [Ard26b]
- Andere Seite des Tasters an GND [Ard26b]
- Interner Pull-up-Widerstand über INPUT_PULLUP [Ardi]

4.4.3. Eigenschaften

- Dient nur als Funktionstest
- Prüft einen digitalen Eingang
- Zwei Zustände: gedrückt und nicht gedrückt
- Wird mit digitalRead() ausgelesen [Arde]

- Funktioniert gut mit internem Pull-up-Widerstand [Ardi]
- Kein eingebaute Bestandteil des Arduino Uno [Ard26c]

Erwartetes Verhalten:

- Nicht gedrückt: HIGH
- Gedrückt: LOW [Ardi]

Wenn der Wert unzuverlässig ist, sollte geprüft werden:

- Taster korrekt mit D2 und GND verbunden [Ard26b]
- Pin im Programm mit INPUT_PULLUP eingestellt [Ardi]
- Taster mechanisch funktionsfähig
- Kontaktprellen im Programm berücksichtigt

4.4.4. Testcode

Manual

Der Testcode prüft, ob der Arduino einen Tastendruck erkennt. Der Taster wird zwischen den digitalen Pin D2 und GND angeschlossen. Eine Seite des Tasters kommt an D2, die andere Seite an GND.

Im Programm wird der Eingang mit `pinMode()` als `INPUT_PULLUP` eingestellt. Dadurch wird kein externer Widerstand benötigt. Der Arduino liest bei nicht gedrücktem Taster den Zustand HIGH. Beim Drücken verbindet der Taster D2 mit GND, wodurch der Zustand LOW gelesen wird.

Der Zustand wird mit `digitalRead()` ausgelesen und mit `Serial.println()` im seriellen Monitor angezeigt [Ardi][Arde][Ard26a].

Code

Listing 4.1.: Test Button

```

1000 \begin{lstlisting}[language=C++]
1001 /**
1002  * @file TestButton.ino
1003  * @brief Tests a push button connected to digital pin D2.
1004  *
1005  * The button is connected between digital pin D2 and GND.
1006  * The internal pull-up resistor is enabled with INPUT_PULLUP.
1007  * Therefore, the input reads HIGH when the button is not pressed
1008  * and LOW when the button is pressed.
1009  */
1010 const int buttonPin = 2; ///< Digital input pin connected to the button
1011
1012 /**
1013  * @brief Initializes serial communication and configures the button pin
1014  */
1015 void setup() {
1016     Serial.begin(115200);
1017     pinMode(buttonPin, INPUT_PULLUP);
1018 }
1019
1020 /**
1021  * @brief Reads the button state and prints it to the serial monitor.
1022  */

```

```

1024 void loop() {
      int buttonState = digitalRead(buttonPin);
1026
      if (buttonState == LOW) {
1028         Serial.println("Button pressed");
      } else {
1030         Serial.println("Button not pressed");
      }
1032
      delay(200);
1034 }
\end{lstlisting}

```

../Code/TestButton/TestButton.ino

4.5. PINs

4.5.1. Hintergrundwissen

Die Pins sind die elektrischen Anschlüsse des Arduino Uno. Über sie wird der Mikrocontroller mit externen Bauteilen verbunden. Der Arduino Uno besitzt 14 digitale Ein- und Ausgänge, 6 analoge Eingänge und mehrere Pins für Versorgung und Kommunikation [Ard26c][Ard26b].

Die Pinbelegung ist im Datenblatt des Arduino Uno beschrieben. Dort sind die digitalen Pins, analogen Eingänge, Versorgungspins und Kommunikationsschnittstellen des Boards aufgeführt [Ard26b].

Der Arduino Uno arbeitet mit einer Betriebsspannung von 5 V. Deshalb verwenden die digitalen Pins 5-V-Logikpegel. Eine zu hohe Spannung an einem Eingangspin kann das Board beschädigen [Ard26b].

4.5.2. Digitale Pins

Hintergrundwissen

Die digitalen Pins des Arduino Uno sind mit D0 bis D13 beschriftet. Sie können als Eingang oder Ausgang verwendet werden [Ard26c][Ard26b].

Digitale Pins arbeiten mit zwei Zuständen: HIGH und LOW. Bei einem Ausgang entspricht HIGH einem hohen Spannungspegel und LOW einem niedrigen Spannungspegel [Ardf]. Beim Arduino Uno liegt der Logikpegel bei 5 V [Ard26b].

Beschreibung

Ein digitaler Pin wird im Programm mit pinMode() als Eingang oder Ausgang festgelegt [Ardi]. Danach kann der Pin gelesen oder gesetzt werden.

Als Eingang wird ein digitaler Pin zum Beispiel für Taster oder digitale Signale verwendet. Der Zustand wird mit digitalRead() ausgelesen [Arde].

Als Ausgang kann ein digitaler Pin LEDs, Steuersignale oder kleine Lasten ansteuern. Der Zustand wird mit digitalWrite() auf HIGH oder LOW gesetzt [Ardf].

Eigenschaften

- Pins: D0 bis D13 [Ard26c]
- Nutzung als Eingang oder Ausgang [Ardi]
- Logikzustände: HIGH und LOW [Arde]
- Logikspannung: 5 V [Ard26b]

- Einstellung mit `pinMode()` [Ardi]
- Auslesen mit `digitalRead()` [Arde]
- Setzen mit `digitalWrite()` [Ardf]

4.5.3. PWM-Pins

Hintergrundwissen

Der Arduino Uno besitzt 6 digitale Pins, die als PWM-Ausgänge verwendet werden können [Ard26c]. PWM steht für Pulsweitenmodulation. Dabei wird ein digitaler Ausgang sehr schnell ein- und ausgeschaltet [Ard26e].

PWM wird verwendet, wenn ein Ausgang nicht nur ein- oder ausgeschaltet werden soll. Typische Anwendungen sind das Dimmen einer LED oder das Steuern der Geschwindigkeit eines Motors über einen Motortreiber [Ard26e].

Beschreibung

Beim Arduino Uno unterstützen die Pins D3, D5, D6, D9, D10 und D11 PWM [Ard26c][Ard26b]. Diese Pins sind auf dem Board meist mit einem Tilde-Zeichen markiert [Ard25c].

PWM wird im Programm mit `analogWrite()` genutzt [Ard26e]. Obwohl die Funktion `analogWrite()` heißt, wird dabei kein echter analoger Spannungswert ausgegeben. Es handelt sich um ein digitales PWM-Signal [Ard26e].

Der Wert bei `analogWrite()` liegt zwischen 0 und 255. Der Wert 0 bedeutet dauerhaft aus. Der Wert 255 bedeutet dauerhaft an. Werte dazwischen erzeugen ein schnelles Ein- und Ausschalten [Ard26e].

Eigenschaften

- PWM-Pins: D3, D5, D6, D9, D10, D11 [Ard26c]
- Nutzung mit `analogWrite()`
- Wertebereich: 0 bis 255
- Geeignet für LED-Helligkeit
- Geeignet für Motorsteuerung über Treiber
- Gibt kein echtes analoges Signal aus [Ard26e]

4.5.4. Analoge Eingänge

Hintergrundwissen

Die analogen Eingänge des Arduino Uno sind mit A0 bis A5 beschriftet. Sie messen Spannungen und wandeln diese in digitale Werte um [Ard26c][Ard26d].

Der Arduino Uno besitzt einen Analog-Digital-Wandler. Dieser wandelt eine Eingangsspannung in einen Zahlenwert um. Der Wertebereich bei `analogRead()` reicht normalerweise von 0 bis 1023 [Ard26d].

Beschreibung

Analoge Eingänge werden mit `analogRead()` ausgelesen. Ein Messwert von 0 entspricht ungefähr 0 V. Ein Messwert von 1023 entspricht ungefähr der verwendeten Referenzspannung [Ard26d].

Analoge Eingänge werden zum Beispiel für Potentiometer, Fotowiderstände, analoge Temperatursensoren oder Spannungsteiler verwendet [Ard26d].

Die analogen Pins können bei Bedarf auch als digitale Pins verwendet werden [Ard26b]. Dann entsprechen sie zusätzlichen digitalen Eingängen oder Ausgängen.

Eigenschaften

- Analoge Eingänge: A0 bis A5 [Ard26c]
- Messbereich abhängig von der Referenzspannung
- Wertebereich bei `analogRead()`: 0 bis 1023
- Auflösung: 10 Bit
- Geeignet für analoge Sensorwerte [Ard26d]
- Können auch als digitale Pins verwendet werden [Ard26b]

4.5.5. Versorgungspins

Hintergrundwissen

Die Versorgungspins liefern Spannungen oder dienen als Masseanschluss. Sie werden verwendet, um kleine externe Schaltungen, Sensoren oder Module mit Strom zu versorgen [Ard26b].

Diese Pins sind keine normalen Ein- oder Ausgänge. Sie dienen nur zur Stromversorgung oder als Bezugspunkt [Ard25c].

Beschreibung

Der Pin 5V liefert die geregelte 5-V-Spannung des Boards. Der Pin 3.3V liefert eine 3,3-V-Spannung für kleine Verbraucher. GND ist der Masseanschluss. VIN kann zur externen Versorgung des Boards verwendet werden [Ard26b].

Bei allen angeschlossenen Bauteilen muss GND mit dem Arduino verbunden sein. Ohne gemeinsame Masse können Signale nicht zuverlässig erkannt werden [Ard25c].

Eigenschaften

- 5V: geregelte 5-V-Versorgung
- 3.3V: 3,3-V-Versorgung für kleine Verbraucher
- GND: Masseanschluss
- VIN: Eingang für externe Versorgungsspannung [Ard26b]
- Nicht für große Verbraucher geeignet
- Wichtig für gemeinsame Masseverbindung [Ard25c]

4.5.6. Serielle Pins D0 und D1

Hintergrundwissen

Die Pins D0 und D1 sind digitale Pins mit einer zusätzlichen Funktion. Sie werden für die serielle Kommunikation verwendet [Ard26b].

D0 ist der Empfangspin RX. D1 ist der Sendepin TX. Über diese Verbindung kommuniziert der Arduino Uno unter anderem mit dem Computer [Ard26b][Ard26a].

Beschreibung

Die serielle Kommunikation wird zum Beispiel für den seriellen Monitor verwendet. Im Programm wird sie mit `Serial.begin()` gestartet. Danach können Daten mit `Serial.print()` oder `Serial.println()` gesendet werden [Ard26a].

Da `D0` und `D1` auch für die USB-Kommunikation verwendet werden, sollten sie bei einfachen Projekten möglichst frei bleiben. Angeschlossene Bauteile können sonst das Hochladen von Programmen oder die serielle Ausgabe stören [Ard25c].

Eigenschaften

- `D0`: RX, Daten empfangen
- `D1`: TX, Daten senden [Ard26b]
- Für serielle Kommunikation
- Verbindung zum seriellen Monitor [Ard26a]
- Können das Hochladen stören, wenn externe Bauteile angeschlossen sind [Ard25c]

4.5.7. I2C-Pins

Hintergrundwissen

I2C ist eine Schnittstelle zur Kommunikation mit mehreren Bauteilen über zwei Datenleitungen. Beim Arduino Uno werden dafür die Pins `A4/SDA` und `A5/SCL` verwendet [Ard26b].

`SDA` ist die Datenleitung. `SCL` ist die Taktleitung. Über diese beiden Leitungen können zum Beispiel Displays, Sensoren oder Erweiterungsmodule angesprochen werden [Ard26a].

Beschreibung

Ein I2C-Bauteil wird mit `SDA`, `SCL`, Versorgungsspannung und `GND` verbunden [Ard26b]. Mehrere I2C-Bauteile können an denselben zwei Leitungen angeschlossen werden, wenn sie unterschiedliche Adressen besitzen [Ard26a].

Im Programm wird I2C häufig über die Bibliothek `Wire` verwendet [Ard26a].

Eigenschaften

- `A4`: `SDA`
- `A5`: `SCL` [Ard26b]
- Zwei Datenleitungen
- Mehrere Geräte an einem Bus möglich
- Geräte benötigen unterschiedliche Adressen
- Häufige Verwendung mit Displays und Sensormodulen [Ard26a]

4.5.8. SPI-Pins

Hintergrundwissen

SPI ist eine schnelle Schnittstelle zur Kommunikation mit Bauteilen wie Displays, Speicherbausteinen oder Funkmodulen [Ard26a]. Beim Arduino Uno liegen die SPI-Signale auf den Pins `D10` bis `D13` [Ard26b].

SPI verwendet mehrere Leitungen. Dadurch ist die Verbindung schneller als einfache digitale Signale, benötigt aber mehr Pins [Ard26a].

Beschreibung

Die wichtigsten SPI-Pins beim Arduino Uno sind:

- D10: SS oder CS
- D11: MOSI
- D12: MISO
- D13: SCK [Ard26b]

MOSI sendet Daten vom Arduino zum Bauteil. MISO empfängt Daten vom Bauteil. SCK ist die Taktleitung. SS oder CS wählt das jeweilige Bauteil aus [Ard26a].

Eigenschaften

- Schnelle Datenübertragung
- Verwendet mehrere Leitungen [Ard26a]
- Pins: D10 bis D13 [Ard26b]
- Geeignet für Displays, Speicher und Funkmodule
- Mehrere Geräte möglich, wenn jedes Gerät eine eigene CS-Leitung besitzt [Ard26a]

4.5.9. AREF-Pin

Hintergrundwissen

Der Pin AREF steht für Analog Reference. Er kann als externe Referenzspannung für die analogen Eingänge verwendet werden [Ard26b].

Normalerweise verwendet der Arduino Uno eine Standardreferenz für `analogRead()`. Mit AREF kann eine andere Referenzspannung verwendet werden, wenn genauere Messungen in einem kleineren Spannungsbereich benötigt werden [Ard26d].

Beschreibung

Der AREF-Pin wird nur für spezielle analoge Messungen verwendet. In einfachen Projekten bleibt dieser Pin normalerweise unbenutzt [Ard25c].

Wenn eine externe Referenzspannung verwendet wird, muss sie im Programm korrekt eingestellt werden. Dafür wird `analogReference()` genutzt [Ard26d].

Eigenschaften

- Referenzspannung für analoge Eingänge [Ard26b]
- Wird für genauere analoge Messungen verwendet [Ard26d]
- In einfachen Projekten meistens unbenutzt [Ard25c]
- Verwendung zusammen mit `analogReference()` [Ard26d]

4.5.10. RESET-Pin

Hintergrundwissen

Der RESET-Pin kann den Mikrocontroller neu starten. Er hat dieselbe Grundfunktion wie der Reset-Taster auf dem Board [Ard26b][Ard25c].

Beschreibung

Wird der RESET-Pin kurz mit GND verbunden, startet der Arduino Uno neu. Das gespeicherte Programm bleibt erhalten und beginnt wieder bei `setup()` [Ard25c][Ard26a].

Eigenschaften

- Startet den Mikrocontroller neu
- Löscht den gespeicherten Code nicht [Ard25c]
- Kann für externe Reset-Schaltungen verwendet werden [Ard26b]
- Wird in einfachen Projekten meistens nicht benötigt [Ard25c]

4.6. Beispiel Code

4.6.1. Manual

Ein einfaches Beispiel ist das Blinken der eingebauten LED. Dafür wird die LED auf Pin D13 verwendet [Ard25c]. Da diese LED bereits auf dem Arduino Uno verbaut ist, muss keine externe LED angeschlossen werden.

Der Arduino Uno muss nur über USB mit dem Computer verbunden sein. Die Stromversorgung und das Hochladen des Programms erfolgen über das USB-Kabel.

Der Pin wird in `setup()` mit `pinMode()` als Ausgang festgelegt [Ardi]. In `loop()` wird die LED mit `digitalWrite()` ein- und ausgeschaltet [Ardf].

Mit `delay()` wird eine Pause zwischen den Zuständen erzeugt. Dadurch blinkt die LED sichtbar [Ard26a].

4.6.2. Code

Listing 4.2.: Blink Internal LED

```
1000 /**
1001  * @file BlinkInternalLED.ino
1002  * @brief Blinks the built-in LED on the Arduino Uno.
1003  *
1004  * The Arduino Uno has a built-in LED connected to digital pin D13.
1005  * No external LED or resistor is required for this test.
1006  * The LED is switched on and off with a fixed delay.
1007  */
1008
1009 const int ledPin = 13; ///< Digital output pin connected to the built-
1010                        in LED.
1011
1012 /**
1013  * @brief Configures the built-in LED pin as an output.
1014  */
1015 void setup() {
1016     pinMode(ledPin, OUTPUT);
1017 }
1018
1019 /**
1020  * @brief Switches the built-in LED on and off repeatedly.
1021  */
1022 void loop() {
1023     digitalWrite(ledPin, HIGH);
1024     delay(1000);
1025
1026     digitalWrite(ledPin, LOW);
1027     delay(1000);
1028 }
```

```
../..Code/BlinkInternalLED/BlinkInternalLED.ino
```


5. CNCShieldV3

5.1. CNC Shields

Ein CNC Shield ist eine Erweiterungsplatine für den Arduino, die für die Ansteuerung von Schrittmotoren in CNC-Anwendungen entwickelt wurde. Es bildet die Verbindung zwischen dem Mikrocontroller, den eingesetzten Motortreibern und den angeschlossenen Komponenten der Maschine. Durch den kompakten Aufbau eignet sich das Board für kleine CNC-Systeme wie Fräsen, Plotter oder Gravurgeräte.

5.2. CNCShieldV3

Das CNC Shield V3 ist eine Erweiterungsplatine für den Arduino UNO,

5.3. Specification

- Kompatibilität: Arduino UNO R3
- Anzahl der Treiber-Steckplätze: 4
- Unterstützte Achsen: X, Y, Z sowie Achse A
- Unterstützte Treibermodule: A4988, DRV8825 und kompatible Module
- Versorgungsspannung der Motorseite: 12 V bis 36 V DC; bei 36 V ist die Eignung des eingesetzten Treibers zu beachten
- Enable-Signal: gemeinsamer Enable-Anschluss über D8
- Standard-Zuordnung X-Achse: X-STP auf D2, X-DIR auf D5
- Standard-Zuordnung Y-Achse: Y-STP auf D3, Y-DIR auf D6
- Standard-Zuordnung Z-Achse: Z-STP auf D4, Z-DIR auf D7
- Zusätzliche Anschlüsse: Endschalter, Spindelsteuerung, Kühlung, serielle Kommunikation und I2C
- Schrittauflösung: über Jumper einstellbar; bei A4988 laut Hersteller Vollschritt bis 1/16 Schritt
- Motoranschlüsse: 4-polige Anschlüsse im 2,54-mm-Raster
- Abmessungen: 69 mm × 54 mm × 15 mm
- Gewicht: 31 g

5.4. Library

grbl

5.4.1. Description

5.4.2. Installation

- data types
- data structure
- data quantity
- data quality

5.4.3. Functions

5.5. Example

In this section, a simple example is described in detail, which demonstrates how the library supports the sensor/actor.

5.5.1. Manual

5.5.2. Inside the Example

5.5.3. Code

5.5.4. Files

5.6. Calibration

cite method

5.7. Simple Code

5.7.1. Manual

Der Code testet den Schrittmotor am CNC Shield V3. Der Motor wird über den STEP-Pin schrittweise bewegt. Über den DIR-Pin wird die Drehrichtung festgelegt.

Benötigte Bauteile

- Arduino Uno
- CNC Shield V3
- A4988 oder DRV8825 Motortreiber
- Schrittmotor
- externe Motorspannung, z. B. 12 V
- USB-Kabel

Aufbau

Das CNC Shield V3 wird auf den Arduino Uno gesteckt. Der Motortreiber wird in den Steckplatz der X-Achse eingesetzt. Dabei muss auf die richtige Ausrichtung des Treibers geachtet werden. Der Schrittmotor wird an den X-Motoranschluss angeschlossen. Zusätzlich wird eine externe Motorspannung an das CNC Shield angeschlossen.

Verwendete Pins

- X-STEP: Pin 2
- X-DIR: Pin 5
- ENABLE: Pin 8

Verwendete Arduino-Befehle

- `setup()`: Wird einmal beim Start des Arduino ausgeführt [Ardj].
- `pinMode()`: Legt STEP, DIR und ENABLE als Ausgänge fest [Ardi].
- `digitalWrite()`: Aktiviert den Motortreiber und setzt die Drehrichtung [Ardf].
- `loop()`: Wird dauerhaft wiederholt [Ardg].
- `digitalWrite()`: Erzeugt die STEP-Impulse durch Wechsel zwischen HIGH und LOW [Ardf].
- `delayMicroseconds()`: Erzeugt kurze Pausen zwischen den Schrittimpulsen [Ard26a].

5.7.2. Code

Listing 5.1.: CNC Shield Schrittmotor

```

1000 /**
1001  * @file cncshieldschrittmotor.ino
1002  * @brief Moves a stepper motor with the CNC Shield V3.
1003  *
1004  * The stepper motor is connected to the X-axis driver of the CNC Shield
1005  * V3.
1006  * The motor moves continuously in one direction.
1007  */
1008
1009 const int stepPin = 2;    ///< STEP pin for the X-axis driver.
1010 const int dirPin = 5;     ///< DIR pin for the X-axis driver.
1011 const int enablePin = 8;  ///< Enable pin for the stepper drivers.
1012
1013 const int stepDelay = 1000; ///< Delay between steps in microseconds.
1014
1015 /**
1016  * @brief Runs once when the Arduino starts.
1017  *
1018  * The STEP, DIR, and ENABLE pins are configured as outputs.
1019  */
1020 void setup() {
1021     pinMode(stepPin, OUTPUT);    ///< Sets the STEP pin as output.
1022     pinMode(dirPin, OUTPUT);    ///< Sets the DIR pin as output.
1023     pinMode(enablePin, OUTPUT); ///< Sets the ENABLE pin as output.
1024
1025     digitalWrite(enablePin, LOW); ///< Enables the stepper driver.
1026     digitalWrite(dirPin, HIGH);   ///< Sets the movement direction.
1027 }
1028
1029 /**
1030  * @brief Main program that runs repeatedly.
1031  *
1032  * One step pulse is created again and again.
1033  * This makes the stepper motor rotate continuously.
1034  */
1035 void loop() {
1036     digitalWrite(stepPin, HIGH); ///< Starts one step pulse.
1037     delayMicroseconds(stepDelay);

```

```
1038   digitalWrite(stepPin, LOW); ///< End of step pulse.  
1040   delayMicroseconds(stepDelay);  
}
```

../Code/cncshieldschrittmotor/cncshieldschrittmotor.ino

5.8. Simple Application

5.8.1. Manual

Der Code kombiniert Schrittmotor und Endschalter. Der Motor fährt so lange in eine Richtung, bis der Endschalter gedrückt wird. Danach stoppt der Motor. So wird eine einfache Referenzfahrt dargestellt.

Benötigte Bauteile

- Arduino Uno
- CNC Shield V3
- A4988 oder DRV8825 Motortreiber
- Schrittmotor
- Endschalter oder Druckknopf
- externe Motorspannung, z. B. 12 V
- USB-Kabel

Aufbau

Das CNC Shield V3 wird auf den Arduino Uno gesteckt. Der Motortreiber wird in den Steckplatz der X-Achse eingesetzt. Der Schrittmotor wird an den X-Motoranschluss angeschlossen. Der Endschalter wird an den X-Endstop-Anschluss angeschlossen. Ein Anschluss des Endschalters wird mit dem Signal-Pin verbunden, der andere Anschluss mit GND. Zusätzlich wird die externe Motorspannung an das CNC Shield angeschlossen.

Verwendete Pins

- X-STEP: Pin 2
- X-DIR: Pin 5
- ENABLE: Pin 8
- X-Endstop: Pin 9

Verwendete Arduino-Befehle

- `setup()`: Wird einmal beim Start des Arduino ausgeführt [Ardj].
- `pinMode()`: Legt STEP, DIR und ENABLE als Ausgänge fest [Ardi].
- `pinMode(..., INPUT_PULLUP)`: Legt den Endstop-Pin als Eingang mit internem Pull-up-Widerstand fest [Ardi][Arda].

- `digitalWrite()`: Aktiviert den Motortreiber und setzt die Richtung für die Referenzfahrt [Ardf].
- `loop()`: Wird dauerhaft wiederholt [Ardg].
- `digitalRead()`: Liest den Zustand des Endstops aus [Arde].
- `digitalWrite()`: Erzeugt STEP-Impulse, solange der Endstop nicht gedrückt ist [Ardf].
- `delayMicroseconds()`: Erzeugt kurze Pausen zwischen den Schrittimpulsen [Ard26a].

5.8.2. Code

Listing 5.2.: CNC Shield Referenzfahrt

```

1000 /**
1001  * @file cncshieldreferenzfahrt.ino
1002  * @brief Performs a simple homing movement with a stepper motor and an
1003  *       endstop.
1004  *
1005  * The stepper motor is connected to the X-axis driver of the CNC Shield
1006  * V3.
1007  * The endstop is connected to the X-endstop input.
1008  * The motor moves until the endstop is pressed.
1009  * When the endstop is reached, the motor stops.
1010 */
1011
1012 const int stepPin = 2;    ///< STEP pin for the X-axis driver.
1013 const int dirPin = 5;     ///< DIR pin for the X-axis driver.
1014 const int enablePin = 8;  ///< Enable pin for the stepper drivers.
1015 const int endstopPin = 9; ///< Input pin for the X-endstop.
1016
1017 const int stepDelay = 1000; ///< Delay between steps in microseconds.
1018
1019 /**
1020  * @brief Runs once when the Arduino starts.
1021  *
1022  * The motor control pins are configured as outputs.
1023  * The endstop pin is configured as an input with internal pull-up.
1024  * The stepper driver is enabled.
1025 */
1026 void setup() {
1027     pinMode(stepPin, OUTPUT);    ///< Sets the STEP pin as output.
1028     pinMode(dirPin, OUTPUT);     ///< Sets the DIR pin as output.
1029     pinMode(enablePin, OUTPUT);  ///< Sets the ENABLE pin as output.
1030     pinMode(endstopPin, INPUT_PULLUP); ///< Endstop input with internal
1031                                     pull-up.
1032
1033     digitalWrite(enablePin, LOW); ///< Enables the stepper driver.
1034     digitalWrite(dirPin, LOW);    ///< Sets the homing direction.
1035 }
1036
1037 /**
1038  * @brief Main program that runs repeatedly.
1039  *
1040  * The motor moves step by step until the endstop is pressed.
1041  * When the endstop is pressed, no more step pulses are created.
1042 */
1043 void loop() {
1044     if (digitalRead(endstopPin) == HIGH) {
1045         digitalWrite(stepPin, HIGH); ///< Starts one step pulse.
1046         delayMicroseconds(stepDelay);
1047
1048         digitalWrite(stepPin, LOW);  ///< Ends one step pulse.
1049     }
1050 }

```

```

1046 |     delayMicroseconds(stepDelay);
      | }
1048 | }

```

../Code/cncshieldreferenzfahrt/cncshieldreferenzfahrt.ino

5.9. Tests

5.9.1. Manual

Der zweite Code testet die Grundfunktion des CNC Shield V3. Dabei werden die STEP-, DIR- und ENABLE-Pins der X-Achse verwendet. Der Motortreiber wird aktiviert und der Motor bewegt sich abwechselnd in beide Richtungen. Dadurch kann geprüft werden, ob das Shield, der Treiber und die Pinbelegung richtig funktionieren.

Benoetigte Bauteile

- Arduino Uno
- CNC Shield V3
- A4988 oder DRV8825 Motortreiber
- Schrittmotor
- externe Motorspannung, z. B. 12 V
- USB-Kabel

Aufbau

Das CNC Shield V3 wird auf den Arduino Uno gesteckt. Der Motortreiber wird in den Steckplatz der X-Achse eingesetzt. Der Schrittmotor wird an den X-Motoranschluss angeschlossen. Anschliessend wird die externe Motorspannung an das CNC Shield angeschlossen. Der Arduino wird über USB mit dem Computer verbunden.

Verwendete Pins

- X-STEP: Pin 2
- X-DIR: Pin 5
- ENABLE: Pin 8

Verwendete Arduino-Befehle

- `setup()`: Wird einmal beim Start des Arduino ausgeführt [Ardj].
- `pinMode()`: Legt STEP, DIR und ENABLE als Ausgänge fest [Ardi].
- `digitalWrite()`: Aktiviert den Motortreiber über ENABLE [Ardf].
- `loop()`: Wird dauerhaft wiederholt [Ardg].
- `digitalWrite()`: Legt die Drehrichtung mit DIR fest [Ardf].
- `moveSteps()`: Eigene Funktion, die eine feste Anzahl an Schritten ausführt.
- `digitalWrite()`: Erzeugt innerhalb von `moveSteps()` die STEP-Impulse [Ardf].
- `delayMicroseconds()`: Erzeugt kurze Pausen zwischen den Schrittimpulsen [Ard26a].
- `delay()`: Erzeugt eine Pause zwischen den Richtungswechseln [Ardd].

5.9.2. Code

Listing 5.3.: CNC Shield Test

```

1000 /**
1001  * @file cncshieldtest.ino
1002  * @brief Tests the CNC Shield V3 with a stepper motor on the X-axis.
1003  *
1004  * The motor driver is enabled through the ENABLE pin.
1005  * The motor moves first in one direction and then in the other
1006  * direction.
1007  * This checks whether the shield, driver, and motor connection work
1008  * correctly.
1009  */
1010
1011 const int stepPin = 2;    ///< STEP pin for the X-axis driver.
1012 const int dirPin = 5;     ///< DIR pin for the X-axis driver.
1013 const int enablePin = 8;  ///< Enable pin for the stepper drivers.
1014
1015 const int stepDelay = 1000;    ///< Delay between steps in microseconds
1016
1017 const int stepsPerMove = 400;  ///< Number of steps for one movement
1018                                ///< direction.
1019 const int pauseTime = 1000;    ///< Pause between direction changes in
1020                                ///< milliseconds.
1021
1022 /**
1023  * @brief Runs once when the Arduino starts.
1024  *
1025  * The STEP, DIR, and ENABLE pins are configured as outputs.
1026  * The stepper driver is enabled.
1027  */
1028 void setup() {
1029     pinMode(stepPin, OUTPUT);    ///< Sets the STEP pin as output.
1030     pinMode(dirPin, OUTPUT);    ///< Sets the DIR pin as output.
1031     pinMode(enablePin, OUTPUT); ///< Sets the ENABLE pin as output.
1032
1033     digitalWrite(enablePin, LOW); ///< Enables the stepper driver.
1034 }
1035
1036 /**
1037  * @brief Main program that runs repeatedly.
1038  *
1039  * The motor moves a defined number of steps in one direction.
1040  * After a short pause, the direction is changed and the motor moves
1041  * back.
1042  */
1043 void loop() {
1044     digitalWrite(dirPin, HIGH); ///< Sets the first movement direction.
1045     moveSteps(stepsPerMove);
1046
1047     delay(pauseTime); ///< Waits before changing direction.
1048
1049     digitalWrite(dirPin, LOW);  ///< Sets the opposite movement direction.
1050     moveSteps(stepsPerMove);
1051
1052     delay(pauseTime); ///< Waits before starting the next cycle.
1053 }
1054
1055 /**
1056  * @brief Moves the stepper motor by a defined number of steps.
1057  *
1058  * A step pulse is created for each step.
1059  *
1060  * @param steps Number of steps the motor should move.
1061  */
1062 void moveSteps(int steps) {
1063     for (int i = 0; i < steps; i++) {

```

```
1058 | digitalWrite(stepPin, HIGH); ///  
1060 | delayMicroseconds(stepDelay);  
1062 |  
1064 | digitalWrite(stepPin, LOW); ///  
1064 | delayMicroseconds(stepDelay);  
1064 | }  
1064 | }
```

../Code/cncshieldtest/cncshieldtest.ino

6. Sensor: Endschalter

6.1. Einleitung: Positionssensoren

In der industriellen Fertigungssteuerung nimmt die Erfassung von Zuständen eine zentrale Rolle ein. Gemäß der Fachliteratur wandelt ein Sensor (lat. sensus, der Sinn) „eine physikalische Größe (z. B. Kraft oder Temperatur) mit Hilfe eines physikalischen Effekts in ein elektrisches Signal um“ [Her+12, S. 433]. Speziell im Maschinenbau wird bei der Steuerung zwischen zwei grundlegenden Messverfahren unterschieden: Der „Positionserfassung durch Schalter“ und der „Positionserfassung durch Wegbeobachtung“. [Her+12, S. 435]

Bevor der Fokus auf kontaktbehaftete Schalter gelegt wird, sind folgende berührungslose Sensoren (Sensorschalter) als moderner Industriestandard zu nennen:

- Induktive Sensoren: Erfassen metallische Elemente „ohne Kontakt zwischen der Bewegung und dem Sensor“ [Her+12, S. 436].
- Kapazitive Sensoren: Nutzen die Beeinflussung eines elektrischen Feldes zur Kapazitätsänderung [Her+12, S. 438].
- Optische Sensoren: Arbeiten mit Infrarot-Lichtstrahlen zur Objekterkennung (z. B. Lichttaster oder Lichtschranken) [Her+12, S. 440].
- Ultraschall-Sensoren: Basieren auf der Messung der Laufzeit von Schallimpulsen [Her+12, S. 443].

6.2. Definition und Funktion: Elektromechanischer Endschalter

Ein elektromechanischer Endschalter ist ein kontaktbehafteter Positionssensor. Ein solcher Schalter „hat die Aufgabe, das Erreichen einer bestimmten Position (Endlage) zu melden oder einen Schaltvorgang auszulösen“ [Her+12, S. 435]. Dabei wird durch das Betätigen eines mechanischen Elements, wie beispielsweise eines Hebels oder Tasters, ein elektrischer Kontakt geöffnet oder geschlossen [Her+12, S. 120–122].

Technische Spezifikationen (Beispiel Creality): Bei dem vorliegenden Bauteil handelt es sich um einen „originalen Endschalter von Creality 3D, der bei fast allen FDM Druckern von Creality 3D verbaut ist“ [3dj].

6.3. Specification

- Kompatibilität: Geeignet für die X-, Y- oder Z-Achse von Modellen wie der Ender-3 oder CR-10 Serie [3dj][Bas].
- Hersteller: Creality
- Hersteller SKU: 4004180004
- Gewicht: 7.3g
- Abmaße: 28mm x 20mm x 9mm

- Auslöseweg: 4mm

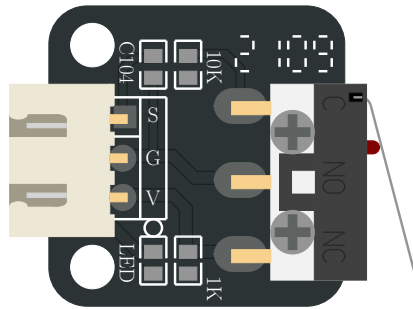


Abbildung 6.1.: Darstellung des im Projekt verwendeten Endschalters

6.4. Verwendung und Schaltfunktionen

Mechanische Endschalter werden zur Überwachung von Bewegungsabläufen eingesetzt. Dabei können verschiedene logische Funktionen realisiert werden:

- Schließer: Der Kontakt wird bei Betätigung geschlossen.
- Öffner: Der Kontakt wird bei Betätigung unterbrochen.
- Antivalenter Sensor: Dieser verfügt über zwei Ausgänge, wobei „stets ein Kontakt geschlossen und der andere offen“ ist [Her+12, S. 435]. Dies ermöglicht neben der reinen Schaltfunktion auch eine „Fehlererkennung und Diagnose durch die SPS“ [Her+12, S. 435–436].

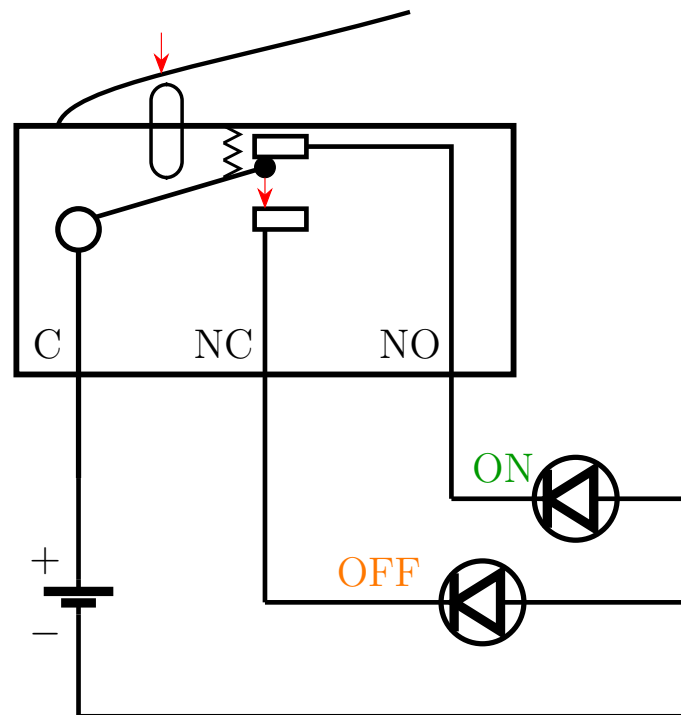


Abbildung 6.2.: Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO

Haupteinsatzgebiete:

- Endstopp-Erkennung: Schutz der Maschine vor dem Überfahren der mechanischen Grenzen.
- Referenzschalter (Homing): Der Schalter dient als „Referenzschalter“, um den Nullpunkt einer Achse (z. B. beim 3D-Drucker) zu kalibrieren [Bas].

6.5. Analyse: Vor- und Nachteile

Vorteile:

- Wirtschaftlichkeit: Sehr kostengünstig (ca. 4-5 € pro Stück) [3DJake, Bastelgarage].
- Einfachheit: Direkte Erzeugung eines binären Signals ohne aufwändige Auswerteelektronik.
- Robustheit gegen EMV: Unempfindlich gegenüber elektromagnetischen Störungen, da kein Oszillator wie bei induktiven Sensoren benötigt wird.

Nachteile:

- Mechanischer Verschleiß: Durch den physischen Kontakt unterliegen die Bauteile einer Abnutzung.
- Geschwindigkeit: Mechanische Kontakte können „prellen“ und sind langsamer als elektronische Lösungen [Her+12, S. 444].
- Technologische Verbreitung: Obwohl immer noch sehr viele mechanische Endschalter eingesetzt werden, ist der berührungslose Endschalter (Sensorschalter) in nahezu allen Bereichen als Standard-Bauelement zu finden [Her+12, S. 435].

6.6. Ausblick: Intelligente und vernetzte Sensorik

Die Zukunft der Positionserfassung liegt in der bidirektionalen Kommunikation. Ein moderner Standard ist hierbei IO-Link, welcher eine „feldbusunabhängige Kommunikation zwischen Sensoren und Aktoren und der Maschinensteuerung“ beschreibt [Her+12, S. 510].

Während klassische Endschalter nur „1 Bit“ an Informationen liefern (Ein/Aus), können intelligente Systeme über IO-Link zusätzlich Servicedaten übertragen, „die zur Einstellung oder Diagnose eines Devices führen“ [Her+12, S. 512].

Dies ermöglicht eine „vorbeugende Instandhaltung“, indem beispielsweise Verschleiß gemeldet wird, bevor es zum Maschinenausfall kommt [Her+12, S. 471, 512].

6.7. Library

Für die Verwendung des Endschalters werden keine zusätzlichen Bibliotheken benötigt. Es werden Funktionen aus der Arduino-Standardbibliothek verwendet:

- `pinMode()`: Konfiguriert einen Pin als Ein- oder Ausgang.
- `digitalRead()`: Liest den aktuellen Zustand eines digitalen Eingangs.
- `digitalWrite()`: Setzt einen digitalen Ausgang auf HIGH oder LOW.
- `Serial.begin()`: Initialisiert die serielle Kommunikation.
- `Serial.print()` / `Serial.println()`: Gibt Daten im Serial Monitor aus.
- `delay()`: Erzeugt eine zeitliche Verzögerung im Programmablauf.

6.8. Beispiel

Dieses Beispiel zeigt, wie der Endschalter über einen digitalen Eingang ausgelesen werden kann. Dabei wird der interne Pull-Up-Widerstand verwendet. Der Zustand des Schalters wird im Serial Monitor angezeigt und ändert sich beim Betätigen von HIGH auf LOW.

6.8.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND.

6.8.2. Erklärung des Beispiel-Codes

Nach der Initialisierung durch die Funktion `setup()` wird das Programm durch die Funktion `loop()` kontinuierlich ausgeführt. Diese Funktion stellt eine Endlosschleife dar, in der der Zustand des Endschalters bei jedem Durchlauf überprüft wird. [Ardj; Ardg]

Der verwendete Pin wird mit der Funktion `pinMode()` als Eingang oder Ausgang konfiguriert. [Ardi] Dabei wird der Modus `INPUT_PULLUP` verwendet, um den internen Pull-Up-Widerstand zu aktivieren. [Arda] Der interne Pull-Up-Widerstand sorgt dafür, dass am Eingang im Ruhezustand ein definiertes HIGH-Signal anliegt. Innerhalb der `loop()`-Funktion wird der Zustand des Endschalters mit der Funktion `digitalRead()` ausgelesen. Der gelesene Wert wird in einer Variablen gespeichert und über die serielle Schnittstelle ausgegeben. [Arde] Die Funktionen `Serial.print()` und `Serial.println()` werden verwendet, um den aktuellen Zustand des Eingangs im Serial

Monitor darzustellen.[Ardc] Durch die Verwendung des internen Pull-Up-Widerstands ergibt sich eine invertierte Logik. Im nicht betätigten Zustand des Endschalters liegt ein HIGH-Signal am Eingang an. Wird der Endschalter betätigt liegt ein LOW-Signal an. Zur besseren Lesbarkeit der Ausgabe wird eine Verzögerung mit der Funktion `delay()` erzeugt.[Ardd]

Listing 6.1.: Einfaches Beispiel zum Lesen eines mechanischen Endschalters

```

1000 /**
1001  * @file MicroSwitch_BasicRead.ino
1002  * @brief Basic example for reading a micro switch.
1003  */
1004 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
1005     switch. */
1006 /**
1007  * @brief Initializes the serial interface and the input pin.
1008  */
1009 void setup()
1010 {
1011     Serial.begin(115200);
1012     pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
1013 }
1014 /**
1015  * @brief Reads the switch state and prints it to the serial monitor.
1016  */
1017 void loop()
1018 {
1019     int switchState = digitalRead(MICRO_SWITCH_PIN);
1020
1021     Serial.print("Micro switch state: ");
1022     Serial.println(switchState);
1023
1024     delay(200);
1025 }

```

../..Code/MicroSwitchBasicRead/MicroSwitchBasicRead.ino

6.9. Justage

Die Justage (auch „Justierung“) ist eine „Tätigkeit, welche ein Messgerät in einen gebrauchstauglichen Betriebszustand versetzt. Sie kann manuell, halb automatisch oder automatisch erfolgen.“ [Hel, S. 17] Sie kann auch als das „abgleichen des Nullpunktes und der Kennliniensteigung“ [Ber24, S. 17] verstanden werden. Da der verwendete Endschalter nur die analogen Werte OPEN = 0 und CLOSED = 1 detektieren kann, sodass keine Kennlinie im herkömmlichen Sinne vorliegt, liegt die anwendungsspezifische Justage-Aufgabe darin, den gesamten Endschalter physisch in seiner Position am Rahmengestell so auszurichten, dass er die entsprechende bewegliche Achse im Rahmen einer Referenzfahrt in ihrer Bewegung eingrenzt, aber gleichzeitig im Normalbetrieb den erforderlichen Bauraum nicht künstlich einschränkt.

6.10. Tests

6.10.1. Einfacher Funktions-Test

Im einfachsten Anwendungstest wird der Endschalter verwendet, um eine LED zu steuern. Hierbei wird der Zustand des Schalters ausgewertet und auf einen Ausgang übertragen. Wird der Endschalter betätigt, schaltet sich die LED ein. Im nicht betätigten Zustand bleibt die LED ausgeschaltet. Durch diesen Test kann die Funktion

des Sensors sowie die Verarbeitung des Signals im System überprüft werden.

6.10.2. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND. Die LED wird an einen digitalen Ausgang (z. B. D13) angeschlossen.

Beim Betätigen des Endschalters wird die LED eingeschaltet, im nicht betätigten Zustand bleibt sie ausgeschaltet.

6.10.3. Einfacher Test-Code

Die Funktion `loop()` läuft dauerhaft und liest den Zustand des Mikroschalters ein. Da `INPUT_PULLUP` verwendet wird, bedeutet `LOW` = gedrückt. Ist der Schalter gedrückt, wird die LED eingeschaltet, sonst ausgeschaltet.

Listing 6.2.: An- und Ausschalten einer LED mittels Endschalter

```

1000 /**
1001  * @file MicroSwitch_LEDEExample.ino
1002  * @brief Simple application example using a micro switch and an LED.
1003  */
1004
1005 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
1006     switch. */
1007 const int LED_PIN = 13;          /**< Digital output pin for the built-in
1008     LED. */
1009
1010 /**
1011  * @brief Initializes the input and output pins.
1012  */
1013 void setup()
1014 {
1015     pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
1016     pinMode(LED_PIN, OUTPUT);
1017 }
1018
1019 /**
1020  * @brief Switches the LED depending on the micro switch state.
1021  */
1022 void loop()
1023 {
1024     int switchState = digitalRead(MICRO_SWITCH_PIN);
1025
1026     if (switchState == LOW)
1027     {
1028         digitalWrite(LED_PIN, HIGH);
1029     }
1030     else
1031     {
1032         digitalWrite(LED_PIN, LOW);
1033     }
1034 }

```

../Code/MicroSwitchLEDEExample/MicroSwitchLEDEExample.ino

6.11. Einfache Anwendung

Das Programm führt eine Referenzfahrt mit einem Schrittmotor aus.

Im `setup()` werden die Pins für den Treiber und den Endschalter eingerichtet. Danach wird der Motortreiber aktiviert und die Referenzfahrt gestartet.

Die Funktion `oneStep()` erzeugt einen einzelnen Schritt des Motors.

Mit `moveSteps()` kann der Motor eine festgelegte Anzahl an Schritten in eine bestimmte Richtung fahren.

In `homeAxis()` fährt der Motor so lange, bis der Endschalter gedrückt wird. Danach wartet das Programm kurz und fährt den Motor einige Schritte zurück.

6.11.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters wird an D2 angeschlossen, der zweite Anschluss liegt auf GND. Der Schrittmotortreiber wird über die Steuerpins STEP und DIR an digitale Ausgänge des Mikrocontrollers angeschlossen (z. B. STEP an D3 und DIR an D4).

Listing 6.3.: Referenzfahrt einer Achse mit Endschalter

```

1000 /**
1001  * @file referenzfahrt.ino
1002  * @brief Homing routine using a stepper motor, A4988 driver, and
1003  *        endstop.
1004  */
1005
1006 /** @brief STEP pin of the driver */
1007 const int stepPin = 2;
1008
1009 /** @brief Direction pin of the driver */
1010 const int dirPin = 5;
1011
1012 /** @brief Enable pin of the driver */
1013 const int enPin = 8;
1014
1015 /** @brief Endstop input pin */
1016 const int endstopPin = 9;
1017
1018 /** @brief Step delay in microseconds */
1019 const int stepDelayUs = 1000;
1020
1021 /** @brief Number of steps for the backoff movement */
1022 const int backoffSteps = 50;
1023
1024 /** @brief Initializes the hardware and starts the homing routine */
1025 void setup() {
1026     pinMode(stepPin, OUTPUT);
1027     pinMode(dirPin, OUTPUT);
1028     pinMode(enPin, OUTPUT);
1029     pinMode(endstopPin, INPUT_PULLUP);
1030
1031     Serial.begin(115200);
1032
1033     digitalWrite(enPin, LOW);
1034     digitalWrite(dirPin, LOW);
1035
1036     homeAxis();
1037 }
1038
1039 /** @brief Main loop (unused in this example) */
1040 void loop() {
1041
1042 }
1043
1044 /** @brief Executes a single motor step */
1045 void oneStep() {
1046     digitalWrite(stepPin, HIGH);
1047     delayMicroseconds(stepDelayUs);
1048     digitalWrite(stepPin, LOW);
1049     delayMicroseconds(stepDelayUs);
1050 }

```

```
1050 /**  
1051  * @brief Moves the motor by a defined number of steps  
1052  * @param steps Number of steps to execute  
1053  * @param dir Direction of movement (LOW/HIGH)  
1054  */  
1055 void moveSteps(int steps, bool dir) {  
1056     digitalWrite(dirPin, dir);  
1057     for (int i = 0; i < steps; i++) {  
1058         oneStep();  
1059     }  
1060 }  
  
1062 /** @brief Performs the homing sequence and backoff movement */  
1063 void homeAxis() {  
1064     while (digitalRead(endstopPin) == HIGH) {  
1065         oneStep();  
1066     }  
  
1068     delay(200);  
1069     moveSteps(backoffSteps, HIGH);  
1070 }
```

../Code/referenzfahrt/referenzfahrt.ino

6.12. Weiterführende Literatur

- Zielke, Dirk: *Mikrosysteme*. Springer, 2025. [Zie25] Das Buch behandelt die Grundlagen und Anwendungen von Mikrosystemen und bietet einen Überblick über Aufbau, Funktion und Einsatz moderner Sensorsysteme.
- Trglqq ankler, Hans-Rolf und Reindl, Leo: *Sensortechnik*. Springer, 2014. [TR14] Das Buch behandelt grundlegende Prinzipien der Sensortechnik und beschreibt verschiedene Sensorarten sowie deren Einsatz in technischen Systemen.

7. OLED-Display

7.1. Allgemein

Organic Light Emitting Diode (OLED)s (Organic Light Emitting Diodes) stellen eine besondere Form der LED-Technologie dar, bei der jedes Pixel aus eigenständig leuchtenden organischen Leuchtdioden besteht. Im Gegensatz zu LCDs mit LED-Hintergrundbeleuchtung benötigen OLEDs keine Hintergrundbeleuchtung oder LC-Zellen, da die einzelnen Pixel selbst Licht erzeugen. Dies ermöglicht nicht nur eine besonders flache Bauweise, sondern auch eine exakte Steuerung einzelner Bildpunkte – inklusive vollständigem Abschalten für tiefstes Schwarz und hohen Kontrast.

Ein wesentlicher Vorteil gegenüber LCD-Technologien ist die deutlich schnellere Reaktionszeit: Helligkeitsänderungen erfolgen bei OLEDs in unter einer Mikrosekunde. Auch der Farbraum und Kontrastumfang sind bei OLED-Bildschirmen in der Regel erweitert. Allerdings ist die Lebensdauer einzelner OLEDs im Vergleich zu anderen Bildschirmstechnologien tendenziell geringer, obwohl bereits von Betriebszeiten im Bereich von 15 Jahren gesprochen wird. In der Praxis kommen verschiedene OLED-Technologien zum Einsatz, insbesondere RGB-SBS-AMOLED (mit drei nebeneinander angeordneten Farb-OLEDs pro Pixel) und WOLED (weiße OLEDs mit nachgeschalteten Farbfiltern). OLED-Elemente werden meist im Dünnschichtverfahren hergestellt, was die Produktion vereinfacht. [Stotz.2019]

Für Projekte mit Mikrocontrollern wie den Arduino oder Raspberry Pi werden oft monochrome OLED-Displays verwendet. Das bedeutet, dass anstelle von RGB-LEDs nur normale, einfarbige LEDs verwendet werden. Oft sind diese monochromen OLED-Displays blau oder weiß, um einen möglichst hohen Kontrast zu bieten. [Schreiter.2019]

7.2. OLED-Display mit SSD1306 Controller

Das OLED-Display mit dem Controller SSD1306 dient der Ausgabe von Messwerten des Arduinos. Es besitzt Abmessungen von 27 x 27 mm und überzeugt durch seinen hohen Kontrast sowie gute Lesbarkeit. Die Darstellung erfolgt in blauer Farbe. Angesteuert wird das Display über die I²C-Schnittstelle (Pins VCC, GND, SCL, SDA) und benötigt eine Betriebsspannung von 3,3 V bis 5 V. Der zulässige Betriebstemperaturbereich liegt zwischen -30 °C und +70 °C. Für die Ansteuerung kommt die U8glib-Bibliothek zum Einsatz, welche eine Vielzahl von Grafikfunktionen für OLED-Displays mit SSD1306-Controller bereitstellt. [Schreiter.2019]

Tabelle 7.1.: Spezifikationen OLED-Display

Identifikation	
Artikel-Nr.	ARD OLED 0.96
EAN/GTIN	5410329729738
Hst.-Teile-Nr.	WPI438
Hauptmerkmale	
Modell	OLED-Display
Abmessungen	27 x 27
Gewicht	10 g

Lieferumfang	WPI438
Weitere Spezifikationen	
Spannungsversorgung	3.3 V bis 5 V
Auflösung	128 x 64 Pixel
Hintergrundfarbe	Schwarz
Betrachtungswinkel	> 160°
Anzeigefarbe	Blau
Interface	I ² C
Controller-Chip	SSD1306

7.2.1. Aufbau und Funktionsweise

In der Abbildung ?? wird der Aufbau des OLED-Displays gezeigt. Über die vier Pins kann später die Stromversorgung und Logikspannung des OLED-Displays mit dem Mikrocontroller verbunden werden.

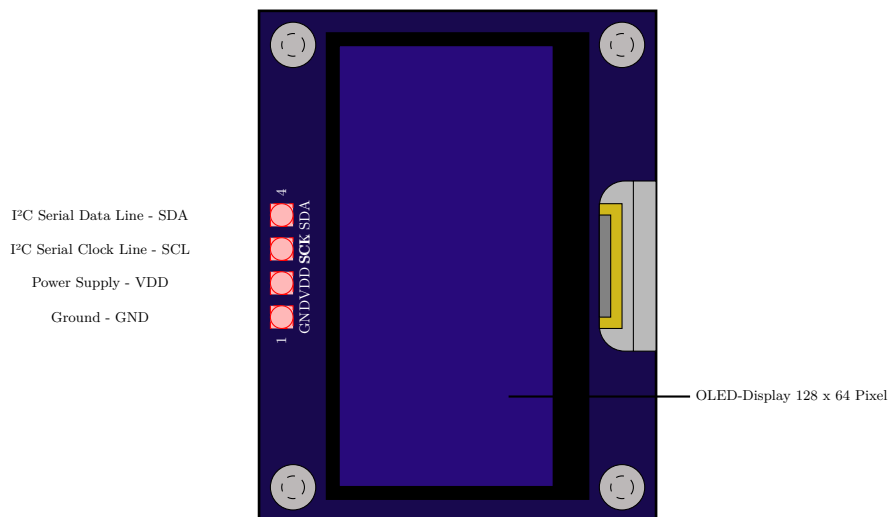


Abbildung 7.1.: OLED-Display nach
fig:OLEDsketch

Bei der Ansteuerung eines OLED-Displays über das I²C-Protokoll übernehmen die beiden Leitungen Serial Data (SDA) und Serial Clock (SCL) den seriellen Datenaustausch zwischen Mikrocontroller (Master) und OLED-Display (Slave).

- SDA überträgt die eigentlichen Datenbits (z.B. Befehle oder Bildinhalte).
- SCL gibt den Takt vor, der bestimmt, wann ein Bit gelesen oder geschrieben wird.

Das I²C-Protokoll ermöglicht eine einfache, bidirektionale Kommunikation über nur zwei Leitungen, wobei mehrere Geräte über individuelle Adressen angesprochen werden können.

Protokoll I²C

In den 1980-er Jahre wurde die I²C-Schnittstelle entwickelt zum Datenaustausch zwischen verschiedenen Komponenten eines Gerätes. Im Gegensatz zur seriellen Verbindung, welche stets nur zwei Kommunikationspartner verbindet, handelt es sich

hierbei um einen Datenbus, welcher optional deutlich mehr Komponenten (ursprünglich bis zu 112, in neueren Versionen nochmals deutlich mehr) miteinander verbinden kann. Elektrisch gesehen besteht der Bus aus lediglich zwei Leitungen, an die alle Busteilnehmer angeschlossen sind, sowie einem gemeinsamen Massebezug. Eine der Leitungen dient als Taktsignal (Clock - CLK), die andere wird zum Datenaustausch (Serial Data – SDA) verwendet. Dabei fungiert stets ein Teilnehmer als Master, er generiert das Taktsignal und koordiniert die Kommunikation – etwa so wie ein Moderator in einer Gesprächsrunde. Alle anderen Komponenten agieren als Slaves, sie ordnen sich also dem Master unter. Jeder Slave benötigt eine eindeutige Adresse (in Form einer Zahl), über die er vom Master angesprochen werden kann. Der Master kann den Slaves jederzeit Daten senden.

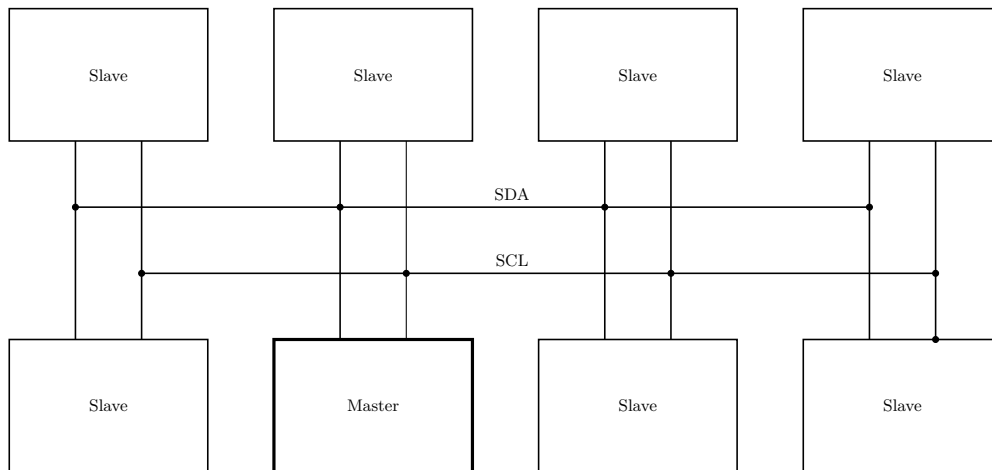


Abbildung 7.2.: I²C-Bus mit 8 Teilnehmern nach [Schreiter.2019]

Die einzelnen Slaves dürfen jedoch nur senden, wenn sie vom Master abgefragt werden, also quasi eine Sendeerlaubnis erhalten. Dadurch wird vermieden, dass mehrere Komponenten gleichzeitig auf die einzige vorhandene Datenleitung senden, denn dann wäre das Signal gestört. Üblicherweise agiert unser Arduino natürlich als Master und kommuniziert mit Komponenten wie Sensoren oder OLED-Displays als Slaves. Allerdings ist es auch möglich, über I²C Verbindungen zwischen mehreren Arduinos herzustellen.

7.2.2. Anwendung

OLED-Displays werden in vielen Bereichen eingesetzt, in denen energieeffiziente und kontrastreiche Anzeigen benötigt werden. Typische Anwendungen finden sich in Fernsehern, tragbaren Geräten wie Smartwatches und medizinischen Geräten. Seit etwa 15 Jahren werden LCDs in vielen Anwendungsbereichen von OLED-Displays ersetzt. Durch die Möglichkeit Pixel einzeln anzusteuern und auch komplett abzuschalten, kann der OLED-Display sehr kontraststarke Bilder erzeugen, welches ebenfalls für eine gute Lesbarkeit sorgt. Durch den geringen Stromverbrauch sind OLED-Displays ebenfalls besonders gut für batteriebetriebene Systeme geeignet.[Schreiter.2019]

7.3. Installation

Zur Nutzung des OLED-Displays muss dieses nach mit dem Arduino nach Tabelle 7.2 angeschlossen werden. Bei der Benutzung des Arduino Shields, können Grove-Stecker verwendet werden.

Tabelle 7.2.: Pinbelegung des OLED-Displays

OLED-Display Pin	Arduino Pin
SDA	A4
SCL	A5
VCC	5V out
GND	GND

7.4. Bibliotheken

Für die Verwendung des OLED-Displays sind die folgenden aufgelisteten Bibliotheken notwendig 7.3. Um eine Bibliothek in ein Programm zu integrieren muss diese in der Arduino IDE heruntergeladen und installiert sein und anschließend im Programmcode mit dem folgenden Befehl eingebunden werden: `#include<name.h>`

7.5. Verwendete Bibliotheken

Tabelle 7.3.: Verwendete Bibliotheken zum Testen des OLED-Displays

Bibliothek	Beschreibung
Wire.h	Version: Arduino-Standardbibliothek Funktion: Dient der Kommunikation über den I ² C-Bus zwischen Mikrocontroller und OLED-Display.
Adafruit_GFX.h	Version: 1.12.6 Grafikbibliothek zur Darstellung von Text, Linien, Rechtecken und weiteren primitiven Elementen auf grafischen Displays.
Adafruit_SSD1306.h	Version: 2.5.16 Treiber für SSD1306-basierte OLED-Displays mit Unterstützung für I ² C.

7.5.1. Wire.h

Die Bibliothek Wire.h ist eine Standardbibliothek für Arduino-Plattformen, welche Funktionen und Methoden für die Kommunikation zur Verfügung stellt. Mit Hilfe der Schnittstelle I²C ermöglicht die Bibliothek die Kommunikation zwischen einem Arduino und einem anderen I²C-fähigen Gerät wie z.B. Sensoren oder Displays. I²C ist ein Kommunikationsbus, der im Vergleich zu seriellen Schnittstellen den Vorteil hat, dass er mit mehr als zwei Geräten kommunizieren kann. Ein I²C-Bus benötigt zwei Leitungen: SCL für ein Taktsignal und SDA für Daten.

7.5.2. Adafruit_GFX.h

Die Adafruit_GFX-Bibliothek bietet grundlegende Grafikfunktionen für viele verschiedene LCD- und OLED-Displays sowie LED-Matrizen von Adafruit. Sie sorgt dafür, dass Programme leicht zwischen unterschiedlichen Displays angepasst werden können. Neue Funktionen und Verbesserungen gelten automatisch für alle unterstützten Displays. Die Bibliothek wird immer zusammen mit einer weiteren Bibliothek verwendet, die speziell für das jeweilige Display gedacht ist, für unser spezifisches OLED-Display, welches den Controller-Chip SSD1306 verwendet, muss die Adafruit_SSD1306.h Bibliothek ebenfalls mit eingebunden werden.

Adafruit_SSD1306.h

Die Adafruit_SSD1306 Bibliothek ist speziell für OLED-Displays mit dem SSD1306-Controller entwickelt worden. Sie arbeitet zusammen mit der Adafruit_GFX-Bibliothek und ermöglicht die Ansteuerung und Darstellung von Grafiken und Text auf dem Display.

7.6. Codebeispiel

7.6.1. Kurzbeschreibung des Programms

Dieses Arduino-Testprogramm dient zur Initialisierung und Darstellung einfacher Texte auf einem SSD1306 OLED-Display. Nach der Initialisierung wird die Nachricht *"Hello, world!"* angezeigt. Anschließend wird die Anzeige im Sekundentakt invertiert, um einen Blinke-Effekt zu erzeugen.

7.6.2. Verwendete Funktionen

Tabelle 7.4.: Funktionen im Testprogramm für das OLED-Display.

Funktion	Beschreibung
<code>Serial.begin()</code>	Startet die serielle Kommunikation mit 9600 Baud.
<code>display.begin()</code>	Initialisiert das OLED-Display mit angegebener I ² C-Adresse.
<code>display.clearDisplay()</code>	Löscht alle Inhalte auf dem Display.
<code>display.setTextSize()</code>	Setzt die Schriftgröße für Textausgabe.
<code>display.setCursor()</code>	Positioniert den Textcursor an einer bestimmten Stelle.
<code>display.println()</code>	Gibt eine Textzeile auf dem Display aus.
<code>display.display()</code>	Überträgt die Zeichenpufferdaten auf das Display.
<code>display.invertDisplay()</code>	Invertiert die Anzeige des Displays.

7.6.3. Globale Variablen und Parameter

Tabelle 7.5.: Globale Variablen und Parameter im Testprogramm für das OLED-Display.

Variable	Beschreibung
<code>SCREEN_WIDTH</code>	Definiert die Breite des Displays in Pixeln (128).
<code>SCREEN_HEIGHT</code>	Definiert die Höhe des Displays in Pixeln (64).
<code>display</code>	Objekt zur Steuerung des OLED-Displays mit Adafruit SSD1306-Treiber.

7.6.4. Testcode

Listing 7.1.: Testprogramm für das OLED-Display

```

1000 /**
      * @file TestOLEDDisplay.ino
1002  *
      * @brief Basic example for initializing and using an SSD1306 OLED
            display with the Arduino Nano 33 BLE Sense.
1004  * @date 2025-04-14
      * @author Wedemann
1006  * @details
      * This sketch demonstrates how to initialize the OLED screen and
            display a simple text message.

```

```

1008 * The display is then set to repeatedly invert its pixels to create a
      * blinking effect.
1010 *
      * Libraries used:
      * - Wire.h: For I2C communication.
1012 * - Adafruit_GFX.h Version 1.12.0: Core graphics library for Adafruit
      * displays.
      * - Adafruit_SSD1306.h Version 2.5.13: Driver for SSD1306-based OLED
      * displays.
1014 *
      * Hardware:
1016 * - Arduino Nano 33 BLE Sense
      * - SSD1306 OLED display (128x64 pixels) connected via I2C.
1018 *
      * Setup:
1020 * - SDA to A4 (or as defined by your board)
      * - SCL to A5
1022 * - Power and ground accordingly
      *
      * Usage:
1024 * - The OLED displays "Hello, world!" upon startup.
1026 * - The display inverts (blinks) every second within the loop.
      *
1028 * Notes:
      * - Ensure the OLED is correctly connected and powered.
1030 * - The display I2C address is set to 0x3C (common default).
      */
1032 #include <Wire.h>
1034 #include <Adafruit_GFX.h>
      #include <Adafruit_SSD1306.h>
1036
      #define SCREEN_WIDTH 128 ///< OLED display width in pixels
1038 #define SCREEN_HEIGHT 64  ///< OLED display height in pixels
1040
      // Create display object for SSD1306 using I2C
      Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
1042
      /**
1044 * @brief Arduino setup function.
      *
1046 * Initializes serial communication, sets up the OLED display, and
      * prints a static message.
      */
1048 void setup() {
      Serial.begin(9600); ///< Start serial communication at 9600 baud
1050
      // Attempt to initialize the OLED display
1052 if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // 0x3C is a common
      I2C address for 128x64 OLEDs
      Serial.println(F("SSD1306 allocation failed"));
1054   for(;;); ///< Infinite loop if initialization fails
      }
1056
      delay(2000); ///< Short delay to allow the display to initialize
1058
      display.clearDisplay(); ///< Clear any previous display content
1060
      display.setTextSize(1); ///< Set text size to smallest
1062 display.setTextColor(BLUE); ///< Set text color to blue
      display.setCursor(0, 10); ///< Position text cursor
1064 display.println("Hello, world!"); ///< Display a static greeting
      display.display(); ///< Update the screen with written content
1066 }
1068
      /**
1070 * @brief Arduino main loop function.
      *

```

```

1072  * Causes the OLED display to blink by inverting and restoring pixels
      every second.
1073  */
1074  void loop() {
      display.invertDisplay(true); ///< Invert the display (white -> black,
      black -> white)
      delay(1000); ///< Wait 1 second
1076  display.invertDisplay(false); ///< Restore the display to normal
      delay(1000); ///< Wait 1 second
1078  }

```

../Code/TestOled/TestOled.ino

7.7. Tests

7.7.1. Test: Fortschrittsbalken per Taster

In diesem Test wird überprüft, ob durch Betätigung eines Tasters eine visuelle Rückmeldung auf dem OLED-Display ausgegeben werden kann. Beim Drücken des Tasters wird ein Fortschrittsbalken von 0 % bis 100 % in 10 Schritten angezeigt.

Der Fortschrittsbalken wird dabei schrittweise gefüllt und zusätzlich der aktuelle Prozentwert auf dem Display ausgegeben. Dadurch kann sowohl die Funktion des Tasters als auch die grafische Darstellung auf dem OLED-Display überprüft werden.

7.7.2. Manual

Der Taster wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Ein Anschluss des Tasters wird mit D2 verbunden, der andere mit GND. Dabei wird der interne Pull-Up-Widerstand verwendet.

Das OLED-Display wird über die I²C-Schnittstelle angeschlossen. Die Pins werden wie folgt verbunden:

- VCC an 3,3 V oder 5 V
- GND an GND
- SDA an A4
- SCL an A5

7.7.3. Test-Code

Im `loop()` wird der Zustand des Tasters abgefragt. Da `INPUT_PULLUP` verwendet wird, entspricht `LOW` einem gedrückten Taster.

Wird der Taster gedrückt, wird ein Fortschrittsbalken in 10 Schritten angezeigt. Nach jedem Schritt wird der aktuelle Fortschritt in Prozent auf dem OLED-Display angezeigt. Zwischen den einzelnen Schritten wird eine kurze Verzögerung eingefügt, um den Fortschritt zu sehen.

Listing 7.2.: Fortschrittsbalken auf dem OLED-Display durch Tastendruck

```

1000 #include <Wire.h>
      #include <Adafruit_GFX.h>
1002 #include <Adafruit_SSD1306.h>
1004 #define SCREEN_WIDTH 128
      #define SCREEN_HEIGHT 64
1006 #define OLED_RESET -1
1008 Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET)
      ;

```

```

1010 #define BUTTON_PIN 2
1012 void setup() {
1013     pinMode(BUTTON_PIN, INPUT_PULLUP);
1014
1015     display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
1016     display.clearDisplay();
1017     display.setTextSize(1);
1018     display.setTextColor(SSD1306_WHITE);
1019 }
1020
1021 void loop() {
1022     if (digitalRead(BUTTON_PIN) == LOW) {
1023
1024         for (int i = 0; i <= 10; i++) {
1025
1026             int percent = i * 10;
1027
1028             display.clearDisplay();
1029
1030             display.setCursor(0, 0);
1031             display.print("Fortschritt:");
1032
1033             display.setCursor(90, 0);
1034             display.print(percent);
1035             display.print("%");
1036
1037             display.drawRect(10, 30, 108, 10, SSD1306_WHITE);
1038
1039             int barWidth = (108 * percent) / 100;
1040             display.fillRect(10, 30, barWidth, 10, SSD1306_WHITE);
1041
1042             display.display();
1043
1044             delay(300);
1045         }
1046         delay(1000);
1047     }
1048 }

```

../Code/OLEDProgressBar/OLEDProgressBar.ino

7.8. Weiterführende Literatur

- **OLED:** Weiterführende Informationen zum Aufbau und zur Funktionsweise von OLEDs enthält das Buch „*Elektronik für Ingenieure und Naturwissenschaftler*“ [Hering.2021].
 - Beschreibt sämtliche Komponenten der Elektrotechnik, welche für Ingenieure relevant sein können.
 - Sehr ausführliche Beschreibung der Struktur und Aufbau von LEDs und auch speziell OLED-Displays.
 - Nennt weitere Displaytypen und Alternativen im Vergleich zu OLED.
- **Ansteuerung:** Sehr gute inoffizielle Anleitung zum Basteln mit dem Arduino und viele Anwendungsbeispiele bietet das Buch „*Arduino-Kompendium*“ [Schreiter.2019].
 - Bietet eine Vielzahl von Kurzanleitungen zu vielen Sensoren und Komponenten die teilweise im offiziellen Handbuch von Arduino fehlen.
 - Enthält viele Praxisprojekte zum Nachbauen

- **Anzeigemöglichkeiten auf dem OLED-Display:** Um herauszufinden was man alles mit dem OLED-Display machen kann, ist die Dokumentation der Adafruit_GFX.h Bibliothek zu empfehlen. learn.adafruit.com/adafruit-gfx-graphics-library. Auf der Webseite werden sämtliche Funktionen der Bibliothek dokumentiert sowie weitere Bibliotheken verlinkt, die für Funktionen des OLED-Displays kompatibel sind.
 - Anzeigen von Bildern, Animationen oder Texten auf dem Display
 - Laden und nutzen von größeren Dateien in den RAM von externen Speichergeräten wie einer SD-Karte
 - Weitere Bibliotheken von Adafruit

8. Druckknopf

8.1. General

Ein elektrischer Druckknopf, auch Drucktaster genannt, ist ein elektromechanisches Bedienelement, mit dem ein Stromkreis durch Drücken geöffnet oder geschlossen wird. Je nach Ausführung kann er als Schließer oder Öffner arbeiten. Nach dem Loslassen kehrt ein Taster wieder in seine Ausgangsstellung zurück. [Rs2]

8.2. Joy IT

Beim Joy-IT Button-Micro handelt es sich um einen kompakten Micro-Arcadebutton mit Mikroschalter. Er besteht aus Nylon, ist in mehreren Farben erhältlich und für eine Belastungsgrenze von 12 V ausgelegt. [Joya]

8.3. Spezifikationen

- Abmessungen: Ø 32.8 x 29.9 mm
- Belastungsgrenze: 10 mA / 12 V DC [Joya]
- Schaltkreis

8.3.1. Functions

Für die Verwendung eines einfachen Druckknopfes werden keine zusätzlichen Bibliotheken benötigt. Die Abfrage kann direkt mit den Standardfunktionen der Arduino IDE erfolgen.

8.4. Beispiel Code

8.4.1. Manual

Der Taster wird mit einem Anschluss an GND und mit dem anderen an Pin 2 angeschlossen. Die LED bekommt einen 220 Ohm oder 330 Ohm Vorwiderstand. Der lange LED-Anschluss ist die Anode (+) und kommt über den Widerstand an Pin 13. Der kurze LED-Anschluss ist die Kathode (-) und kommt an GND. Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()` legt fest, ob ein Pin als Eingang oder Ausgang verwendet wird. [Ardi]
- `INPUT_PULLUP` aktiviert den interne Pull-up-Widerstand des Arduino. [Arda]
- `digitalRead()` liest den Zustand des Druckknopfes aus. [Arde]
- `digitalWrite()` wird die LED ein- oder ausgeschalten. [Ardf]

8.4.2. Code

Listing 8.1.: Led mit Druckknopf

```

1000 /**
1001  * @file ledmitdruckknopf.ino
1002  * @brief Turns an LED on while a push button is pressed.
1003  *
1004  * The push button is connected between pin 2 and GND.
1005  * The LED is connected to pin 13 with a 220 ohm or 330 ohm series
1006  * resistor.
1007  * The long LED leg is the anode (+), and the short LED leg is the
1008  * cathode (-).
1009  */
1010
1011 const int druckknopfPin = 2; ///< Pin where the push button is connected
1012
1013 const int ledPin = 13;      ///< Pin where the LED is connected.
1014
1015 /**
1016  * @brief Runs once when the Arduino starts.
1017  *
1018  * The pins are configured as input or output here.
1019  */
1020 void setup() {
1021     pinMode(druckknopfPin, INPUT_PULLUP); ///< Push button input with
1022     internal pull-up.
1023     pinMode(ledPin, OUTPUT);              ///< LED pin as output.
1024 }
1025
1026 /**
1027  * @brief Main program that runs repeatedly.
1028  *
1029  * If the push button is pressed, the LED is turned on.
1030  * If the push button is released, the LED is turned off.
1031  */
1032 void loop() {
1033     int druckknopfStatus = digitalRead(druckknopfPin); ///< Current state
1034     of the push button.
1035
1036     if (druckknopfStatus == LOW) {
1037         digitalWrite(ledPin, HIGH); ///< Turns the LED on.
1038     } else {
1039         digitalWrite(ledPin, LOW);  ///< Turns the LED off.
1040     }
1041 }

```

../Code/ledmitdruckknopf/ledmitdruckknopf.ino

8.5. Einfacher Testcode

8.5.1. Manual

Der Taster wird mit einem Anschluss an GND und mit dem anderen an Pin 2 angeschlossen. Im seriellen Monitor der Arduino IDE wird dann angezeigt, ob der Druckknopf gedrückt ist oder nicht.

Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()`
- `INPUT_PULLUP`
- `digitalRead()`
- `Serial.begin()` Startet die serielle Verbindung zum Computer.[Ardb]

- `Serial.println()` Gibt den Wert 1 oder 0 im seriellen Monitor aus. [Ardc]
- `delay()` Wartet kurz, damit die Ausgabe lesbar bleibt. [Ardd]

8.5.2. Code

Listing 8.2.: Druckknopf Test

```

1000 /**
1001  * @file druckknopftest.ino
1002  * @brief Tests a push button and prints 0 or 1 to the serial monitor.
1003  *
1004  * The push button is connected between pin 2 and GND.
1005  * The internal pull-up resistor is used.
1006  */
1007
1008 const int druckknopfPin = 2; ///< Pin where the push button is connected
1009
1010 /**
1011  * @brief Runs once when the Arduino starts.
1012  *
1013  * The push button pin is set as an input with the internal pull-up
1014  * resistor.
1015  * The serial output is started.
1016  */
1017 void setup() {
1018     pinMode(druckknopfPin, INPUT_PULLUP); ///< Sets the pin as input with
1019     internal pull-up.
1020     Serial.begin(115200);                ///< Starts the serial
1021     connection.
1022 }
1023
1024 /**
1025  * @brief Main program that runs repeatedly.
1026  *
1027  * Prints 1 if the push button is pressed.
1028  * Prints 0 if the push button is not pressed.
1029  */
1030 void loop() {
1031     int druckknopfStatus = digitalRead(druckknopfPin); ///< Reads the
1032     state of the push button.
1033
1034     if (druckknopfStatus == LOW) {
1035         Serial.println(1); ///< Push button is pressed.
1036     } else {
1037         Serial.println(0); ///< Push button is not pressed.
1038     }
1039
1040     delay(200); ///< Short pause for readable output.
1041 }

```

../Code/druckknopftest/druckknopftest.ino

8.6. Einfache Anwendung

Interrupt? Für Pause oder Start und mehrere LEDS gehen nach einander an.

8.6.1. Manual

Erklärung: In diesem Programm werden drei Druckknöpfe verwendet: Start, Pause und Stop. Der Start-Druckknopf startet eine LED-Abfolge, bei der fünf LEDs nacheinander aufleuchten. Der Pause-Druckknopf hält die Abfolge an und setzt sie beim erneuten Drücken fort. Der Stop-Druckknopf beendet die Abfolge und schaltet alle LEDs aus.

Anschluss: Die drei Druckknöpfe werden jeweils mit einem Anschluss an GND angeschlossen. Die anderen Anschlüsse kommen an Pin 2, 3 und 4. Die LEDs werden jeweils mit 220 Ohm oder 330 Ohm Vorwiderstand an Pin 8 bis 12 angeschlossen. Der lange Anschluss der LED kommt zum Arduino-Pin, der kurze Anschluss zu GND. Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()`
- `digitalRead()`
- `digitalWrite()`
- `digitalWrite()`
- `millis()` Liefert die Zeit seit dem Start des Arduino in Millisekunden. [Ardh]
- `delay()`

8.6.2. Code

Listing 8.3.: Druckknöpfe Led Abfolge

```

1000 /**
1001  * @file druckknoepfedabfolge.ino
1002  * @brief Controls an LED sequence with three push buttons.
1003  *
1004  * The start push button starts the LED sequence.
1005  * The pause push button pauses or continues the sequence.
1006  * The stop push button stops the sequence and turns all LEDs off.
1007  */
1008
1009 const int startdruckknopf = 2; ///< Input pin for the start push button.
1010 const int pausedruckknopf = 3; ///< Input pin for the pause push button.
1011 const int stopdruckknopf = 4;  ///< Input pin for the stop push button.
1012
1013 const int ledPins[] = {8, 9, 10, 11, 12}; ///< Output pins for the LEDs.
1014 const int anzahlLeds = 5;                ///< Number of connected LEDs.
1015
1016 bool programmLaeuft = false; ///< Shows whether the LED sequence is
1017                               active.
1018 bool programmPause = false;  ///< Shows whether the LED sequence is
1019                               paused.
1020
1021 int aktuelleLed = 0;          ///< Number of the currently active
1022                               LED.
1023 unsigned long letzteZeit = 0; ///< Time of the last LED change.
1024 const unsigned long intervall = 500; ///< Time between LED changes in
1025                                       milliseconds.
1026
1027 bool letzterPauseStatus = HIGH; ///< Previous state of the pause push
1028                                   button.
1029
1030 /**
1031  * @brief Runs once when the Arduino starts.
1032  *
1033  * The push buttons are set as inputs with internal pull-up resistors.
1034  * The LED pins are set as outputs.
1035  */
1036 void setup() {
1037     pinMode(startdruckknopf, INPUT_PULLUP); ///< Start push button as
1038     input.
1039     pinMode(pausedruckknopf, INPUT_PULLUP); ///< Pause push button as
1040     input.
1041     pinMode(stopdruckknopf, INPUT_PULLUP);  ///< Stop push button as input
1042     .

```

```

1036   for (int i = 0; i < anzahlLeds; i++) {
1038       pinMode(ledPins[i], OUTPUT); ///< Sets the LED pin as output.
1040   }
1042   alleLedsAus();
1044   /**
1046   * @brief Main program that runs repeatedly.
1048   * It checks the three push buttons and controls the LED sequence
1050   * depending on the input.
1052   */
1054   void loop() {
1056       if (digitalRead(startdruckknopf) == LOW) {
1058           programmLaeuft = true;
1060           programmPause = false;
1062       }
1064       if (digitalRead(stopdruckknopf) == LOW) {
1066           programmLaeuft = false;
1068           programmPause = false;
1070           aktuelleLed = 0;
1072           alleLedsAus();
1074       }
1076       bool aktuellerPauseStatus = digitalRead(pausedruckknopf); ///< Current
1078       state of the pause push button.
1080       if (letzterPauseStatus == HIGH && aktuellerPauseStatus == LOW) {
1082           programmPause = !programmPause;
1084           delay(200); ///< Simple debounce for the push button.
1086       }
1088       letzterPauseStatus = aktuellerPauseStatus;
1090       if (programmLaeuft && !programmPause) {
1092           ledAbfolge();
1094       }
1096   }
1098   /**
1100   * @brief Turns all LEDs off.
1102   * All LED pins are set to LOW.
1104   */
1106   void alleLedsAus() {
1108       for (int i = 0; i < anzahlLeds; i++) {
1110           digitalWrite(ledPins[i], LOW);
1112       }
1114   }
1116   /**
1118   * @brief Makes the LEDs light up one after another.
1120   * After the defined interval has passed, the current LED is turned off
1122   * and the next LED is turned on.
1124   */
1126   void ledAbfolge() {
1128       unsigned long aktuelleZeit = millis(); ///< Current runtime of the
1130       Arduino in milliseconds.
1132       if (aktuelleZeit - letzteZeit >= intervall) {
1134           letzteZeit = aktuelleZeit;
1136           alleLedsAus();
1138           digitalWrite(ledPins[aktuelleLed], HIGH);
1140           aktuelleLed++;

```

```
1104     if (aktuelleLed >= anzahlLeds) {  
1106         aktuelleLed = 0;  
1108     }  
}
```

../..Code/druckknoepfedabfolge/druckknoepfedabfolge.ino

8.7. Further Readings

9. MOSFET PWM-Schaltmodul

9.1. Einleitung

Das MOSFET PWM-Schaltmodul dient zur Steuerung elektrischer Lasten mit Hilfe eines Mikrocontrollers. MOSFET-Module werden in der Leistungselektronik eingesetzt, da sie hohe Ströme bei geringem Steuerstrom schalten können [All24].

Durch die Ansteuerung mit Pulsweitenmodulation (PWM) kann die mittlere Leistung eines Verbrauchers geregelt werden. Dadurch eignen sich MOSFET-Module besonders für Anwendungen wie Motorsteuerungen, Heizsysteme oder LED-Dimmer [TSG23].

9.2. Grundlegende Informationen

MOSFETs (Metal-Oxide-Semiconductor Field-Effect Transistors) gehören zur Gruppe der spannungsgesteuerten Halbleiterbauelemente. Sie werden als elektronische Schalter oder zur Leistungsregelung eingesetzt [All24].

Ein MOSFET besitzt die drei Anschlüsse Gate, Drain und Source. Über den Gate-Anschluss wird die Leitfähigkeit zwischen Drain und Source gesteuert. Liegt eine ausreichende Gate-Source-Spannung an, entsteht ein leitfähiger Kanal, durch den Strom fließen kann [TSG23].

MOSFETs besitzen einen sehr hohen Eingangswiderstand und benötigen daher nur einen sehr kleinen Steuerstrom. Dadurch eignen sie sich besonders zur direkten Ansteuerung durch Mikrocontroller [All24].

Zur Leistungsregelung wird häufig die Pulsweitenmodulation verwendet. Dabei wird ein Verbraucher periodisch ein- und ausgeschaltet. Durch die Veränderung des Tastverhältnisses lässt sich die mittlere Ausgangsleistung beeinflussen [TSG23].

Für Schaltanwendungen wird der MOSFET möglichst im ohmschen Bereich betrieben. In diesem Bereich verhält sich der MOSFET näherungsweise wie ein kleiner Widerstand zwischen Drain und Source [All24].

9.3. Spezifisches MOSFET-Modul

Verwendet wird ein JOY-IT MOSFET PWM-Schaltmodul. Das Modul dient zur Schaltung und Regelung von Gleichstromverbrauchern über ein PWM-Signal.

Auf dem Modul ist ein Leistungs-MOSFET vom Typ IRF9540N verbaut. Der IRF9540N ist ein p-Kanal-Leistungs-MOSFET und wird für Schaltanwendungen in der Leistungselektronik verwendet. Durch den geringen Durchlasswiderstand können höhere Ströme mit vergleichsweise geringer Verlustleistung geschaltet werden.

Das MOSFET-Modul kann mit Mikrocontrollern angesteuert werden und eignet sich zur Steuerung von Lasten mit höherer Stromaufnahme [Joyb].

9.4. Spezifikation

Die technischen Daten wurden dem Datenblatt des Herstellers entnommen [Joyb].

- Eingangsspannung: DC 5 V bis 36 V
- Steuerspannung: 3,3 V bis 20 V

- PWM-Frequenzbereich: 0 kHz bis 20 kHz
- Maximale Ausgangsleistung: 400 W
- Maximaler Dauerstrom: 15 A
- Betriebstemperatur: -40 °C bis +85 °C
- Abmessungen: 34 mm × 17 mm × 12 mm

9.4.1. Eigenschaften des IRF9540N

Der verwendete IRF9540N besitzt folgende Eigenschaften:

- p-Kanal-Leistungs-MOSFET
- Einsatz als elektronischer Schalter
- hoher Eingangswiderstand
- geringer Drain-Source-Durchlasswiderstand
- geeignet für PWM-Anwendungen

Der MOSFET wird auf dem Modul zur Schaltung der angeschlossenen Last verwendet.

9.4.2. Anschlüsse

Das Modul besitzt folgende Anschlüsse:

- SIG
- VCC
- GND
- VIN
- VOUT

Das PWM-Steuersignal wird über den SIG-Anschluss eingespeist. Die Versorgung der Last erfolgt über die Schraubklemmen VIN und VOUT [Joyb].

9.5. Bibliothek

9.5.1. Beschreibung

Für das MOSFET PWM-Schaltmodul wird keine zusätzliche Bibliothek benötigt. Die Ansteuerung erfolgt über die Standardfunktionen der Arduino-IDE.

9.5.2. Installation

Das Modul wird mit einem PWM-fähigen Ausgang des Mikrocontrollers verbunden.

- SIG → PWM-Ausgang des Mikrocontrollers
- VCC → Versorgungsspannung
- GND → gemeinsame Masse

Die Last wird über die Schraubklemmen VIN und VOUT angeschlossen.

Die Masse des Mikrocontrollers und die Masse der externen Spannungsversorgung müssen miteinander verbunden werden, damit ein gemeinsames Bezugspotential vorhanden ist.

9.5.3. Funktionen

Zur Steuerung des Moduls können folgende Arduino-Funktionen verwendet werden:

- `pinMode()`
- `digitalWrite()`
- `analogWrite()`
- `delay()`

Mit `digitalWrite()` kann die Last ein- oder ausgeschaltet werden. Mit `analogWrite()` kann ein PWM-Signal erzeugt werden, wodurch sich die mittlere Ausgangsleistung regeln lässt.

9.6. Beispiel

In diesem Beispiel wird eine elektrische Last über ein PWM-Signal angesteuert.

9.6.1. Versuchsaufbau

Das PWM-Signal des Mikrocontrollers wird mit dem SIG-Eingang des MOSFET-Moduls verbunden. Die Last wird über die Schraubklemmen mit einer externen Spannungsversorgung verbunden.

Als Testlast kann zunächst eine LED mit Vorwiderstand oder eine kleine Gleichstromlampe verwendet werden.

9.6.2. Funktionsweise des Beispiels

Der Mikrocontroller erzeugt mit der Funktion `analogWrite()` ein PWM-Signal. Das MOSFET-Modul schaltet dadurch den Stromfluss periodisch ein und aus. Durch die Veränderung des Tastverhältnisses wird die mittlere Leistung der Last beeinflusst.

9.6.3. Code

Ein einfacher Arduino-Code kann verwendet werden, um unterschiedliche PWM-Werte zu testen.

9.6.4. Dateien

- `MosfetSimpleSwitch.ino`
- `MosfetPWMTTest.ino`
- `MosfetHotwireTest.ino`

9.7. Kalibrierung

Für Anwendungen mit Heizdraht muss ein geeigneter PWM-Wert experimentell bestimmt werden.

Dazu wird die PWM-Stufe schrittweise erhöht, bis die gewünschte Temperatur erreicht wird. Dabei dürfen die zulässige Stromaufnahme des Moduls und die Belastbarkeit der Spannungsversorgung nicht überschritten werden.

Listing 9.1.: Einfacher Funktionstest des MOSFET PWM-Schaltmoduls

```

1000 /**
1001  * @file MosfetPWMTTest.ino
1002  * @brief Tests different PWM output values for a MOSFET PWM switching
1003  *       module.
1004  */
1005
1006 /// PWM-capable output pin connected to the MOSFET module signal input.
1007 const int MOSFET_PIN = 9;
1008
1009 /// PWM values used for the test sequence.
1010 const int PWM_VALUES[] = {0, 64, 128, 192, 255};
1011
1012 /// Number of PWM values in the test sequence.
1013 const int NUMBER_OF_VALUES = sizeof(PWM_VALUES) / sizeof(PWM_VALUES[0]);
1014
1015 /// Time in milliseconds for each PWM value.
1016 const unsigned long PWM_STEP_DELAY_MS = 5000;
1017
1018 /**
1019  * @brief Initializes the MOSFET control pin.
1020  */
1021 void setup() {
1022     pinMode(MOSFET_PIN, OUTPUT);
1023     analogWrite(MOSFET_PIN, 0);
1024 }
1025
1026 /**
1027  * @brief Cycles through predefined PWM values.
1028  */
1029 void loop() {
1030     for (int i = 0; i < NUMBER_OF_VALUES; i++) {
1031         analogWrite(MOSFET_PIN, PWM_VALUES[i]);
1032         delay(PWM_STEP_DELAY_MS);
1033     }
1034     analogWrite(MOSFET_PIN, 0);
1035     delay(PWM_STEP_DELAY_MS);
1036 }

```

../Code/MosfetSimpleSwitch/MosfetSimpleSwitch.ino

9.8. Einfacher Code

Der folgende Code prüft, ob das MOSFET PWM-Schaltmodul eine Last ein- und ausschalten kann.

9.9. Einfache Anwendung

Das MOSFET PWM-Schaltmodul kann zur Regelung von Gleichstromverbrauchern wie Heizdrähten, Motoren oder LED-Beleuchtungen verwendet werden.

Für einen CNC-Hot-Wire-Foam-Cutter kann das Modul die Leistung des Heizdrahts über PWM verändern. Dadurch kann die Temperatur des Drahts an das verwendete Material angepasst werden.

Listing 9.2.: PWM-Test des MOSFET PWM-Schaltmoduls

```

1000 /**
1001  * @file MosfetHotwireTest.ino
1002  * @brief PWM test for controlling a hot wire with a MOSFET module.
1003  */
1004
1005 /// PWM-capable output pin connected to the MOSFET module signal input.
1006 const int MOSFET_PIN = 9;
1007
1008 /// Conservative PWM values for a careful hot wire test.
1009 const int PWM_VALUES[] = {0, 30, 60, 90, 120, 150};
1010
1011 /// Number of PWM values in the test sequence.
1012 const int NUMBER_OF_VALUES = sizeof(PWM_VALUES) / sizeof(PWM_VALUES[0]);
1013
1014 /// Time in milliseconds for each PWM test step.
1015 const unsigned long PWM_STEP_DELAY_MS = 8000;
1016
1017 /// Cooling pause in milliseconds after each full test cycle.
1018 const unsigned long COOLING_DELAY_MS = 10000;
1019
1020 /**
1021  * @brief Initializes the MOSFET control pin and keeps the output off.
1022  */
1023 void setup() {
1024     pinMode(MOSFET_PIN, OUTPUT);
1025     analogWrite(MOSFET_PIN, 0);
1026 }
1027
1028 /**
1029  * @brief Applies increasing PWM values and then switches the output off
1030  */
1031 void loop() {
1032     for (int i = 0; i < NUMBER_OF_VALUES; i++) {
1033         analogWrite(MOSFET_PIN, PWM_VALUES[i]);
1034         delay(PWM_STEP_DELAY_MS);
1035     }
1036
1037     analogWrite(MOSFET_PIN, 0);
1038     delay(COOLING_DELAY_MS);
1039 }
1040

```

../Code/MosfetPWMTTest/MosfetPWMTTest.ino

9.10. Tests

9.10.1. Einfacher Funktionstest

Beim einfachen Funktionstest wird eine Testlast angeschlossen. Das Modul wird für mehrere Sekunden eingeschaltet und anschließend wieder ausgeschaltet.

Dabei wird überprüft:

- ob die Last korrekt schaltet
- ob die Verdrahtung korrekt ist
- ob sich das Modul ungewöhnlich stark erwärmt

9.10.2. Test aller Funktionen

Beim vollständigen Test werden mehrere PWM-Werte nacheinander ausgegeben. Dabei wird überprüft, ob sich die Leistung der Last sichtbar oder messbar verändert.

Für den Test werden folgende Punkte kontrolliert:

- Schalten der Last bei digitalem Signal
- Veränderung der Lastleistung bei verschiedenen PWM-Werten
- Erwärmung des MOSFET-Moduls
- Erwärmung der Anschlussleitungen
- Stromaufnahme der Last

9.11. Weiterführende Literatur

- Alles, Martin: *Einführung in die elektronische Schaltungstechnik*. Springer Vieweg, 2024.
- Tietze, Ulrich; Schenk, Christoph: *Halbleiter-Schaltungstechnik*. Springer Vieweg, 2023.

Listing 9.3.: Testprogramm für die Heizdrahtansteuerung

```
1000 /**
1001  * @file A4988SimpleStep.ino
1002  * @brief Basic function test for an A4988 stepper motor driver.
1003  */
1004
1005 const uint8_t STEP_PIN = 2;
1006 const uint8_t DIR_PIN  = 5;
1007 const uint8_t EN_PIN   = 8;
1008
1009 const unsigned int STEP_DELAY_US = 1000;
1010
1011 /**
1012  * @brief Initializes the output pins and enables the driver.
1013  */
1014 void setup()
1015 {
1016     pinMode(STEP_PIN, OUTPUT);
1017     pinMode(DIR_PIN, OUTPUT);
1018     pinMode(EN_PIN, OUTPUT);
1019
1020     digitalWrite(DIR_PIN, HIGH);
1021     digitalWrite(EN_PIN, LOW);
1022 }
1023
1024 /**
1025  * @brief Generates continuous step pulses.
1026  */
1027 void loop()
1028 {
1029     digitalWrite(STEP_PIN, HIGH);
1030     delayMicroseconds(STEP_DELAY_US);
1031
1032     digitalWrite(STEP_PIN, LOW);
1033     delayMicroseconds(STEP_DELAY_US);
1034 }
```

../Code/MosfetHotwireTest/MosfetHotwireTest.ino

10. A4988-Schrittmotortreiber

10.1. Einleitung

Für die Ansteuerung der Schrittmotoren im Projekt wird der Schrittmotortreiber A4988 verwendet. Der A4988 ist ein integrierter DMOS-Microstepping-Treiber für bipolare Schrittmotoren und wird in Positioniersystemen, CNC-Systemen und ähnlichen Anwendungen eingesetzt [All22].

Im Projekt übernimmt der A4988 die Leistungsansteuerung der Achsmotoren. Dadurch können die Bewegungen des Heizdrahts definiert und wiederholbar ausgeführt werden.

10.2. Grundlagen

Schrittmotoren gehören zu den elektrischen Antrieben, bei denen die Drehbewegung in einzelne Schritte unterteilt wird. Dadurch können Positionen gezielt angefahren werden, ohne dass zwingend ein zusätzliches Positionsmesssystem erforderlich ist [Ise08].

Für die Ansteuerung eines bipolaren Schrittmotors muss die Stromrichtung in den Motorwicklungen umgeschaltet werden. Diese Aufgabe wird durch leistungselektronische Schaltungen übernommen [Zac22]. Der A4988 enthält hierfür integrierte H-Brücken und eine interne Stromregelung.

Die Ansteuerung erfolgt über die Signale STEP und DIR. Mit jeder steigenden Flanke am STEP-Eingang bewegt der Treiber den Motor um einen Schritt beziehungsweise Mikroschritt weiter. Das DIR-Signal bestimmt die Drehrichtung [All22].

10.3. Spezifischer Sensor/Aktor

Im Projekt wird ein A4988-kompatibles Treibermodul verwendet. Der eigentliche Treiber-IC stammt von Allegro MicroSystems. Laut Herstellerdatenblatt unterstützt der A4988 Voll-, Halb-, Viertel-, Achtel- und Sechzehntelschrittbetrieb [All22]. Durch Microstepping kann die Bewegung der Achsen gleichmäßiger ausgeführt werden.

Der A4988 besitzt eine integrierte PWM-Stromregelung. Dadurch wird der Strom durch die Motorwicklungen begrenzt. Dies ist wichtig, da der Motor mit einer höheren Versorgungsspannung betrieben werden kann, ohne die Wicklungen zu überlasten.

Zusätzlich verfügt der Treiber über Schutzfunktionen gegen Überstrom, Kurzschluss, Unterspannung und Übertemperatur [All22].

10.4. Spezifikation

Die technischen Daten des verwendeten A4988 wurden dem Herstellerdatenblatt entnommen [All22].

- Motorspannung: 8 V bis 35 V
- Logikspannung: 3 V bis 5,5 V
- maximaler Ausgangsstrom: bis ± 2 A
- Schrittmodi: Vollschritt, Halbschritt, Viertelschritt, Achtelschritt und Sechzehntelschritt

- Steuerung über STEP- und DIR-Eingang
- integrierte PWM-Stromregelung
- Schutzfunktionen gegen Übertemperatur, Unterspannung und Überstrom

Der maximale Motorstrom ergibt sich aus:

$$I_{TripMAX} = \frac{V_{REF}}{8 \cdot R_S}$$

Dabei beschreibt R_S den Sense-Widerstand und V_{REF} die Referenzspannung [All22].

10.5. Bibliothek

10.5.1. Beschreibung

Für den A4988 wird keine zusätzliche Arduino-Bibliothek benötigt. Die Ansteuerung erfolgt direkt über digitale Ausgänge des Mikrocontrollers. Im späteren Betrieb des CNC-Hot-Wire-Foam-Cutters erzeugt die GRBL-Firmware die notwendigen STEP- und DIR-Signale.

10.5.2. Installation

Der A4988 wird direkt auf das CNC Shield V3 gesteckt. Bei Verwendung des CNC Shields sind die Anschlüsse für STEP, DIR und ENABLE bereits den jeweiligen Achsen zugeordnet.

Vor dem Anschluss des Motors muss die Strombegrenzung eingestellt werden. Außerdem müssen die Masse des Mikrocontrollers und die Masse der externen Motorspannungsvorsorgung miteinander verbunden werden.

10.5.3. Funktionen

Zur direkten Steuerung des A4988 werden Standardfunktionen der Arduino-IDE verwendet. Mit `pinMode()` werden die verwendeten Pins als Ausgänge konfiguriert. Mit `digitalWrite()` werden STEP-, DIR- und ENABLE-Signale ausgegeben. Die Funktion `delayMicroseconds()` bestimmt die Zeit zwischen den Schrittimpulsen und damit die Drehgeschwindigkeit.

10.6. Beispiel

In diesem Abschnitt wird ein einfaches Beispiel beschrieben, das die grundlegende Funktion des A4988 zeigt. Der Mikrocontroller erzeugt Schrittimpulse, die vom Treiber in Motorbewegungen umgesetzt werden.

10.6.1. Versuchsaufbau

Der A4988 wird mit einem bipolaren Schrittmotor verbunden. Die Motorspannung wird über ein externes Netzteil bereitgestellt. Der Arduino erzeugt die STEP- und DIR-Signale. Bei Verwendung des CNC Shield V3 wird der Treiber auf den Steckplatz der gewünschten Achse gesteckt.

10.6.2. Funktionsweise des Beispiels

Im Beispiel wird zunächst die Drehrichtung festgelegt. Anschließend werden am STEP-Eingang periodische Impulse ausgegeben. Jeder Impuls bewegt den Motor um einen Schritt oder Mikroschritt weiter.

10.6.3. Code

Für die Dokumentation werden drei Programme verwendet. Das erste Programm prüft die Grundfunktion des Treibers. Das zweite Programm zeigt eine einfache projektbezogene Anwendung. Das dritte Programm dient als vollständiger Funktionstest.

10.6.4. Dateien

- A4988SimpleStep.ino
- A4988FoamCutterAxis.ino
- A4988DriverTest.ino

10.7. Kalibrierung

Vor dem Betrieb muss die Strombegrenzung des A4988 eingestellt werden. Die Einstellung erfolgt über ein Potentiometer auf dem Treibermodul.

Eine zu hohe Strombegrenzung führt zu starker Erwärmung und kann eine thermische Abschaltung verursachen. Eine zu niedrige Strombegrenzung kann Schrittverluste verursachen. Für den CNC-Hot-Wire-Foam-Cutter wird die Strombegrenzung so eingestellt, dass der Motor zuverlässig läuft und der Treiber nicht übermäßig heiß wird.

10.8. Einfacher Code

Der einfache Code dient zur Überprüfung der Grundfunktion des A4988. Der Motor wird kontinuierlich in eine Richtung bewegt. Dadurch können Verdrahtung, Spannungsversorgung, STEP-Signal und Treiberfunktion überprüft werden.

Listing 10.1.: Einfacher Funktionstest des A4988-Schrittmotortreibers

```
1000 /**
1001  * @file A4988SimpleStep.ino
1002  * @brief Basic function test for an A4988 stepper motor driver.
1003  */
1004
1005 const uint8_t STEP_PIN = 2;
1006 const uint8_t DIR_PIN = 5;
1007 const uint8_t EN_PIN = 8;
1008
1009 const unsigned int STEP_DELAY_US = 1000;
1010
1011 /**
1012  * @brief Initializes the output pins and enables the driver.
1013  */
1014 void setup()
1015 {
1016     pinMode(STEP_PIN, OUTPUT);
1017     pinMode(DIR_PIN, OUTPUT);
1018     pinMode(EN_PIN, OUTPUT);
1019
1020     digitalWrite(DIR_PIN, HIGH);
1021     digitalWrite(EN_PIN, LOW);
1022 }
1023
1024 /**
1025  * @brief Generates continuous step pulses.
1026  */
1027 void loop()
1028 {
1029     digitalWrite(STEP_PIN, HIGH);
1030     delayMicroseconds(STEP_DELAY_US);
1031
1032     digitalWrite(STEP_PIN, LOW);
1033     delayMicroseconds(STEP_DELAY_US);
1034 }
```

../Code/A4988SimpleStep/A4988SimpleStep.ino

10.9. Einfache Anwendung

Die einfache Anwendung simuliert eine Achsbewegung des CNC-Hot-Wire-Foam-Cutters. Die Achse bewegt sich langsam in Schneidrichtung und anschließend schneller zurück. Dadurch wird der praktische Einsatz des A4988 im Projekt demonstriert.

Listing 10.2.: Einfache Achsbewegung des CNC-Hot-Wire-Foam-Cutters

```

1000 /**
1001  * @file A4988FoamCutterAxis.ino
1002  * @brief Simple CNC axis movement for a hot wire foam cutter.
1003  */
1004
1005
1006 const uint8_t STEP_PIN = 2;
1007 const uint8_t DIR_PIN  = 5;
1008 const uint8_t EN_PIN   = 8;
1009
1010 const unsigned long AXIS_TRAVEL_STEPS = 1600;
1011
1012 const unsigned int CUTTING_FEED_DELAY_US = 1500;
1013 const unsigned int RETURN_FEED_DELAY_US  = 700;
1014
1015 /**
1016  * @brief Moves the axis with a defined speed.
1017  *
1018  * @param steps Number of steps.
1019  * @param delayUs Delay between step edges in microseconds.
1020  */
1021 void moveAxis(unsigned long steps, unsigned int delayUs)
1022 {
1023     for (unsigned long i = 0; i < steps; i++)
1024     {
1025         digitalWrite(STEP_PIN, HIGH);
1026         delayMicroseconds(delayUs);
1027
1028         digitalWrite(STEP_PIN, LOW);
1029         delayMicroseconds(delayUs);
1030     }
1031 }
1032
1033 /**
1034  * @brief Initializes the driver pins.
1035  */
1036 void setup()
1037 {
1038     pinMode(STEP_PIN, OUTPUT);
1039     pinMode(DIR_PIN, OUTPUT);
1040     pinMode(EN_PIN, OUTPUT);
1041
1042     digitalWrite(EN_PIN, LOW);
1043 }
1044
1045 /**
1046  * @brief Simulates a simple CNC axis movement.
1047  */
1048 void loop()
1049 {
1050     digitalWrite(DIR_PIN, HIGH);
1051     moveAxis(AXIS_TRAVEL_STEPS, CUTTING_FEED_DELAY_US);
1052
1053     delay(1500);
1054
1055     digitalWrite(DIR_PIN, LOW);
1056     moveAxis(AXIS_TRAVEL_STEPS, RETURN_FEED_DELAY_US);
1057
1058     delay(1500);
1059 }

```

../Code/A4988FoamCutterAxis/A4988FoamCutterAxis.ino

10.10. Tests

10.10.1. Einfacher Funktionstest

Beim einfachen Funktionstest wird überprüft, ob der Motor grundsätzlich auf STEP-Signale reagiert. Dieser Test erfolgt mit dem Programm aus Abschnitt 10.1. Dabei werden die Verdrahtung, die Versorgungsspannung und die Drehrichtung kontrolliert.

10.10.2. Test aller Funktionen

Beim vollständigen Test werden Drehrichtung, Geschwindigkeit, ENABLE-Signal und Temperaturverhalten überprüft. Der Motor wird in beide Richtungen bewegt. Zusätzlich wird der Treiber kurz deaktiviert und anschließend wieder aktiviert. Während des Tests wird beobachtet, ob der Motor Schritte verliert oder ob sich der Treiber übermäßig erwärmt.

Listing 10.3.: Testprogramm für den A4988-Schrittmotortreiber

```

1000 /**
1001  * @file A4988DriverTest.ino
1002  * @brief Test program for the A4988 stepper motor driver.
1003  */
1004
1005 const uint8_t STEP_PIN = 2;
1006 const uint8_t DIR_PIN  = 5;
1007 const uint8_t EN_PIN   = 8;
1008
1009 const unsigned int STEPS_PER_TEST = 400;
1010
1011 /**
1012  * @brief Generates a defined number of step pulses.
1013  *
1014  * @param steps Number of step pulses.
1015  * @param delayUs Delay between HIGH and LOW level in microseconds.
1016  */
1017 void moveSteps(unsigned int steps, unsigned int delayUs)
1018 {
1019     for (unsigned int i = 0; i < steps; i++)
1020     {
1021         digitalWrite(STEP_PIN, HIGH);
1022         delayMicroseconds(delayUs);
1023
1024         digitalWrite(STEP_PIN, LOW);
1025         delayMicroseconds(delayUs);
1026     }
1027 }
1028
1029 /**
1030  * @brief Initializes the output pins.
1031  */
1032 void setup()
1033 {
1034     pinMode(STEP_PIN, OUTPUT);
1035     pinMode(DIR_PIN, OUTPUT);
1036     pinMode(EN_PIN, OUTPUT);
1037
1038     digitalWrite(EN_PIN, LOW);
1039 }
1040
1041 /**
1042  * @brief Executes a complete driver function test.
1043  */
1044 void loop()
1045 {
1046     digitalWrite(DIR_PIN, HIGH);
1047     moveSteps(STEPS_PER_TEST, 1200);
1048
1049     delay(1000);
1050
1051     digitalWrite(DIR_PIN, LOW);
1052     moveSteps(STEPS_PER_TEST, 1200);
1053
1054     delay(1000);
1055
1056     digitalWrite(DIR_PIN, HIGH);
1057     moveSteps(STEPS_PER_TEST, 600);
1058
1059     delay(1000);
1060
1061     digitalWrite(EN_PIN, HIGH);
1062
1063     delay(2000);
1064
1065     digitalWrite(EN_PIN, LOW);
1066
1067     delay(1000);
1068 }

```

../Code/A4988DriverTest/A4988DriverTest.ino

10.11. Weiterführende Literatur

- Isermann, Rolf: *Mechatronische Systeme*. Springer, 2008. [Ise08]
- Zach, Franz: *Leistungselektronik*. Springer Vieweg, 2022. [Zac22]
- Allegro MicroSystems: *A4988 DMOS Microstepping Driver with Translator and Overcurrent Protection*. Datenblatt, 2022. [All22]

Teil III.

Domain Knowledge - Tools

Teil IV.

Development

Teil V.

Monitoring

Teil VI.

**Verification - Evaluation -
Conclusion**

Teil VII.

Anhang

11. Bill of Material

11.1. Mechanische Komponenten

Tabelle 11.1.: Mechanische Komponenten

Bild	Menge	Beschreibung	Lieferant	Preis (€)
–	6 (4 benötigt)	Basisplatte für Sensor	Creality 3D	
–	6 (4 benötigt)	Sensor Gehäuse	Creality 3D	
–	2	Zahnriemen 5,5 x 1200		
–	10	Rollen (gelagert), außen 24 mm, innen 4,5 mm, Breite 11 mm	Creality 3D	
–	4	Führungsstange d=10 mm, l=525 mm	Creality 3D	
–	4 (2 benötigt)	Führungsstange d=10 mm, l=645 mm		
–	4	Alu-Extrusionsprofil 20x20, l=554 mm		
–	4	Alu-Extrusionsprofil 20x20, l=498 mm		
–	4	Alu-Extrusionsprofil 20x20, l=510 mm		
–	2	Alu-Extrusionsprofil 20x20, l=514 mm		
–	2	Welle für Riemenantrieb d=8 mm		
–	2	Spindel d=8 mm, l=470 mm		
–	2	Spindelmuffe		
–	2	Flachspiralfeder		
–		Befestigung für Führung		

11.2. Elektronische Komponenten

Tabelle 11.2.: Elektronische Komponenten

Bild	Menge	Beschreibung	Lieferant	Preis (€)
–	1	Arduino Uno R3		
–	1	CNC Shield V3		
–	4	Stepper Motor Driver		
–		Jumper Kabel		
–	4	Stepper Motor	Creality 3D	
–	8 (6 benötigt)	Endschalter	www.3djake.de	4,99
–	4 (4 benötigt)	Druckknopf	reichelt.de	
–	1 (1 benötigt)	OLED Display 0,96"	reichelt.de	15,20
–	1 (1 benötigt)	MOSFET (IRLZ44N)	reichelt.de	0,70
–	1	Netzteil		
–	1	Netzkabel (Computerkabel)		
–	1	Heizdraht (Nichrom)		

11.3. Schrauben und Befestigungen

Tabelle 11.3.: Schrauben und Befestigungen

Bild	Menge	Beschreibung	Lieferant	Preis (€)
–		Zylinderschraube M5x20	Creality 3D	
–		Linsenkopfschraube M5x18	Creality 3D	
–		Zylinderschraube M4x8	Creality 3D	
–		Spanplattenschraube / Holzschraube 4x13 Senkkopf Kreuzschlitz	Creality 3D	
–		Zylinderschraube M3x14	Creality 3D	
–		Linsenkopfschraube M3x8	Creality 3D	
–		Federring M5	Creality 3D	
–		Federring M4	Creality 3D	
–		Federring M3	Creality 3D	

Literaturverzeichnis

- [3dj] *Creality Endschalter*. 2026. URL: <https://www.3djake.de/creality-3d-drucker-ersatzteile/endschalter> (besucht am 26.04.2026).
- [All22] Allegro MicroSystems. *A4988 DMOS Microstepping Driver with Translator and Overcurrent Protection*. 4988-DS Rev. 8. Datasheet. Allegro MicroSystems. 5. Apr. 2022. URL: <https://www.allegromicro.com>.
- [All24] M. Alles. *Einführung in die elektronische Schaltungstechnik*. Springer Vieweg, 2024. ISBN: 978-3-658-45570-5.
- [Arda] *Digital Pins - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/variablen/constants/inputOutputPullup/> (besucht am 25.04.2026).
- [Ardb] *Serial.begin() - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/funktionen/communication/serial/begin/> (besucht am 10.05.2026).
- [Ardc] *Serial.print() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/communication/serial/print/> (besucht am 25.04.2026).
- [Ardd] *delay() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/time/delay/> (besucht am 25.04.2026).
- [Arde] *digitalRead() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/digitalread/> (besucht am 25.04.2026).
- [Ardf] *digitalWrite() - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/funktionen/digital-io/digitalwrite/> (besucht am 10.05.2026).
- [Ardg] *loop() - Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/loop/> (besucht am 25.04.2026).
- [Ardh] *millis() - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/funktionen/time/millis/> (besucht am 10.05.2026).
- [Ardi] *pinMode() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/pinMode/> (besucht am 25.04.2026).
- [Ardj] *setup() - Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/setup/> (besucht am 25.04.2026).
- [Ard24] Arduino Documentation. *The Arduino Web Editor*. 2024. URL: <https://docs.arduino.cc/learn/starting-guide/the-arduino-web-editor/>.
- [Ard25a] Arduino. *Arduino Cloud*. 2025. URL: <https://docs.arduino.cc/cloud/iot-cloud> (besucht am 14.05.2026).
- [Ard25b] Arduino. *Libraries*. 2025. URL: <https://docs.arduino.cc/libraries/> (besucht am 14.05.2026).

- [Ard25c] Arduino. *Overview of the Arduino UNO Components*. 2025. URL: <https://docs.arduino.cc/tutorials/uno-rev3/intro-to-board/> (besucht am 12.05.2026).
- [Ard26a] Arduino. *Arduino Language Reference*. 2026. URL: <https://docs.arduino.cc/language-reference/> (besucht am 12.05.2026).
- [Ard26b] Arduino. *Arduino UNO R3 Datasheet*. 2026. URL: <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf> (besucht am 12.05.2026).
- [Ard26c] Arduino. *UNO R3*. 2026. URL: <https://docs.arduino.cc/hardware/uno-rev3/> (besucht am 12.05.2026).
- [Ard26d] Arduino. *analogRead()*. 2026. URL: <https://docs.arduino.cc/language-reference/en/functions/analog-io/analogread/> (besucht am 12.05.2026).
- [Ard26e] Arduino. *analogWrite()*. 2026. URL: <https://docs.arduino.cc/language-reference/en/functions/analog-io/analogwrite/> (besucht am 12.05.2026).
- [Bas] *Creality Endschanter*. 2026. URL: <https://www.bastelgarage.ch/creality-endschanter-limit-switch> (besucht am 26.04.2026).
- [Ber24] H. Bernstein. *Messelektronik und Sensoren. Grundlagen der Messtechnik, Sensoren, analoge und digitale Signalverarbeitung*. 2. Aufl. Springer Vieweg Wiesbaden, 25. Jan. 2024. ISBN: 978-3-658-38929-1. DOI: <https://doi.org/10.1007/978-3-658-38929-1>.
- [FAD18] M. Fezari und A. Al Dahoud. "Integrated Development Environment "IDE" For Arduino". In: (Okt. 2018).
- [Hel] W. Helbig. *Praxiswissen in der Messtechnik. Arbeitsbuch für Techniker, Ingenieure und Studenten*.
- [Her+12] E. Hering u. a. *Elektrotechnik und Elektronik für Maschinenbauer*. Bd. 2. Springer, 14. Feb. 2012. ISBN: 978-3-642-12881-3.
- [Ise08] R. Isermann. *Mechatronische Systeme. Grundlagen*. 2. Aufl. Springer, 2008. ISBN: 978-3-540-32336-5. DOI: <https://doi.org/10.1007/978-3-540-32512-3>.
- [Joya] *ARKADE-BUTTONS MIT MIKROSCHALTER*. 2024.
- [Joyb] *MOSFET PWM-Schaltmodul Datenblatt*. 2025. URL: <https://joy-it.net> (besucht am 20.05.2026).
- [Rs2] *Drucktaster von Eaton: Ein Leitfaden*. 2026. URL: <https://de.rs-online.com/web/content/discovery-portal/produktatgeber/drucktaster-leitfaden> (besucht am 09.05.2026).
- [Smi11] A. G. Smith. *Introduction to Arduino: A Piece of Cake!* Self-published, 2011. ISBN: 978-1463698348.
- [TR14] H.-R. Tränkler und L. Reindl, Hrsg. *Sensortechnik: Handbuch für Praxis und Wissenschaft*. 2. Aufl. VDI-Buch. Published: 13 January 2015. eBook Packages: Computer Science and Engineering (German Language). Berlin, Heidelberg: Springer Vieweg Berlin, Heidelberg, 2014, S. XIV, 1596. ISBN: 978-3-642-29942-1. DOI: [10.1007/978-3-642-29942-1](https://doi.org/10.1007/978-3-642-29942-1).
- [TSG23] U. Tietze, C. Schenk und E. Gamm. *Halbleiter-Schaltungstechnik*. Springer Vieweg, 2023. ISBN: 978-3-662-68591-4.
- [Zac22] F. Zach. *Leistungselektronik. Ein Handbuch Band 1 / Band 2*. 6. Aufl. Springer Vieweg, 2022. ISBN: 978-3-658-31435-4. DOI: <https://doi.org/10.1007/978-3-658-31436-1>.

- [Zie25] D. Zielke. *Mikrosysteme*. Bd. 1. Springer, 5. März 2025. ISBN: 978-3-662-71232-0.

